

RTL Design Sherpa

RAPIDS Beats Hardware Architecture Specification 0.12

July 2, 2026

Table of Contents

1 Rapids Beats Has Index....27

2 Document Information....28

2.1 Purpose....28

2.2 Scope....28

2.3 Audience....28

2.4 Document Conventions....29

2.4.1 Requirement Levels....29

2.4.2 Signal Directions....29

2.4.3 Timing Diagrams....29

2.5 References....29

3 Revision History....31

3.1 Change Summary....33

3.1.1 Version 0.8 (2026-07-13)....33

3.1.2 Version 0.7 (2026-07-13)....34

3.1.3 Version 0.6 (2026-07-02)....34

3.1.4 Version 1.0 (2026-01-17)....34

4 Acronyms and Definitions....36

4.1 Acronyms....36

4.2 Definitions....36

4.3 Signal Naming Conventions....37

5 Product Overview....38

5.1 Introduction....38

5.2 System Context....38
5.2.1 Figure 1.1.1: RAPIDS Beats in Its System....38
5.3 Target Applications....38
5.4 Architecture Highlights....39
5.4.1 Phase 1 “Beats” Simplifications....39
5.4.2 Key Capabilities....39
6 Key Features....40
6.1 Feature Summary....40
6.1.1 Multi-Channel Architecture....40
6.1.2 Descriptor-Based Control....40
6.1.3 High-Performance Data Paths....40
6.1.4 Monitoring and Debug....40
6.2 Feature Details....41
6.2.1 SRAM Buffering....42
6.2.2 AXI Burst Optimization....42
6.2.3 Error Handling....43
7 System Context....44
7.1 Integration Overview....44
7.2 System Block Diagram....44
7.2.1 Figure 1.3.1: System Integration....44
7.3 Interface Connections....44
7.3.1 Memory Interface....44
7.3.2 Network Interface....45
7.3.3 Configuration Interface....46

7.4 Clock and Reset..... 46
7.4.1 Clock Domains..... 46
7.4.2 Reset Requirements..... 46
7.5 Power Considerations..... 47
7.5.1 Clock Gating..... 47
7.5.2 Power States..... 47
8 Block Diagram..... 48
8.1 Top-Level Architecture..... 48
8.1.1 Figure 2.1.1: RAPIDS Beats Whole Unit..... 48
8.2 Top-Level Integration (rapids_beats_top, rapids_beats_cxs_top)..... 48
8.2.1 Figure 2.1.2: rapids_beats_top / rapids_beats_cxs_top Integration..... 49
8.3 Component Summary..... 49
8.3.1 Scheduler Group Array..... 49
8.3.2 Sink Data Path..... 50
8.3.3 Source Data Path..... 50
8.4 Hierarchy..... 50
8.5 Internal Buses..... 51
9 Data Flow..... 53
9.1 Overview..... 53
9.2 Sink Path Data Flow..... 53
9.3 Concurrent Operation..... 55
9.3.1 Deadlock Prevention..... 56
10 Channel Architecture..... 57
10.1 Multi-Channel Design..... 57

10.2 Channel Resource Allocation....57
10.2.1 Figure 2.3.1: Per-Channel and Shared Resources....57
10.3 Per-Channel Resources....57
10.4 Shared Resources....58
10.5 Channel Scheduling....58
10.5.1 Arbitration Policy....58
10.5.2 Figure 2.3.2: Channel Arbitration onto a Shared AXI Port....58
10.6 Channel State Machine....59
10.7 Channel Isolation....61
10.7.1 Per-Channel Error Handling....61
11 Interface Summary....63
11.1 External Interface Overview....63
11.1.1 Figure 3.1.1: External Interfaces....63
11.2 Interface Port Summary....63
11.3 Clock and Reset....64
11.4 Interface Feature Matrix....64
11.5 Signal Count Summary....65
12 AXI4 Interface Specification....67
12.1 Overview....67
12.2 Descriptor AXI Master....67
12.2.1 Purpose....67
12.2.2 Configuration....67
12.2.3 Signal List....67
12.3 Sink AXI Master (Write)....69

12.3.1 Purpose..... 69
12.3.2 Configuration..... 69
12.3.3 Signal List..... 69
12.4 Source AXI Master (Read)..... 71
12.4.1 Purpose..... 71
12.4.2 Configuration..... 71
12.4.3 Signal List..... 72
12.5 AXI Protocol Constraints..... 73
12.5.1 Address Alignment..... 73
12.5.2 4KB Boundary Handling..... 73
12.5.3 Response Handling..... 74
13 AXIS Interface Specification..... 75
13.1 Overview..... 75
13.2 Sink AXIS Slave..... 75
13.2.1 Purpose..... 75
13.2.2 Configuration..... 75
13.2.3 Signal List..... 75
13.2.4 Backpressure Behavior..... 76
13.3 Source AXIS Master..... 77
13.3.1 Purpose..... 77
13.3.2 Configuration..... 77
13.3.3 Signal List..... 77
13.3.4 Data Availability..... 78
13.4 AXIS Protocol Notes..... 78

13.4.1 Handshake Rules....78
13.4.2 TLAST Semantics....79
13.4.3 TKEEP Usage....79
13.4.4 Channel Multiplexing via TID....79
14 APB Configuration Interface....80
14.1 Overview....80
14.2 APB Slave and Address Routing....80
14.2.1 APB Slave Signal List....81
14.2.2 Descriptor Kick-Off....81
14.3 Status Interface....82
14.3.1 Per-Channel Status Signals....82
14.3.2 Channel State Encoding....82
14.4 Configuration Registers....82
14.4.1 Address Range Validation....82
14.4.2 Configuration Registers....83
14.5 Programming Sequence....83
14.5.1 Initialization....83
14.5.2 Descriptor Kick-Off....84
15 Monitor Bus Interface....85
15.1 Overview....85
15.2 MonBus Packet Format....85
15.2.1 64-bit Packet Structure....85
15.2.2 Signal List....85
15.3 Packet Types....86

15.4 Agent IDs....86
15.5 Event Codes....87
15.5.1 Scheduler Events (Agent 0x30-0x37)....87
15.5.2 Descriptor Engine Events (Agent 0x10-0x17)....88
15.6 Event Data Encoding....88
15.6.1 Completion Event Data....88
15.6.2 Error Event Data....88
15.6.3 State Transition Event Data....88
15.7 MonBus Arbitration....88
15.7.1 Figure 3.5.1: MonBus Arbitration Tree....89
15.7.2 Arbitration Policy....89
15.8 Top-Level MonBus Delivery (monbus_axil4_axil4_group)....89
15.9 Integration Notes....90
15.9.1 Downstream Connection....90
15.9.2 Packet Rate....90
16 CXS Network Interface....91
16.1 Overview....91
16.1.1 Figure 3.6.1: Network Planes....91
16.2 Link Profile....91
16.3 Credits....92
16.4 Signal List....93
16.5 Flow Control and the Sink Contract....95
16.6 Monitoring....95
16.7 Routing and Payload Type (Proposed)....95

17 Use Case: Network to Memory (Sink Path)....96
17.1 Overview....96
17.2 Buffer Management....97
17.2.1 SRAM Buffering Strategy....97
17.2.2 Figure 4.1.1: Sink Channel Buffer....97
17.2.3 Flow Control....98
17.3 Performance Considerations....98
17.3.1 Throughput....98
17.3.2 Latency....98
17.4 Error Handling....99
17.4.1 Error Sources....99
18 Use Case: Memory to Network (Source Path)....102
18.1 Overview....102
18.2 Buffer Management....103
18.2.1 SRAM Buffering Strategy....103
18.2.2 Figure 4.2.1: Source Channel Buffer....103
18.2.3 Flow Control....104
18.3 Concurrent Operation....104
18.3.1 Read-Ahead Pipeline....104
18.4 Performance Considerations....105
18.4.1 Throughput....105
18.4.2 Latency....105
18.5 Error Handling....105
18.5.1 Error Sources....105

19 Use Case: Descriptor Chaining....107
19.1 Overview....107
19.2 Chain Structure....107
19.2.1 Figure 4.3.1: Descriptor Chain Structure....107
19.3 Descriptor Fields for Chaining....109
19.3.1 Relevant Fields....109
19.3.2 Chain Termination....109
19.4 Prefetch Optimization....111
19.4.1 Descriptor Prefetching....111
19.5 Multi-Channel Chaining....111
19.5.1 Figure 4.3.2: Independent Per-Channel Chains....112
19.6 Error During Chain....112
19.6.1 Error Handling....112
19.6.2 Recovery....113
20 Use Case: Multi-Channel Operation....114
20.1 Overview....114
20.2 Channel Independence....114
20.2.1 Figure 4.4.1: Independent Channels....114
20.3 Resource Arbitration....115
20.3.1 Descriptor Port Arbitration....115
20.3.2 Sink/Source Arbitration....115
20.3.3 Figure 4.4.2: Data-Path AXI Arbitration....115
20.4 Channel Isolation....116
20.4.1 Error Isolation....116

20.4.2 Per-Channel Soft Reset....116
20.5 Use Case Examples....116
20.5.1 Mixed Traffic Pattern....116
20.5.2 Bidirectional Transfer....117
20.5.3 Figure 4.4.3: Bidirectional Transfer....117
20.5.4 Load Balancing....117
20.5.5 Figure 4.4.4: Load Balancing by tid....118
20.6 Performance Considerations....118
20.6.1 Bandwidth Sharing....118
20.6.2 Latency Impact....119
20.7 MonBus Events....119
21 Descriptor Format....120
21.1 Overview....120
21.2 Descriptor Layout....120
21.2.1 256-Bit Structure....120
21.3 Field Descriptions....122
21.3.1 Control Bits [195:192]....122
21.3.2 Direction....123
21.3.3 Transfer Length [159:128]....123
21.3.4 Next Pointer [191:160]....123
21.3.5 Address Fields [127:0]....123
21.4 Descriptor Opcodes (DATA / CTRL_READ / CTRL_WRITE)....123
21.5 Extended Descriptors (row/col-major addressing)....124
21.6 Alignment Requirements....126

21.7 Descriptor Examples....127
21.7.1 Sink Descriptor (Network to Memory)....127
21.7.2 Source Descriptor (Memory to Network)....127
21.8 Software Construction....128
21.8.1 C Structure Example....128
21.8.2 Descriptor Ring Setup....128
22 Register Map....130
22.1 Overview....130
22.2 Address Space Layout....130
22.3 Base Registers (0x100-0x3FF)....131
22.4 Monitor Registers (+0x800 within each half)....132
22.5 Key Register Fields....133
22.5.1 GLOBAL_CTRL (0x100) - Read/Write....133
22.5.2 SCHED_TIMEOUT_LIMIT (0x208) - Read/Write....134
22.5.3 DESCENG_CONFIG (0x220) - Read/Write....134
22.6 Channel Kick Sequence....135
22.7 Address Decoding....136
23 Initialization Sequence....137
23.1 Overview....137
23.2 Boot Sequence....138
23.2.1 Figure 5.3.1: Bring-Up Sequence....138
23.3 What the Hardware Requires....139
23.4 Detailed Steps....139
23.4.1 Step 1: Verify Hardware Presence....139

23.4.2 Step 2: Optional Reset....140
23.4.3 Step 3: Configure the Engine....140
23.4.4 Step 4: Optional Monitor and Interrupt Egress....141
23.4.5 Step 5: Build Descriptors....141
23.4.6 Step 6: Enable....142
23.5 Starting Transfers....142
23.5.1 Kick a Channel....142
23.6 Completion and Interrupt Handling....143
23.7 Reset and Recovery....144
23.7.1 Per-Channel Reset....144
23.7.2 Global Reset....144
24 Error Handling....145
24.1 Overview....145
24.2 Error Categories....145
24.2.1 Figure 5.4.1: Error Sources....145
24.3 Error Detection....145
24.3.1 AXI Response Errors....145
24.3.2 Descriptor Errors....146
24.3.3 Timeout Detection....147
24.4 Error Reporting....149
24.4.1 Status Registers....149
24.4.2 MonBus Error Events....149
24.4.3 Interrupt Generation....150
24.4.4 Figure 5.4.2: Error Reporting....151

24.5 Error Recovery....152
24.5.1 Recovery Flow....152
24.5.2 Figure 5.4.3: Error Recovery....152
24.5.3 Finding the Faulted Channel....152
24.5.4 AXI Error Recovery....153
24.5.5 Descriptor Error Recovery....153
24.5.6 Timeout Recovery....154
24.6 Error Prevention....154
24.6.1 Best Practices....154
24.6.2 Descriptor Validation Example....154
25 Throughput Targets....156
25.1 Theoretical Maximum....156
25.1.1 Per-Interface Bandwidth....156
25.1.2 Aggregate Bandwidth....156
25.2 Practical Throughput....157
25.2.1 Single-Channel Performance....157
25.2.2 Multi-Channel Performance....157
25.3 Throughput Limiting Factors....157
25.3.1 Memory / Network System Factors....157
25.3.2 RAPIDS Internal Factors....158
25.4 Performance Targets....159
25.4.1 Design Targets....159
25.4.2 Verification Criteria....159
25.5 On-Silicon Validation....159

26 Latency Characteristics....161
26.1 Latency Components....161
26.1.1 Kick-to-First-Beat Breakdown....161
26.1.2 Total Latency Formula....162
26.2 First-Beat Latency....162
26.3 Control-Descriptor Latency....162
26.4 Latency Variability....163
26.4.1 Sources of Variability....163
26.4.2 Worst-Case Latency....163
26.5 Latency Optimization....163
26.5.1 Software Recommendations....163
26.5.2 Hardware Configuration....163
27 Resource Estimates....165
27.1 FPGA Resource Summary....165
27.2 Resource Breakdown by Block....165
27.2.1 APB Configuration Slave + Register Block....165
27.2.2 Scheduler Group (per direction)....166
27.2.3 Control Engines (per direction)....166
27.2.4 AXI Read / Write Engines....166
27.2.5 SRAM Buffers (sink + source)....166
27.2.6 MonBus / AXI Monitors (USE_AXI_MONITORS)....166
27.3 SRAM Sizing....167
27.4 Scaling Notes....167
28 RAPIDS Beats Architecture MAS....168

28.1 Document Organization....168
28.1.1 Front Matter....168
28.1.2 Chapter 1: Overview....168
28.1.3 Chapter 2: FUB (Functional Unit Blocks)....168
28.1.4 Chapter 3: Macro (Integration Blocks)....168
28.1.5 Chapter 4: Interfaces....169
28.2 Quick Reference....169
28.2.1 FUB Modules (fub_beats/)....169
28.2.2 Macro Modules (macro_beats/)....170
28.3 Architecture Comparison: RAPIDS vs STREAM....171
28.4 Clock and Reset Summary....172
28.4.1 Clock Domains....172
28.4.2 Reset Signals....172
28.5 Interface Summary....173
28.5.1 External Interfaces....173
28.5.2 Internal Buses....173
28.6 Related Documentation....173
28.7 Specification Conventions....174
28.7.1 Signal Naming....174
28.7.2 Parameter Naming....174
28.7.3 Figure and Table Numbering....174
29 Product Requirements Document (PRD)....175
29.1 RAPIDS - Rapid AXI Programmable In-band Descriptor System....175
29.2 1. Executive Summary....175

29.2.1 1.1 Quick Stats....175
29.2.2 1.2 Subsystem Goals....176
29.3 2. Documentation Structure....176
29.3.1 Complete RAPIDS Specification....176
29.3.2 Known Issues....177
29.3.3 Other Documentation....177
29.4 2.4 Organizational Standards - RAPIDS Code Location....177
29.4.1 Code Organization Principle....177
29.4.2 RAPIDS Directory Structure....177
29.4.3 What Goes Where?....178
29.4.4 Import Pattern for RAPIDS Tests....178
29.4.5 Benefits of This Organization....179
29.4.6 Compliance Status....179
29.5 3. Architecture Overview....180
29.5.1 3.1 Top-Level Block Diagram....180
29.5.2 3.2 Data Flow....180
29.6 4. Key Features....181
29.6.1 4.1 Descriptor-Based Operation....181
29.6.2 4.2 Data Path Features....181
29.6.3 4.3 Scheduler Features....181
29.6.4 4.4 Monitoring Integration....182
29.7 5. Interfaces....182
29.7.1 5.1 External Interfaces....182
29.7.2 5.2 Configuration Parameters....183

29.8 6. Use Cases....183
29.8.1 6.1 DMA-Style Transfers....183
29.8.2 6.2 Network Packet Processing....183
29.8.3 6.3 Custom Data Path Acceleration....183
29.9 7. Test Coverage....184
29.9.1 7.1 Current Status....184
29.9.2 7.2 Test Strategy....185
29.10 8. Known Issues Summary....185
29.10.1 8.1 Critical Issues....185
29.10.2 8.2 Medium Priority Issues....186
29.11 9. Integration Guidelines....186
29.11.1 9.1 Quick Start....186
29.11.2 9.2 Configuration Steps....186
29.12 10. Development Status....187
29.12.1 10.1 Current Phase....187
29.12.2 10.2 Roadmap....187
29.13 11. Performance Characteristics....188
29.13.1 11.1 Throughput....188
29.13.2 11.2 Latency....188
29.13.3 11.3 Resource Utilization....188
29.14 12. Verification Infrastructure....188
29.14.1 12.1 Test Organization....188
29.14.2 12.2 CocoTB Framework....188
29.14.3 12.2.1 MANDATORY: BFM Usage for FUB Tests....189

29.14.4 12.3 Test File Structure (Standard Pattern)....190
29.15 13. Quick Reference....195
29.15.1 13.1 Key Files....195
29.15.2 13.2 Commands....195
29.16 14. Success Criteria....196
29.16.1 14.1 Functional....196
29.16.2 14.2 Quality....196
29.16.3 14.3 Documentation....196
29.17 15. Educational Value....196
29.18 16. Documentation Generation....197
29.18.1 16.1 Generating PDF/DOCX from Specification....197
29.19 16.2 PDF Generation Location....198
29.20 17. References....198
29.20.1 16.1 Internal Documentation....198
29.20.2 16.2 Related Subsystems....199
29.20.3 16.3 External References....199
29.21 Navigation....199
30 Address Increment Patterns for DMA and Descriptor-Based Data Movement200
30.1 Table of Contents....200
30.2 1. Executive Summary....200
30.2.1 Pattern Categories....201
30.3 2. RAPIDS Current Implementation....201
30.3.1 2.1 Descriptor Format (256-bit)....201
30.3.2 2.2 Current Address Increment Modes....201

30.3.3 2.3 Alignment Information Structure....202
30.3.4 2.4 AXI Translation....202
30.4 3. AXI Protocol Burst Modes....203
30.4.1 3.1 FIXED Burst (AxBURST = 2'b00)....203
30.4.2 3.2 INCREMENT Burst (AxBURST = 2'b01)....203
30.4.3 3.3 WRAP Burst (AxBURST = 2'b10)....203
30.5 4. Linear Address Increment Patterns....204
30.5.1 4.1 Simple Contiguous....204
30.5.2 4.2 Unaligned Start/End....204
30.5.3 4.3 Size-Constrained Bursts....204
30.6 5. 2D Transfer Patterns (Stride-Based)....205
30.6.1 5.1 Concept....205
30.6.2 5.2 Descriptor Fields (Extended)....205
30.6.3 5.3 Address Calculation....205
30.6.4 5.4 Use Cases....205
30.6.5 5.5 Analog Devices AXI DMAC Implementation....206
30.7 6. 3D Transfer Patterns....206
30.7.1 6.1 Concept....206
30.7.2 6.2 Descriptor Fields (Extended)....206
30.7.3 6.3 Address Calculation....207
30.7.4 6.4 Use Cases....207
30.8 7. Scatter-Gather Patterns....207
30.8.1 7.1 Concept....207
30.8.2 7.2 SG Descriptor Fields....207

30.8.3 7.3 Operation Modes....208
30.8.4 7.4 AMD/Xilinx AXI DMA SG Descriptor Format....208
30.8.5 7.5 Use Cases....208
30.9 8. Circular and Ring Buffer Patterns....209
30.9.1 8.1 Concept....209
30.9.2 8.2 Descriptor Fields (Extended)....209
30.9.3 8.3 Address Calculation....209
30.9.4 8.4 Ping-Pong Double Buffering....209
30.9.5 8.5 Use Cases....209
30.10 9. Tensor Accelerator Memory Access Patterns (PRIORITY 0)....210
30.10.1 9.1 Systolic Array Dataflow Fundamentals....210
30.10.2 9.2 Data Cube Model (NVDLA Reference)....211
30.10.3 9.3 Tensor Memory Accelerator (TMA) - NVIDIA Hopper....212
30.10.4 9.4 Convolution Address Patterns....212
30.10.5 9.5 Gemmini Accelerator Architecture (UC Berkeley Reference)....214
30.10.6 9.6 Tiling for Large Matrices....215
30.10.7 9.7 Sparse Tensor Patterns....216
30.10.8 9.8 Transformer/Attention Patterns....217
30.10.9 9.9 Programmable Address Generation Unit (PAGU)....218
30.11 10. Implementation Comparison Matrix....219
30.11.1 10.1 Pattern Complexity vs. Capability....219
30.11.2 10.2 Use Case Mapping....220
30.11.3 10.3 RAPIDS Current vs. Extended Capability....220
30.12 11. RAPIDS Extension Recommendations....220

30.12.1 11.0 PRIORITY 0: Tensor Accelerator Addressing (CRITICAL).....220
30.12.2 11.1 Priority 1: 2D Stride Support.....226
30.12.3 11.2 Priority 2: Circular Buffer Mode.....226
30.12.4 11.3 Priority 3: Enhanced Scatter-Gather.....227
30.13 12. References.....228
30.13.1 Industry Documentation.....228
30.13.2 GPU and Accelerator Patterns.....228
30.13.3 DMA Architecture.....228
30.13.4 Neural Network Accelerators.....228
30.13.5 Convolution and Tensor Operations.....228
30.13.6 Sparse Tensor Formats.....228
30.14 Document History.....229

List of Figures

Figure 1.1.1: RAPIDS Beats in Its System....37
Figure 1.3.1: System Integration....43
Figure 2.1.1: RAPIDS Beats Whole Unit....47
Figure 2.1.2: rapids_beats_top / rapids_beats_cxs_top Integration....48
Figure 2.3.1: Per-Channel and Shared Resources....56
Figure 2.3.2: Channel Arbitration onto a Shared AXI Port....57
Figure 3.1.1: External Interfaces....62
Figure 3.5.1: MonBus Arbitration Tree....87
Figure 3.6.1: Network Planes....89
Figure 4.1.1: Sink Channel Buffer....95
Figure 4.2.1: Source Channel Buffer....101
Figure 4.3.1: Descriptor Chain Structure....105
Figure 4.3.2: Independent Per-Channel Chains....110
Figure 4.4.1: Independent Channels....112
Figure 4.4.2: Data-Path AXI Arbitration....113
Figure 4.4.3: Bidirectional Transfer....115
Figure 4.4.4: Load Balancing by tid....116
Figure 5.3.1: Bring-Up Sequence....135
Figure 5.4.1: Error Sources....142
Figure 5.4.2: Error Reporting....147
Figure 5.4.3: Error Recovery....148

List of Tables

Table 1: Target Audience....28
Table 2: Reference Documents....29
Table 3: Document Revision History....31
Table 4: Acronym Definitions....35
Table 5: Term Definitions....35
Table 6: Signal Naming Conventions....36
Table 7: Target Applications....37
Table 8: Phase 1 Simplifications....38
Table 9: Data Path Summary....39
Table 10: SRAM Buffer Parameters....41
Table 11: AXI Optimization Features....41
Table 12: Error Handling Summary....42
Table 13: AXI Memory Interface Requirements....43
Table 14: AXIS Network Interface....44
Table 15: Configuration Methods....45
Table 16: Clock Domains....45
Table 17: Reset Signals....45
Table 18: Power States....46
Table 19: Scheduler Array Components....48
Table 20: Sink Path Components....49
Table 21: Source Path Components....49
Table 22: Internal Bus Summary....50
Table 23: Resource Isolation....55
Table 24: Per-Channel Resources....56
Table 25: Shared Resources....57
Table 26: Channel Isolation Features....60
Table 27: External Interface Summary....62
Table 28: Clock and Reset Signals....63
Table 29: Interface Feature Matrix....63
Table 30: Signal Count Summary....64
Table 31: Descriptor AXI Configuration....65
Table 32: Descriptor AXI Signals....65
Table 33: Sink AXI Configuration....67
Table 34: Sink AXI Signals....67
Table 35: Source AXI Configuration....69
Table 36: Source AXI Signals....70

Table 37: Address Alignment Requirements....71
Table 38: AXI Response Handling....72
Table 39: Sink AXIS Configuration....73
Table 40: Sink AXIS Signals....73
Table 41: Source AXIS Configuration....75
Table 42: Source AXIS Signals....75
Table 43: TLAST Usage....77
Table 44: TKEEP Examples....77
Table 45: APB Address Routing....78
Table 46: APB Slave Signals....79
Table 47: Status Signals....80
Table 48: Scheduler State Encoding....80
Table 49: Key Configuration Registers....81
Table 50: MonBus Signals....83
Table 51: MonBus Packet Types....83
Table 52: RAPIDS Agent IDs....84
Table 53: Scheduler Event Codes....85
Table 54: Descriptor Engine Event Codes....85
Table 55: Top-Level MonBus AXI-Lite Group....87
Table 56: MonBus Packet Rates....88
Table 57: CXS Link Profile....90
Table 58: CXS Top Parameters....91
Table 59: CXS Link Signals....91
Table 60: CXS Status Signals....92
Table 61: Sink Path Flow Control....96
Table 62: Sink Path Throughput Factors....96
Table 63: Sink Path Latency Breakdown....96
Table 64: Sink Path Error Handling....97
Table 65: Source Path Flow Control....102
Table 66: Source Path Throughput Factors....103
Table 67: Source Path Latency Breakdown....103
Table 68: Source Path Error Handling....103
Table 69: Chaining-Related Descriptor Fields....107
Table 70: Mixed Traffic Channel Assignment....114
Table 71: Bandwidth Distribution....116
Table 72: Arbitration Latency....117
Table 73: Proposed Descriptor Fields....118
Table 74: Control Bits....120

Table 75: Address Alignment Requirements....123
Table 76: Base Registers....128
Table 77: Monitor Registers (+0x800 within each half; SRC at 0x0800, SNK at 0x1800)....129
Table 78: GLOBAL_CTRL Register....130
Table 79: SCHED_TIMEOUT_LIMIT Register....131
Table 80: DESCENG_CONFIG Register....131
Table 81: Conditions for a Channel to Run....136
Table 82: AXI Response Codes....142
Table 83: Descriptor Checks....143
Table 84: Timeout Mechanisms....144
Table 85: Error-Related Status Registers....145
Table 86: MonBus Error Packets....146
Table 87: Error Prevention Best Practices....150
Table 88: FUB Module Summary....164
Table 89: Macro Module Summary....165
Table 90: RAPIDS Beats vs STREAM Comparison....166
Table 91: Clock Domains....167
Table 92: Reset Signals....167
Table 93: External Interfaces....168
Table 94: Internal Buses....168

1 Rapids Beats Has Index

Generated: 2026-10-16

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

2 Document Information

Document Title: RAPIDS Beats Hardware Architecture Specification (HAS) **Document Number:** RAPIDS-HAS-001 **Version:** 0.9 **Date:** 2026-09-21 **Status:** Released

2.1 Purpose

This Hardware Architecture Specification (HAS) defines the external interfaces, system integration requirements, and high-level architecture of the RAPIDS Beats accelerator. It is intended for system architects and integration engineers who need to understand RAPIDS Beats from an external perspective without requiring knowledge of internal implementation details.

2.2 Scope

This document covers:

- External interface specifications (AXI4, AXIS, APB, MonBus)
- System-level block diagrams and data flows
- Programming model and descriptor format
- Use cases and operational sequences
- Performance characteristics and constraints

This document does NOT cover:

- Internal module architecture (see MAS)
- RTL implementation details (see MAS)
- Verification test plans
- Physical design constraints

2.3 Audience

Target Audience

Role	Relevance
System Architect	Primary - system integration
Hardware Integration Engineer	Primary - interface connection
Software Engineer	Primary - driver development
Verification Engineer	Reference - interface protocols
RTL Designer	Reference - external constraints

2.4 Document Conventions

2.4.1 Requirement Levels

Term	Meaning
SHALL	Mandatory requirement
SHOULD	Recommended but not mandatory
MAY	Optional feature

2.4.2 Signal Directions

- **Input:** Signal driven by external logic into RAPIDS
- **Output:** Signal driven by RAPIDS to external logic
- **Bidirectional:** Signal can be driven by either side (rare)

2.4.3 Timing Diagrams

All timing diagrams use WaveDrom format with the following conventions:

- `p` = Positive clock edge
- `n` = Negative clock edge
- `0/1` = Logic low/high
- `x` = Unknown/don't care
- `=` = Data value (with label)
- `.` = Previous value continues

2.5 References

Reference Documents

Document	Description
RAPIDS Beats MAS	Module Architecture Specification
ARM AMBA AXI4 Specification	AXI4 protocol reference
ARM AMBA AXI-Stream Specification	AXIS protocol reference
Arm AMBA CXS Protocol Specification (IHI 0079, Issue D)	CXS protocol reference (CXS tops)
RAPIDS PRD	Product Requirements Document

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

3 Revision History

Document Revision History

Version	Date	Author	Changes
1.0	2026-01-17	RTL Design Sherpa	Initial release
0.6	2026-07-02	RTL Design Sherpa	Resynced to RTL top-level integration
0.7	2026-07-13	RTL Design Sherpa	Added the control-descriptor feature (CTRL_READ consumer gate / CTRL_WRITE producer doorbell)
0.8	2026-07-13	RTL Design Sherpa	Added Chapter 6: Performance (throughput targets, latency characteristics, measured resource estimates)
0.9	2026-09-21	RTL Design Sherpa	Removed a vestigial diagram-asset layer: 19 image references and 35 `**Source:**` links pointed at `assets/mermaid/` and `assets/wavedrom/` files that were never created, while a live fenced mermaid or wavedrom block sat immediately below each one carrying the same diagram. No diagram content

Version	Date	Author	Changes
			changed – all 31 mermaid and 39 wavedrom fences render as before.
0.10	2026-10-16	RTL Design Sherpa	`SRAM_DEPTH` stated as beats per channel, per direction
0.11	2026-10-16	RTL Design Sherpa	Network planes and CXS: new CXS Network Interface chapter (planes, link profile, credits, signals, monitoring, planned routing / payload-type fields); interface summary, block diagram, hierarchy, system context, key features and MonBus agent table updated for the AXIS / CXS planes; the Top-Level MonBus Delivery section corrected (the data masters carry no monitors); all block diagrams updated and a network-planes figure added
0.12	2026-10-16	RTL Design Sherpa	Routing and payload-type PROPOSAL (not implemented): the tile packet header is

Version	Date	Author	Changes
			defined in the compute-engine tile's TILE_SPEC Section 11; CXS only transports it. Descriptor Format adds the proposed `ptype`, destination-coordinate and target-address fields; the CXS Network Interface chapter points at TILE_SPEC

3.1 Change Summary

3.1.1 Version 0.8 (2026-07-13)

Performance Chapter

- Added Chapter 6: Performance, mirroring the STREAM HAS performance chapter:
  - Section 6.1 Throughput Targets – per-interface and aggregate bandwidth (512-bit @ 100 MHz = 6.4 GB/s per direction, 12.8 GB/s full-duplex with concurrent sink + source), practical single/multi-channel efficiency, limiting factors, design targets, and on-silicon golden-CRC validation (48/48 configs).
  - Section 6.2 Latency Characteristics – kick-to-first-beat breakdown, control-descriptor latency (`CTRL_READ` retry budget, `CTRL_WRITE` doorbell), variability, and worst-case bounds.
  - Section 6.3 Resource Estimates – measured post-implementation utilization from the Nexys A7-100T characterization build (37,555 LUT / 28,683 FF / 22 BRAM / 0 DSP at NUM_CHANNELS=4, timing met at 100 MHz), per-block breakdown, and scaling notes.

3.1.2 Version 0.7 (2026-07-13)

Control-Descriptor Feature

- Documented the control-descriptor opcodes in Descriptor Format (Section 5.1): `CTRL_READ` (consumer gate – poll a memory location until `(read & mask) == expected`) and `CTRL_WRITE` (producer doorbell – single 32-bit write), for in-memory producer/consumer synchronization without moving payload. Added the per-opcode field layouts.
- Added `CTRL_CONFIG` @ 0x240 (`CTRLRD_MAX_TRY[8:0]`, the control-read poll retry budget) to the register map (Section 5.2) and a Key Features entry.

3.1.3 Version 0.6 (2026-07-02)

RTL Resync

- Documented the `rapids_beats_top` integration: single APB slave routed to descriptor kick-off (0x000-0x03F) and the `rapids_regs` register block (base 0x100-0x3FF plus monitor regfile @ 0x1000).
- Refreshed the register map with the actual PeakRDL-generated addresses, including the monitor regfile at 0x1000 and `SCHED_TIMEOUT_LIMIT` at 0x208.
- Added the MonBus AXI-Lite group (error-drain slave `s_axil_err_*`, capture master `m_axil_mon_*`, `mon_irq`) and the USE_AXI_MONITORS-gated rd/wr AXI monitors to the block diagram, APB, and MonBus interface chapters.
- Reflected scheduler recoverable write-progress timeout + B-response commit gating and functional descriptor prefetch.

3.1.4 Version 1.0 (2026-01-17)

Initial Release

- Complete HAS for RAPIDS Beats Phase 1 architecture
- External interface specifications (AXI4, AXIS, APB, MonBus)
- System block diagrams with Mermaid
- Timing diagrams with WaveDrom
- Programming model and descriptor format
- Use case documentation

4 Acronyms and Definitions

4.1 Acronyms

Acronym Definitions

Acronym	Definition
AXI	Advanced eXtensible Interface
AXIS	AXI-Stream
APB	Advanced Peripheral Bus
DMA	Direct Memory Access
FIFO	First-In First-Out
FSM	Finite State Machine
HAS	Hardware Architecture Specification
MAS	Module Architecture Specification
MonBus	Monitor Bus
RAPIDS	Rapid AXI Programmable In-band Descriptor System
SRAM	Static Random Access Memory

4.2 Definitions

Term Definitions

Term	Definition
Beat	Single data transfer unit (512 bits = 64 bytes in default configuration)
Channel	Independent DMA context with dedicated descriptor processing
Descriptor	256-bit control structure defining a single DMA transfer
Descriptor Chain	Linked list of descriptors for multi-transfer operations
Sink Path	Data flow from network (AXIS slave) to memory

Term	Definition
	(AXI write)
Source Path	Data flow from memory (AXI read) to network (AXIS master)
Fill	Operation writing data into SRAM buffer (sink path input)
Drain	Operation reading data from SRAM buffer (source path output)

4.3 Signal Naming Conventions

Signal Naming Conventions

Prefix/Suffix	Meaning
`m_axi_*`	AXI Master signals
`s_axis_*`	AXIS Slave signals
`m_axis_*`	AXIS Master signals
`cfg_*`	Configuration signals
`*_valid`	Handshake valid signal
`*_ready`	Handshake ready signal
`*_n`	Active-low signal

5 Product Overview

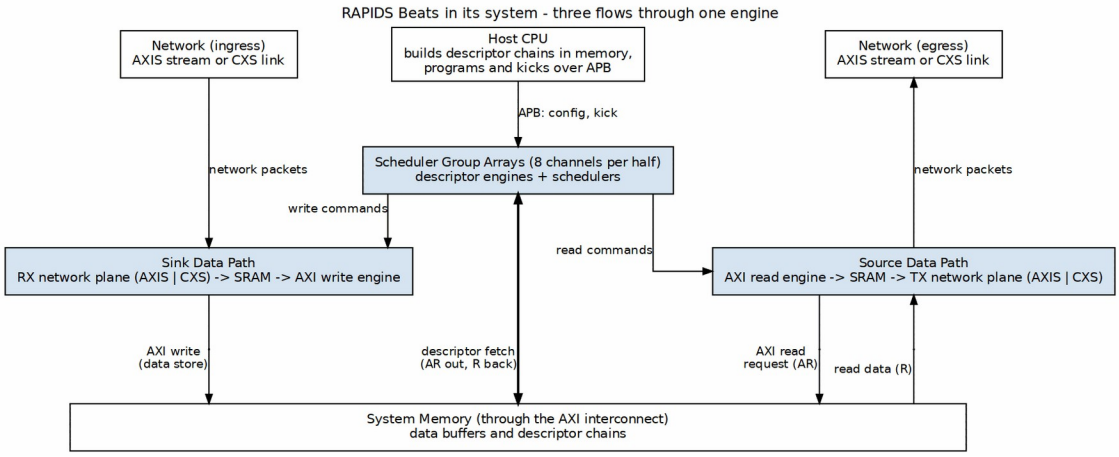
5.1 Introduction

RAPIDS Beats is a high-performance network-to-memory accelerator designed for descriptor-based data movement operations. It bridges network interfaces (AXI-Stream) with system memory (AXI4), enabling efficient DMA transfers without CPU intervention.

The “Beats” architecture represents Phase 1 of the RAPIDS family, optimized for simplicity and throughput in streaming applications.

5.2 System Context

5.2.1 Figure 1.1.1: RAPIDS Beats in Its System



RAPIDS Beats in Its System

Source: ch01_product_context.dot

5.3 Target Applications

Target Applications

Application	Description	Key Requirements
Network Offload	Receive packets from network, store to memory	Low latency, high throughput
Scatter-Gather DMA	Multi-descriptor data movement	Descriptor chaining
Streaming	Front-end for	Continuous data flow

Application	Description	Key Requirements
Accelerator	compute engines	
Protocol Processing	Header/payload separation	Multi-channel isolation

5.4 Architecture Highlights

5.4.1 Phase 1 "Beats" Simplifications

The Beats architecture prioritizes simplicity over feature completeness:

Phase 1 Simplifications

Feature	Beats (Phase 1)	Future Phases
Transfer Unit	Full beats only	Chunk-level alignment
Flow Control	Backpressure-based	Credit management
Address Alignment	Pre-aligned required	Hardware alignment fixup
Network Interface	Direct AXIS, or CXS through a swappable network plane	Protocol conversion inside the core

5.4.2 Key Capabilities

1. **8 Independent Channels** - Concurrent transfers with dedicated resources
2. **Descriptor Chaining** - Automatic next-descriptor fetch
3. **Dual Data Paths** - Simultaneous sink (write) and source (read)
4. **Monitor Integration** - Real-time event reporting via MonBus
5. **Swappable Network Plane** - AXI-Stream or CXS (Arm IHI 0079 Issue D) at the top, same core
6. **Error Detection** - Address range checking, AXI error propagation

6 Key Features

6.1 Feature Summary

6.1.1 Multi-Channel Architecture

- **8 Independent Channels** with dedicated schedulers
- **Per-Channel State Machines** for concurrent operation
- **Shared AXI Infrastructure** with arbitrated access
- **Channel Isolation** prevents cross-channel interference

6.1.2 Descriptor-Based Control

- **256-bit Descriptor Format** with all transfer parameters
- **Automatic Chaining** follows next_descriptor_ptr links
- **Last Descriptor Flag** terminates chain processing
- **IRQ Generation** via gen_irq descriptor flag
- **Control Descriptors** (`CTRL_READ` consumer gate / `CTRL_WRITE` producer doorbell) for in-memory producer/consumer synchronization without moving payload – see Descriptor Format (Section 5.1)

6.1.3 High-Performance Data Paths

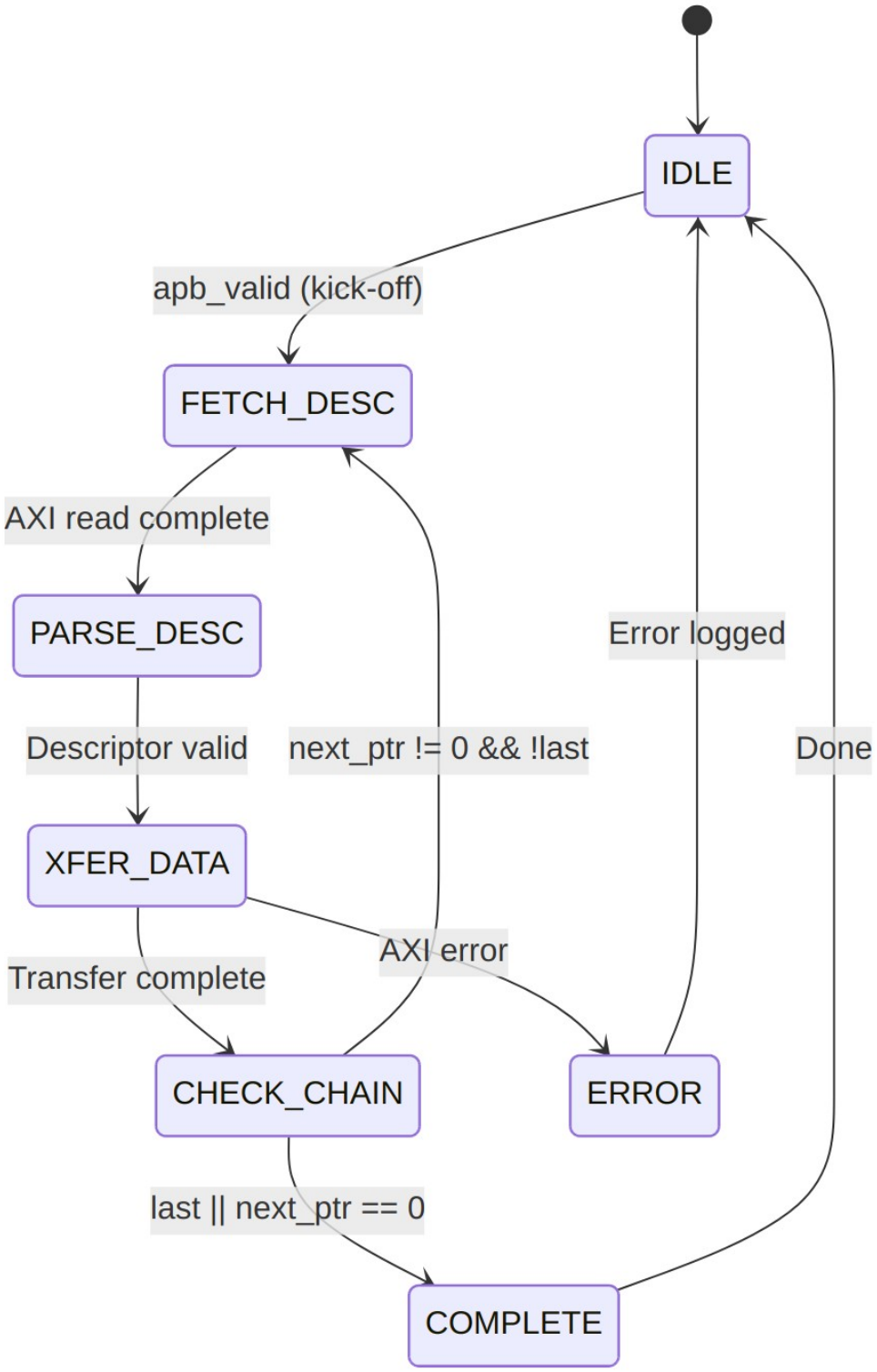
Data Path Summary

Path	Direction	Bandwidth	Features
Sink	Network to Memory	512-bit @ clock	SRAM buffering, AXI burst writes
Source	Memory to Network	512-bit @ clock	AXI burst reads, SRAM buffering

6.1.4 Monitoring and Debug

- **64-bit MonBus Packets** for all significant events
- **State Transition Reports** track FSM activity
- **Error Event Codes** for fault diagnosis
- **Performance Metrics** for throughput analysis
- **Network Monitor-Lite** in each network plane (AXIS or CXS), agents 0x09 / 0x0A

6.2 Feature Details



Descriptor Processing

6.2.1 SRAM Buffering

Each channel has its own buffer in each data path (sink and source), `SRAM_DEPTH` beats deep:

SRAM Buffer Parameters

Parameter	Default	Range	Description
SRAM_DEPTH	4096	64-4096	Beats per channel, per direction (parameter of `rapids_beats_top`)
Data Width	512	256 / 512	Bits per beat (`DATA_WIDTH`)
Buffer size	256 KB	2 KB-256 KB	Per channel, per direction: `SRAM_DEPTH x DATA_WIDTH/8`

The defaults are a simulation size. The Genesys 2 builds use 128 beats of 32 bytes, 4 KB per channel.

Buffer Operation:

- **Sink Path:** Fill from AXIS (from the RX network plane, AXIS or CXS), drain to AXI write engine
- **Source Path:** Fill from AXI read engine, drain to AXIS (to the TX network plane, AXIS or CXS)
- **Flow Control:** Backpressure when buffer full/empty

6.2.2 AXI Burst Optimization

RAPIDS optimizes AXI transactions for maximum bandwidth:

AXI Optimization Features

Feature	Implementation
Burst Length	Up to 256 beats (AXI4 max)
Outstanding Transactions	Configurable (default 8)
Address Alignment	64-byte aligned (beat size)
4KB Boundary	Automatic burst splitting

6.2.3 Error Handling

Error Handling Summary

Error Type	Detection	Response
Address Range	Pre-fetch validation	Descriptor rejected
AXI SLVERR	Response check	Transfer aborted
AXI DECERR	Response check	Transfer aborted
Timeout	Watchdog timer	Channel reset

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

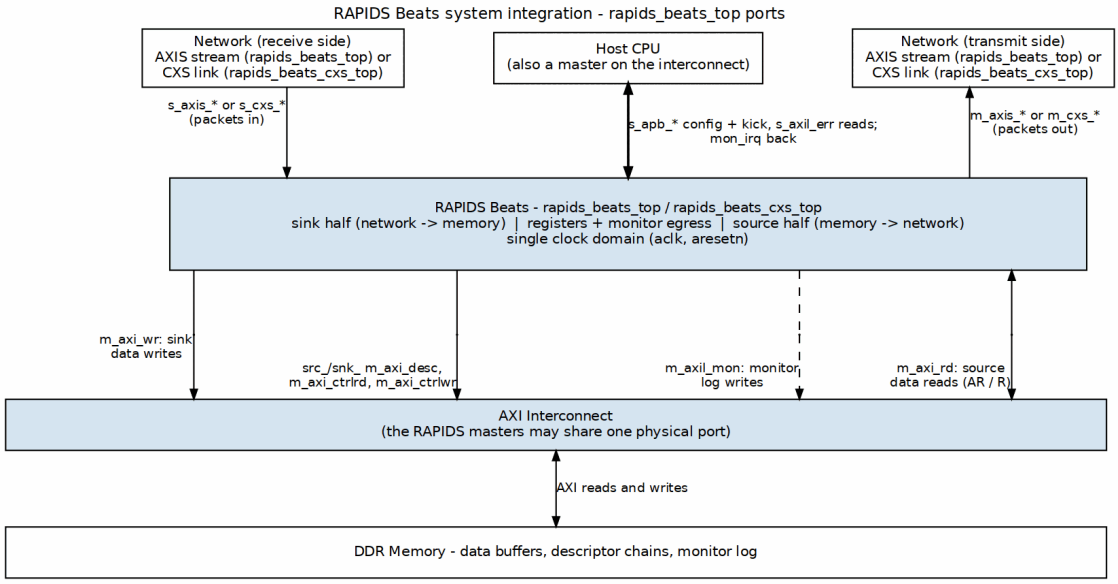
7 System Context

7.1 Integration Overview

RAPIDS Beats integrates into a system as a DMA engine bridging network interfaces and system memory. This section describes the external connections and system-level requirements.

7.2 System Block Diagram

7.2.1 Figure 1.3.1: System Integration



System Integration

Source: ch01_system_integration.dot

7.3 Interface Connections

7.3.1 Memory Interface

RAPIDS Beats reaches system memory through the AXI4 masters below. Each half (`src_`, `snk_`) has its own descriptor master and its own two control masters, so the descriptor and control rows are two ports each. With monitors built, an AXI-Lite master (`m_axil_mon`) also writes the monitor log:

AXI Memory Interface Requirements

Port	Access Pattern	Typical Bandwidth
Descript	Random read (256-bit)	Low (<100 MB/s)

Port	Access Pattern	Typical Bandwidth
or AXI (`m_axi_desc`)		
Control AXI (`m_axi_ctrlrd`, `m_axi_ctrlwr`)	Single 32-bit reads and writes	Very low
Sink AXI	Sequential write (512-bit)	High (up to interface max)
Source AXI	Sequential read (512-bit)	High (up to interface max)

Interconnect Requirements:

- All of these ports MAY share a single physical AXI port with arbitration
- Descriptor port has lowest bandwidth requirement
- Sink and Source ports should have balanced bandwidth allocation

7.3.2 Network Interface

The network side is a swappable plane. The AXIS tops use standard AXI-Stream, below; the CXS tops (`*_cxs_top`) replace the two streams with one CXS link per direction, with `CXSSRCID` carrying the channel (see the CXS Network Interface chapter):

AXIS Network Interface

Interface	Role	Signals
Sink AXIS	Receives data from network	TDATA, TVALID, TREADY, TLAST
Source AXIS	Sends data to network	TDATA, TVALID, TREADY, TLAST

Protocol Notes:

- Beat-granular: the beats design drives every byte valid (`m_axis_tstrb` all ones) and does not use the ingress `s_axis_tstrb`; a transfer is a whole number of beats. Partial beats (packed bytes, a contiguous `tstrb` on the last beat) are the byte-granular RAPIDS in `rtl/top/rapids_top.sv` (rapids TASK-019), which this document does not describe.

- TID/TDEST may be used for channel routing (optional)
- TUSER available for sideband information

7.3.3 Configuration Interface

Software configures RAPIDS through register writes:

Configuration Methods

Method	Interface	Usage
Direct Write	Memory-mapped	Register configuration
Descriptor Kick	APB-style	Start descriptor processing

7.4 Clock and Reset

7.4.1 Clock Domains

Clock Domains

Clock	Frequency	Usage
`aclk`	100-500 MHz	All RAPIDS logic

Note: RAPIDS Beats uses a single clock domain. CDC (clock domain crossing) to different frequency domains must be handled externally.

7.4.2 Reset Requirements

Reset Signals

Signal	Type	Description
`aresetn`	Active-low	Async assert, sync deassert

Reset Sequence:

1. Assert `aresetn` low for minimum 4 clock cycles
2. Ensure all AXI interfaces are idle before reset
3. Release reset synchronously to `aclk` rising edge
4. Wait 16 clock cycles before initiating transfers

7.5 Power Considerations

7.5.1 Clock Gating

RAPIDS supports optional clock gating for power reduction:

- **Idle Detection:** Automatic when no transfers pending
- **Per-Channel Gating:** Individual channels can be clock-gated
- **External Control:** `cfg_clock_enable` override available

7.5.2 Power States

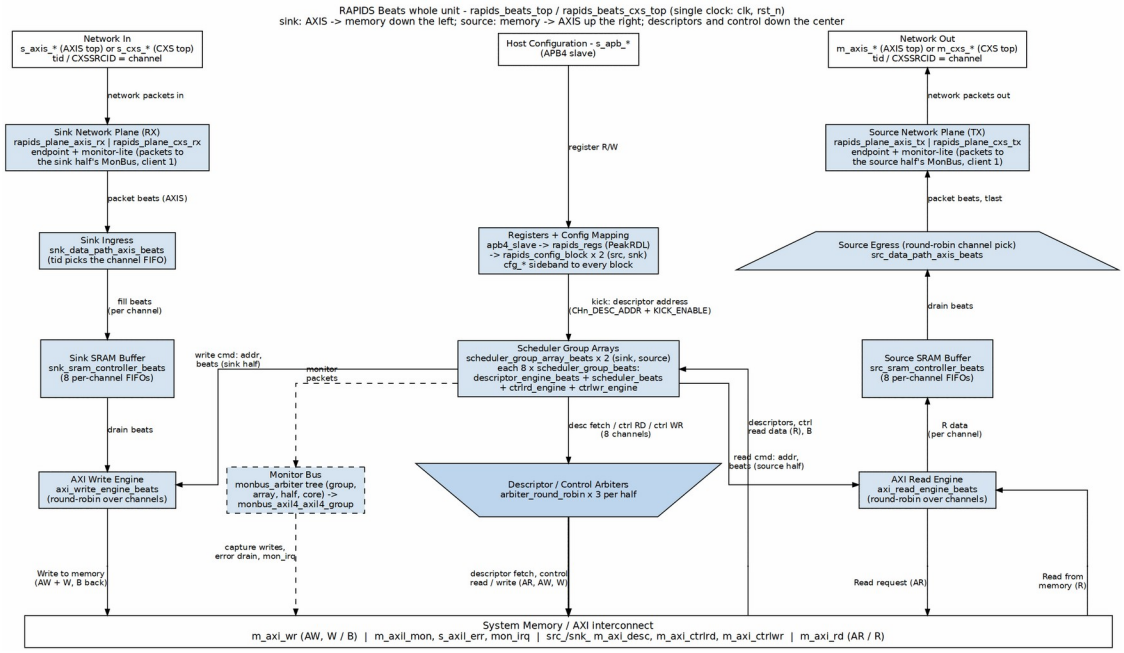
Power States

State	Description	Exit Latency
Active	Normal operation	0 cycles
Idle	No pending transfers	1 cycle
Gated	Clock disabled	4 cycles

8 Block Diagram

8.1 Top-Level Architecture

8.1.1 Figure 2.1.1: RAPIDS Beats Whole Unit



RAPIDS Beats Whole Unit

Source: ch02_whole_unit.dot

8.2 Top-Level Integration (rapids_beats_top, rapids_beats_cxs_top)

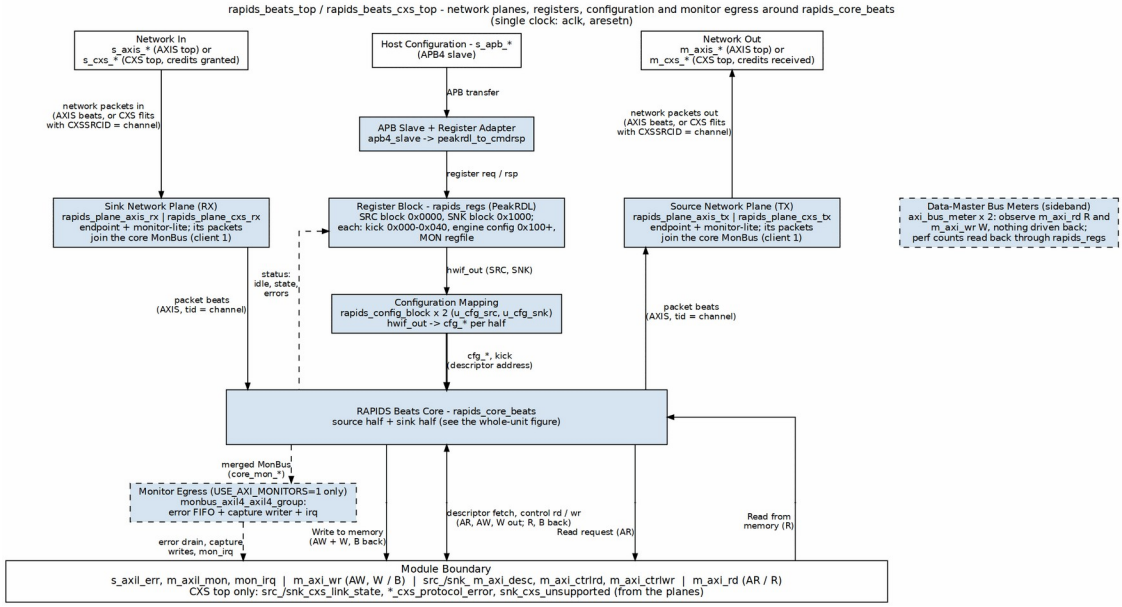
`rapids_core_beats` is wrapped by `rapids_beats_top`, which adds the software register interface, configuration mapping, and monitoring infrastructure. A single APB slave feeds a command/response router into the `rapids_regs` register block (base config at 0x100-0x3FF, monitor regfile at 0x1000); the per-channel kick registers live at 0x000-0x03F inside that same block. `rapids_config_block` translates the register `hwif_out` into the core/monitor `cfg_*` signals.

The network endpoints are not in the core. They sit in two **network planes** between the core and the pins: an RX plane in front of the sink half and a TX plane behind the source half (`rtl/plane/`, rapids TASK-024). The AXIS top (`rapids_beats_top`) uses `rapids_plane_axis_rx` / `rapids_plane_axis_tx` (`axis4_slave_monlite`, `axis4_master_monlite`); the CXS top (`rapids_beats_cxs_top`) uses `rapids_plane_cxs_rx` / `rapids_plane_cxs_tx` (`cxs_slave_monlite` + `cxs_to_axis`, `axis_to_cxs` + `cxs_master_monlite`). Both tops are

generated from `rapids_beats_top_base` by `bin/gen_plane_tops.py`; the CXS link is described in the CXS Network Interface chapter.

When `USE_AXI_MONITORS = 1`, each half's descriptor read master carries an `axi4_master_rd_monlite`, and each plane carries a monitor-lite on its network port (AXIS or CXS; rapids TASK-015). The descriptor-monitor packets join the eight scheduler groups in the array's 9:1 `monbus_arbiter`; the plane's packets enter the core on `snk_plane_mon_*` / `src_plane_mon_*` and join the array output in the half's 3:1 `monbus_arbiter` (client 1; third input reserved), the core merges the two halves, and the top delivers the stream through a `monbus_axil4_axil4_group` to an AXI-Lite error-drain slave, an AXI-Lite capture master, and a `mon_irq` interrupt. The data masters `m_axi_rd` and `m_axi_wr` carry no monitors.

8.2.1 Figure 2.1.2: rapids_beats_top / rapids_beats_cxs_top Integration



rapids_beats_top Integration

Source: ch02_top_integration.dot

8.3 Component Summary

8.3.1 Scheduler Group Array

The scheduler group array manages 8 independent channels:

Scheduler Array Components

Component	Instance Count	Purpose
`scheduler_beats`	8	Transfer coordination per

Component	Instance Count	Purpose
		channel
descriptor_engine_beats	8	Descriptor fetch and parse
Descriptor AXI Arbiter	1	Shared AXI4 for descriptor fetch

8.3.2 Sink Data Path

Network-to-memory data flow:

Sink Path Components

Component	Purpose
snk_sram_controller_beats	Multi-channel SRAM management
axi_write_engine_beats	AXI4 burst write generation
alloc_ctrl_beats	Space allocation tracking
drain_ctrl_beats	Data availability tracking

8.3.3 Source Data Path

Memory-to-network data flow:

Source Path Components

Component	Purpose
axi_read_engine_beats	AXI4 burst read generation
src_sram_controller_beats	Multi-channel SRAM management
alloc_ctrl_beats	Space allocation tracking
drain_ctrl_beats	Data availability tracking

8.4 Hierarchy

```
rapids_beats_top | rapids_beats_cxs_top   (generated by
bin/gen_plane_tops.py)
├── u_base: rapids_beats_top_base   (regs, rapids_config_block x2,
monbus group)
│   └── u_core: rapids_core_beats
├── u_snk_plane: rapids_plane_axis_rx | rapids_plane_cxs_rx
```

```
│   └── axis4_slave_monlite | cxs_slave_monlite + cxs_to_axis
└── u_src_plane: rapids_plane_axis_tx | rapids_plane_cxs_tx
    └── axis4_master_monlite | axis_to_cxs + cxs_master_monlite

rapids_core_beats
├── monbus_arbiter
├── rapids_src_beats
│   ├── scheduler_group_array_beats
│   │   ├── scheduler_group_beats [0..7]
│   │   │   ├── scheduler_beats
│   │   │   ├── descriptor_engine_beats
│   │   │   ├── ctrlrd_engine
│   │   │   └── ctrlwr_engine
│   │   ├── arbiter_round_robin [x3]
│   │   ├── axi4_master_rd_monlite
│   │   └── axi_bus_meter
│   ├── monbus_arbiter (3:1; client 1 = src_plane_mon_*)
│   └── src_data_path_axis_beats
│       └── src_data_path_beats
│           ├── axi_read_engine_beats
│           └── src_sram_controller_beats
│               └── sram_controller (STREAM)
│                   └── sram_controller_unit [0..7]
│                       ├── stream_alloc_ctrl
│                       ├── gaxi_fifo_sync
│                       ├── stream_drain_ctrl
│                       └── stream_latency_bridge
└── rapids_snk_beats
    ├── scheduler_group_array_beats (same shape)
    ├── monbus_arbiter (3:1; client 1 = snk_plane_mon_*)
    └── snk_data_path_axis_beats
        └── snk_data_path_beats
            ├── axi_write_engine_beats
            └── snk_sram_controller_beats
                └── sram_controller (STREAM)
                    └── sram_controller_unit [0..7]
```

8.5 Internal Buses

Internal Bus Summary

Bus	Width	Description
Descriptor Bus	256-bit	Parsed descriptor to scheduler
Scheduler Command	Variable	Transfer parameters to data paths

Bus	Width	Description
SRAM Data	512-bit	Data to/from SRAM buffers
MonBus	64-bit	Monitor packets (aggregated)

9 Data Flow

9.1 Overview

RAPIDS Beats implements two independent data paths that can operate concurrently:

1. **Sink Path:** Network to Memory (AXIS Slave -> AXI Write)
2. **Source Path:** Memory to Network (AXI Read -> AXIS Master)

9.2 Sink Path Data Flow

The sink accepts a beat for channel `tid` only while that channel holds an SRAM reservation. `snk_data_path_axis_beats` drives

```
s_axis_tready = fill_ready && (r_pending_alloc[tid] > 0 || fill_alloc_req)
```

where `r_pending_alloc[tid]` is the number of reserved beats the channel has not yet filled. When the head beat arrives for a channel that holds no reservation, the wrapper raises `fill_alloc_req` in the same cycle. The request is for `AXI_XFER_CONFIG.ALLOC_SIZE` beats, or for whatever remains when less than that is free, and the beat is accepted on that cycle. A channel whose partition is full, or whose last allocation is still settling into `fill_space_free` (three cycles), gets no reservation, and `tready` stays low.

The reservation does not depend on the descriptor. Data can therefore reach a channel's SRAM partition before its descriptor has been fetched, but only up to the space the partition has free. After that the channel stops accepting until the write engine drains. The write engine drains only after the scheduler has a descriptor in `CH_XFER_DATA` and presents `sched_wr_valid`, so a channel that is never kicked fills its partition once and then stalls. `tready` is evaluated for the `tid` of the beat at the head of the single `s_axis` port, so a stalled channel also blocks beats for every other channel behind it (head-of-line blocking). The AXIS monitor-lite's skid stage in front of the wrapper absorbs a few beats before the stall reaches the network.

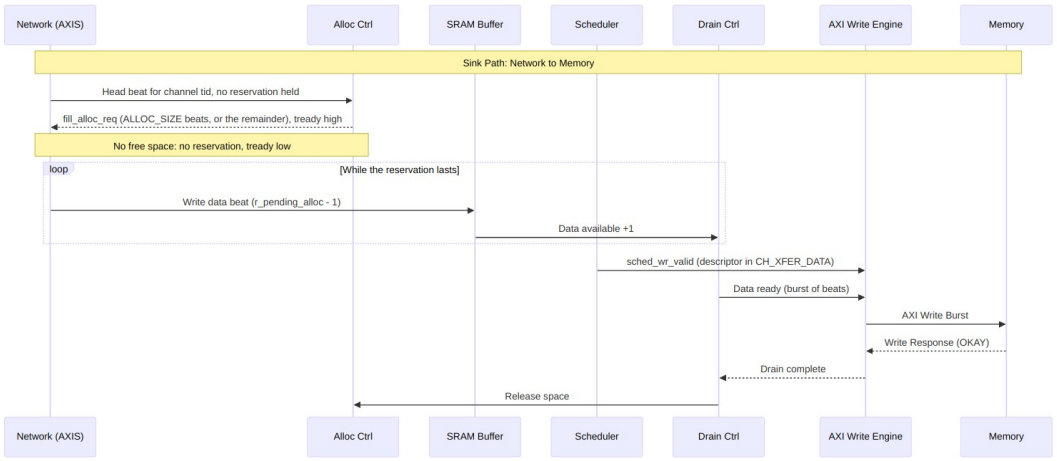
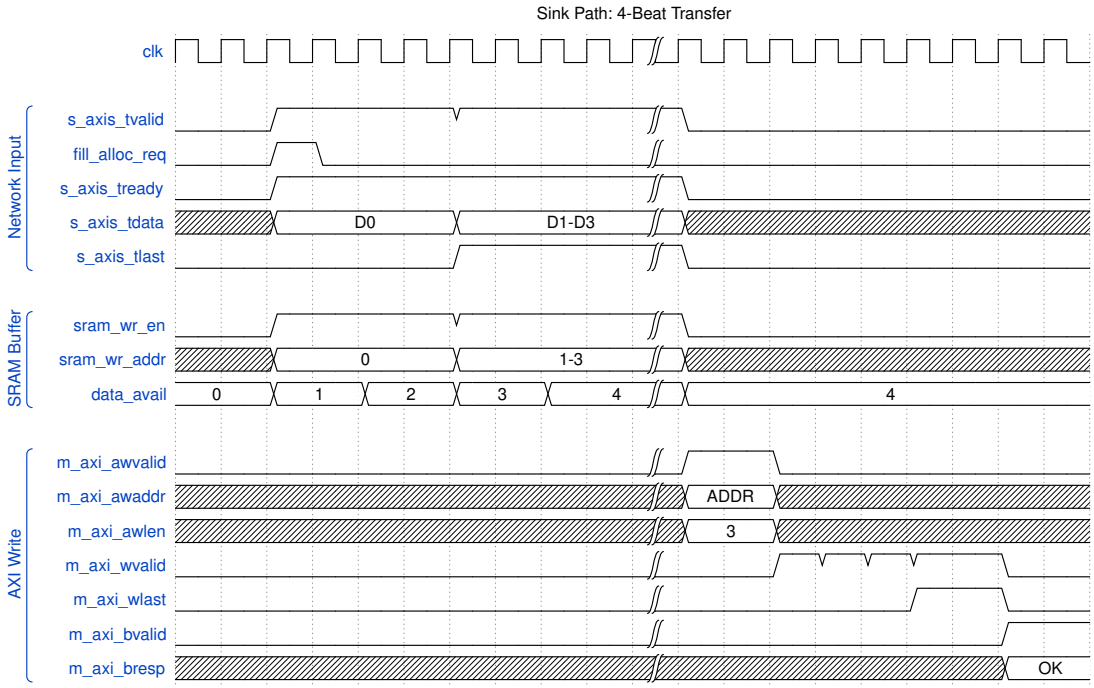
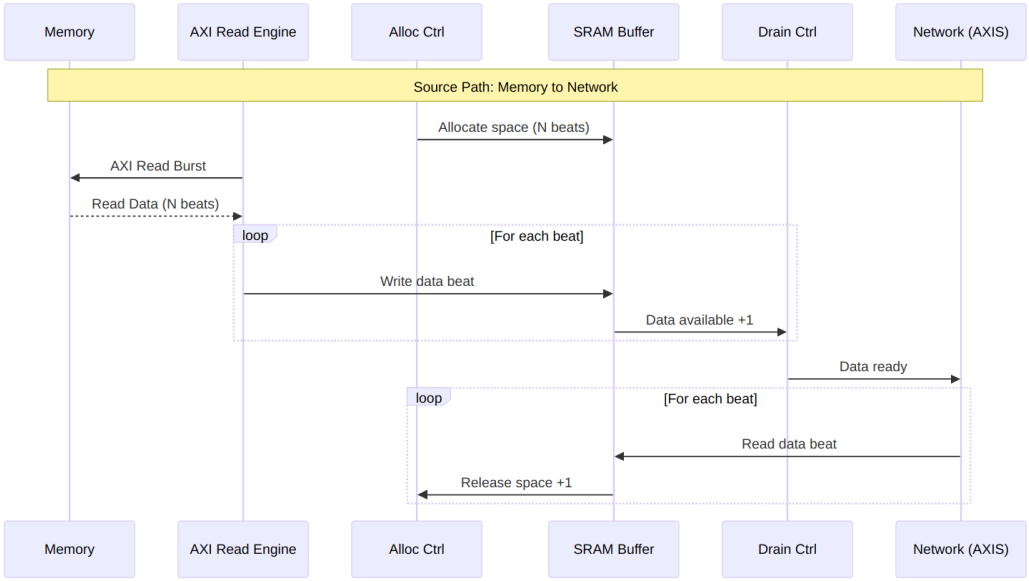


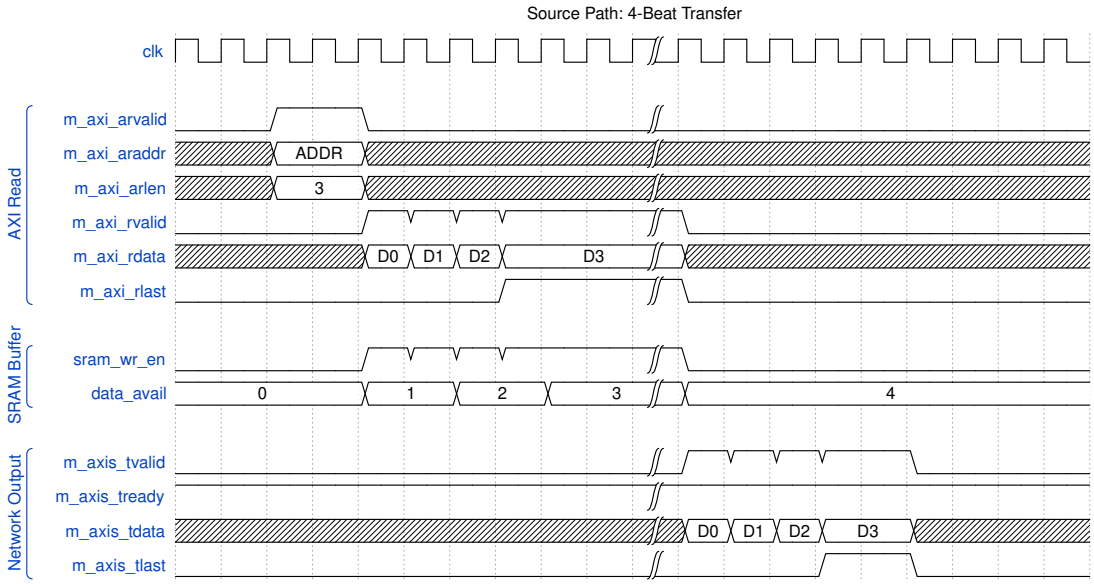
Diagram 2



Sink Path Timing



Source Path Data Flow



Source Path Timing

9.3 Concurrent Operation

Both data paths operate independently and can run simultaneously:

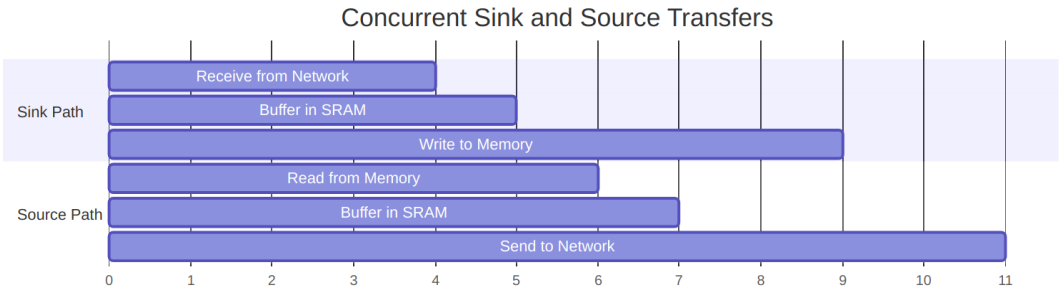


Diagram 4

9.3.1 Deadlock Prevention

RAPIDS prevents deadlock through independent resource allocation:

Resource Isolation

Resource	Sink Path	Source Path	Shared
SRAM Buffer	Dedicated	Dedicated	No
AXI Port	Write only	Read only	No
Scheduler	Per-channel	Per-channel	No

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

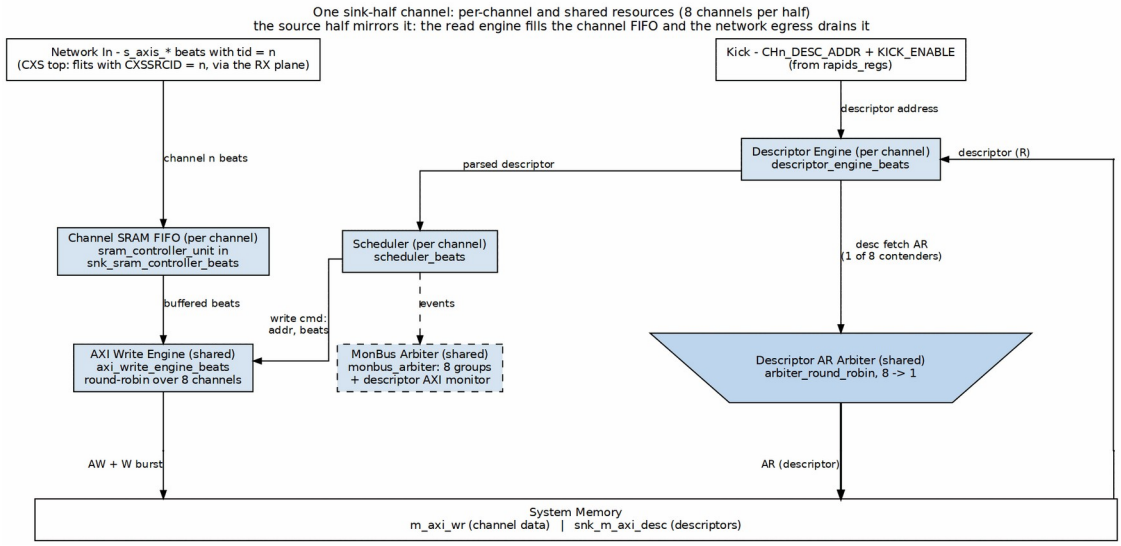
10 Channel Architecture

10.1 Multi-Channel Design

RAPIDS Beats supports 8 independent DMA channels, each with dedicated resources for concurrent operation.

10.2 Channel Resource Allocation

10.2.1 Figure 2.3.1: Per-Channel and Shared Resources



Per-Channel and Shared Resources

Source: ch02_channel_resources.dot

10.3 Per-Channel Resources

Each channel has dedicated:

Per-Channel Resources

Resource	Description	Size/Capacity
Scheduler	Transfer coordination FSM	1 instance
Descriptor Engine	Descriptor fetch/parse	8-deep FIFO
Sink SRAM Unit	Sink buffer, its own `sram_controller`	`SRAM_DEPTH` beats

Resource	Description	Size/Capacity
Source SRAM Unit	Source buffer, its own `sram_controller`	`SRAM_DEPTH` beats
Configuration Registers	Per-channel config	~16 registers

10.4 Shared Resources

Resources shared across all channels:

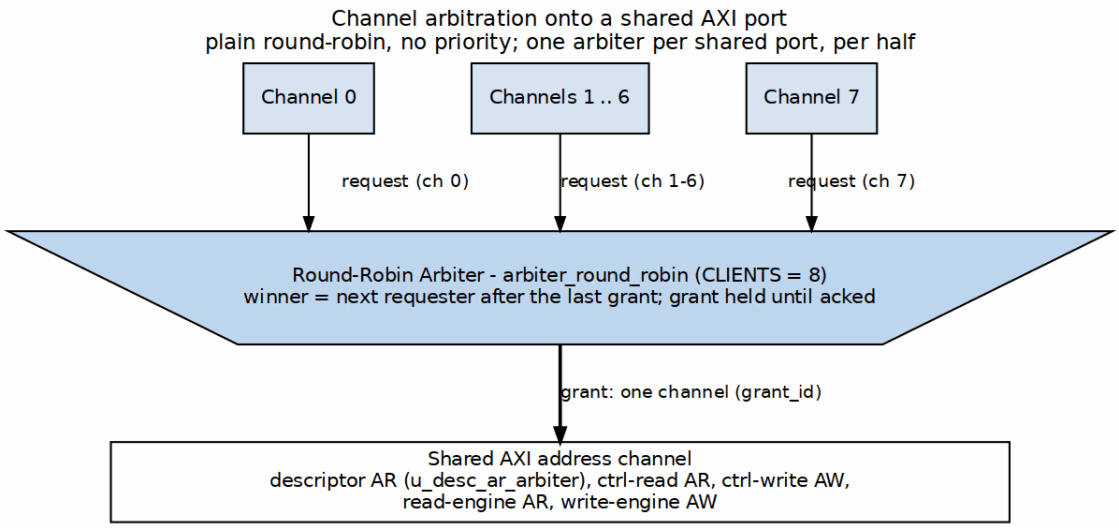
Shared Resources

Resource	Arbitration	Max Outstanding
Descriptor AXI	Round-robin	8 (1 per channel)
Sink AXI Write	Round-robin	8 configurable
Source AXI Read	Round-robin	8 configurable
MonBus Output	Round-robin (`monbus_arbiter` tree)	1 (serialized)

10.5 Channel Scheduling

10.5.1 Arbitration Policy

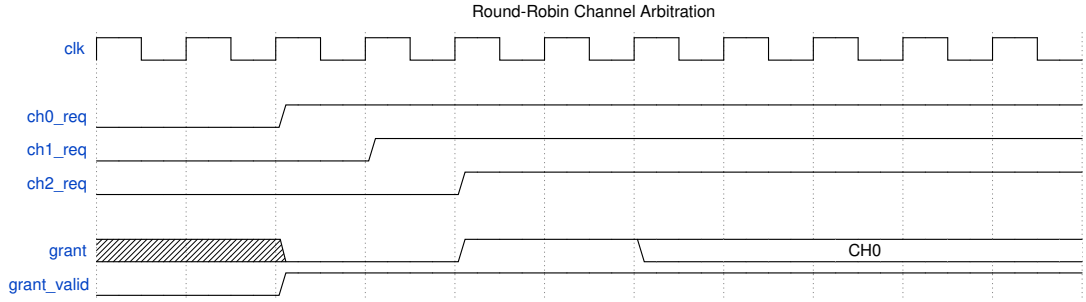
10.5.2 Figure 2.3.2: Channel Arbitration onto a Shared AXI Port



Channel Arbitration onto a Shared AXI Port

Source: ch02_channel_arbitration.dot

Arbitration Timing:



Waveform 3

10.6 Channel State Machine

Each channel scheduler follows this state machine:

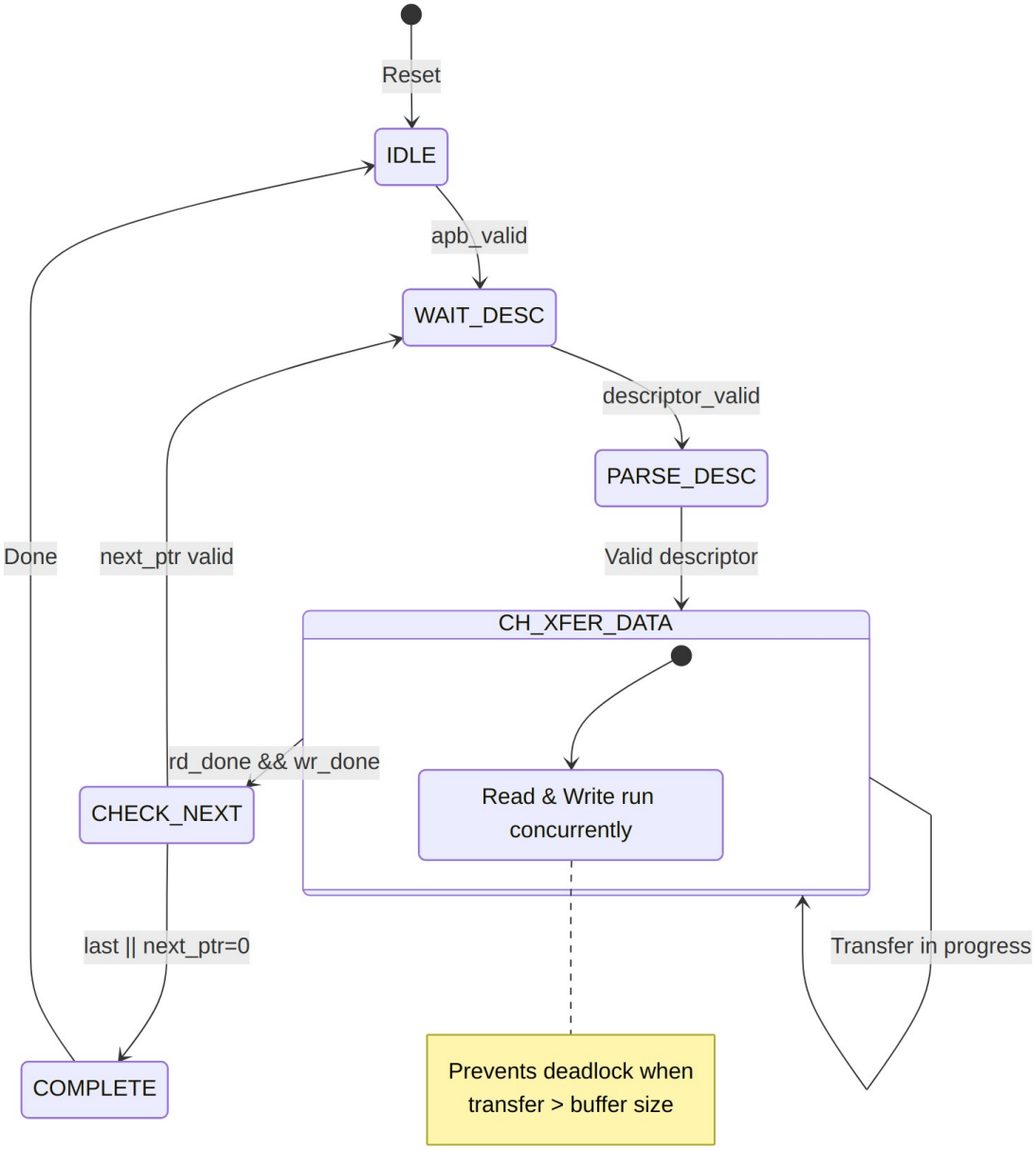
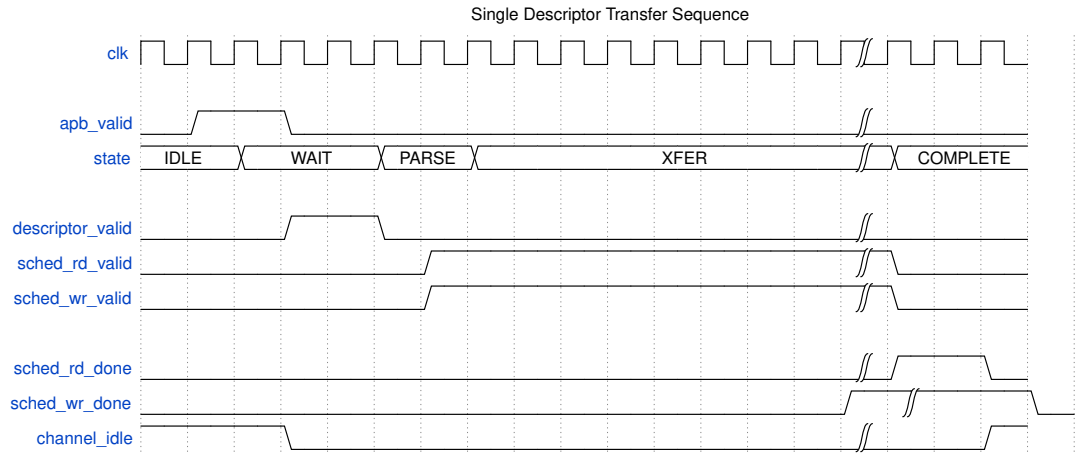

Reset
IDLE
apb_valid
WAIT_DESC
descriptor_valid
PARSE_DESC
Valid descriptor
Done
next_ptr valid
CH_XFER_DATA
rd_done && wr_done
CHECK_NEXT
Read & Write run concurrently
last || next_ptr=0
Transfer in progress
COMPLETE
Prevents deadlock when transfer > buffer size

Diagram 5



Channel State Timing

10.7 Channel Isolation

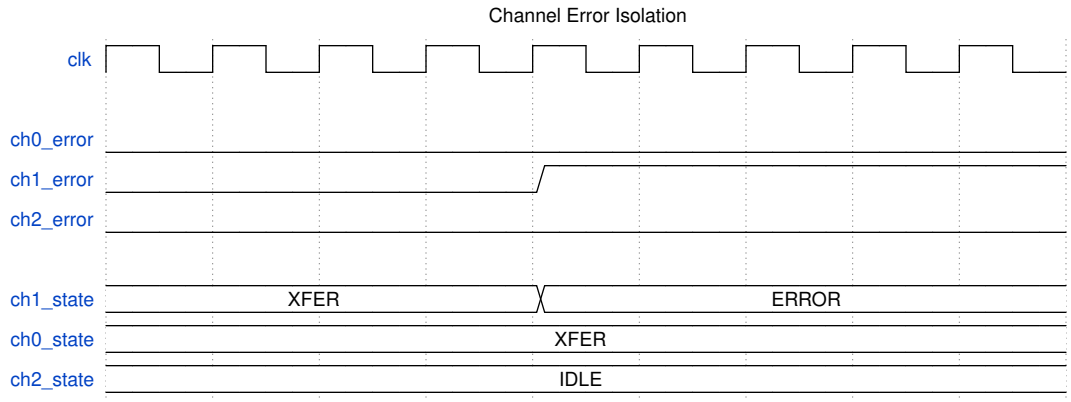
Channels are isolated to prevent interference:

Channel Isolation Features

Isolation Type	Implementation
State Isolation	Separate FSM per channel
Buffer Isolation	Partitioned SRAM per channel
Error Isolation	Per-channel error flags
Reset Isolation	Per-channel soft reset

10.7.1 Per-Channel Error Handling

Each channel maintains independent error state:



Waveform 5

Channel 1 error does not affect Channel 0 or Channel 2 operation.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

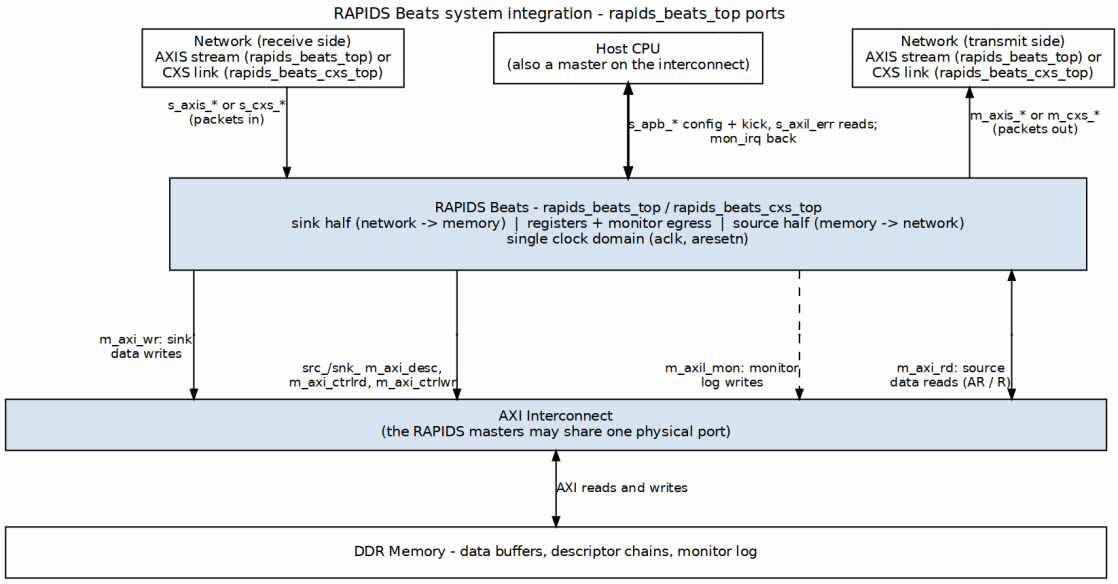
---

11 Interface Summary

11.1 External Interface Overview

RAPIDS Beats exposes the following external interfaces. The network side is a swappable plane: the AXIS tops (`rapids_beats_top`, `rapids_top`) carry the `s_axis_*` / `m_axis_*` streams, and the CXS tops (`rapids_beats_cxs_top`, `rapids_cxs_top`) carry one CXS link per direction in their place (see the CXS Network Interface chapter). Everything else is identical.

11.1.1 Figure 3.1.1: External Interfaces



External Interfaces

Source: ch01_system_integration.dot

11.2 Interface Port Summary

External Interface Summary

Interface	Type	Direction	Data Width	Address Width
Descriptor AXI	AXI4 Master	Read	256-bit	64-bit
Sink AXI	AXI4 Master	Write	512-bit	64-bit
Source AXI	AXI4 Master	Read	512-bit	64-bit

Interface	Type	Direction	Data Width	Address Width
Sink AXIS	AXIS Slave	Input	512-bit	-
Source AXIS	AXIS Master	Output	512-bit	-
Sink CXS (CXS tops only, replaces Sink AXIS)	CXS Receiver	Input	`DATA_WIDTH` (up to 1024-bit)	-
Source CXS (CXS tops only, replaces Source AXIS)	CXS Transmitter	Output	`DATA_WIDTH` (up to 1024-bit)	-
APB Config	APB-like	Input	32-bit	12-bit
MonBus	Custom	Output	64-bit	-

11.3 Clock and Reset

All interfaces operate in the same clock domain:

Clock and Reset Signals

Signal	Type	Description
`aclk`	input	System clock (100-500 MHz typical)
`aresetn`	input	Active-low reset (async assert, sync deassert)

11.4 Interface Feature Matrix

Interface Feature Matrix

Feature	Desc AXI	Sink AXI	Src AXI	Sink AXIS	Src AXIS
Protocol	AXI4	AXI4	AXI4	AXIS	AXIS
Directio	Read	Write	Read	Slave	Master

Feature	Desc AXI	Sink AXI	Src AXI	Sink AXIS	Src AXIS
n					
Burst Support	INCR	INCR	INCR	N/A	N/A
Max Burst	1	256	256	N/A	N/A
Outstanding	8	8	8	N/A	N/A
ID Width	8-bit	8-bit	8-bit	Optional	Optional

11.5 Signal Count Summary

Signal Count Summary

Interface	Input Signals	Output Signals	Total
Descriptor AXI	~20	~30	~50
Sink AXI	~15	~40	~55
Source AXI	~25	~25	~50
Sink AXIS	~20	~5	~25
Source AXIS	~5	~20	~25
Sink CXS (CXS tops)	~8	~7	~15
Source CXS (CXS tops)	~3	~10	~13
APB Config	~15	~5	~20
MonBus	~2	~5	~7
Total	**~102**	**~130**	**~232**

Note: Signal counts are approximate and vary with configuration parameters. The total counts an AXIS top; a CXS top replaces the two AXIS rows with the two CXS rows (including the link-status pins).

12 AXI4 Interface Specification

12.1 Overview

RAPIDS Beats uses three AXI4 master interfaces for memory access:

1. **Descriptor AXI** - Fetches 256-bit descriptors (read-only)
2. **Sink AXI** - Writes data to memory (write-only)
3. **Source AXI** - Reads data from memory (read-only)

12.2 Descriptor AXI Master

12.2.1 Purpose

Fetches descriptor structures from system memory for all 8 channels.

12.2.2 Configuration

Descriptor AXI Configuration

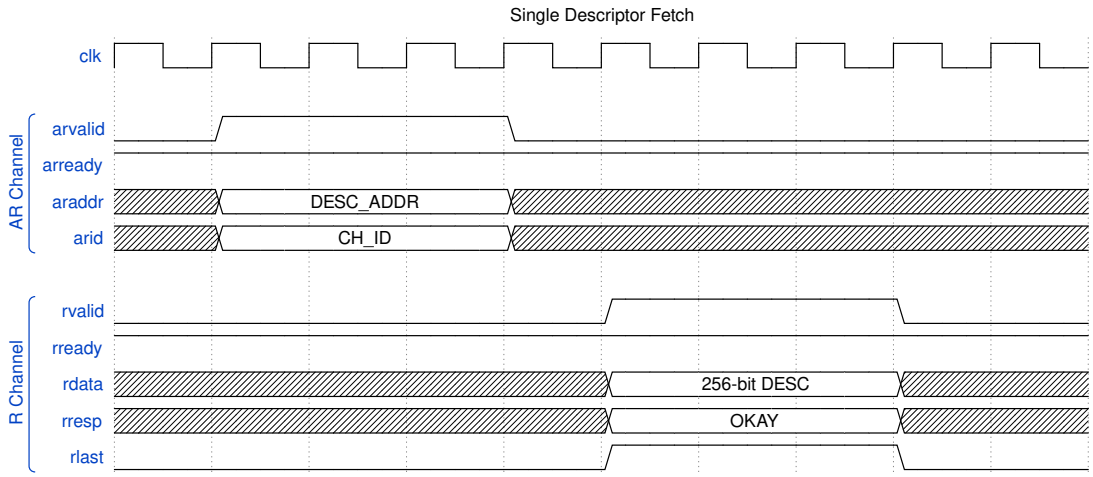
Parameter	Value	Description
Data Width	256-bit	Fixed descriptor size
Address Width	64-bit	Configurable
ID Width	8-bit	Configurable
Burst Length	1	Single-beat only
Burst Type	INCR	Fixed

12.2.3 Signal List

Descriptor AXI Signals

Signal	Width	Direction	Description
`m_axi_desc_aclk`	1	input	Clock
`m_axi_desc_aresetn`	1	input	Reset (active-low)
`m_axi_desc_arid`	8	output	Read ID (channel ID)
`m_axi_desc_araddr`	64	output	Read address
`m_axi_desc_arlen`	8	output	Burst length

Signal	Width	Direction	Description
			(always 0)
m_axi_desc_arsize	3	output	Beat size (5 = 32 bytes)
m_axi_desc_arburst	2	output	Burst type (INCR)
m_axi_desc_arvalid	1	output	Address valid
m_axi_desc_arready	1	input	Address ready
m_axi_desc_rid	8	input	Response ID
m_axi_desc_rdata	256	input	Read data
m_axi_desc_rresp	2	input	Read response
m_axi_desc_rlast	1	input	Last beat
m_axi_desc_rvalid	1	input	Data valid
m_axi_desc_rready	1	output	Data ready



Timing Diagram

12.3 Sink AXI Master (Write)

12.3.1 Purpose

Writes data from SRAM buffer to system memory.

12.3.2 Configuration

Sink AXI Configuration

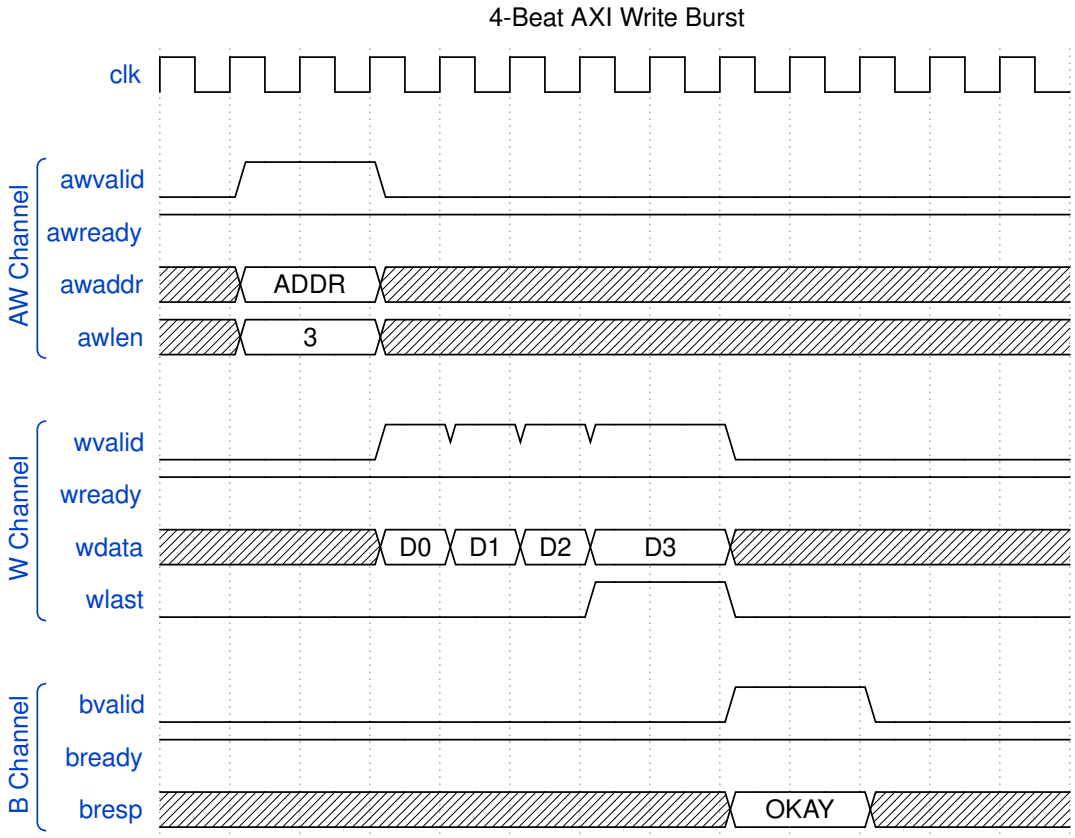
Parameter	Default	Range	Description
Data Width	512-bit	Configurable	Match SRAM width
Address Width	64-bit	Configurable	System address space
ID Width	8-bit	Configurable	Transaction tracking
Max Burst	256	1-256	AXI4 maximum
Outstanding	8	1-16	Concurrent AW requests

12.3.3 Signal List

Sink AXI Signals

Signal	Width	Direction	Description
`m_axi_sink_awid`	8	output	Write ID
`m_axi_sink_awaddr`	64	output	Write address
`m_axi_sink_awlen`	8	output	Burst length - 1
`m_axi_sink_awsize`	3	output	Beat size (6 = 64 bytes)
`m_axi_sink_awburst`	2	output	Burst type (INCR)
`m_axi_sink_awvalid`	1	output	Address valid
`m_axi_sink_awready`	1	input	Address ready

Signal	Width	Direction	Description
m_axi_sink_wdata	512	output	Write data
m_axi_sink_wstrb	64	output	Write strobes
m_axi_sink_wlast	1	output	Last beat
m_axi_sink_wvalid	1	output	Data valid
m_axi_sink_wready	1	input	Data ready
m_axi_sink_bid	8	input	Response ID
m_axi_sink_bresp	2	input	Write response
m_axi_sink_bvalid	1	input	Response valid
m_axi_sink_bready	1	output	Response ready



Timing Diagram

12.4 Source AXI Master (Read)

12.4.1 Purpose

Reads data from system memory to SRAM buffer.

12.4.2 Configuration

Source AXI Configuration

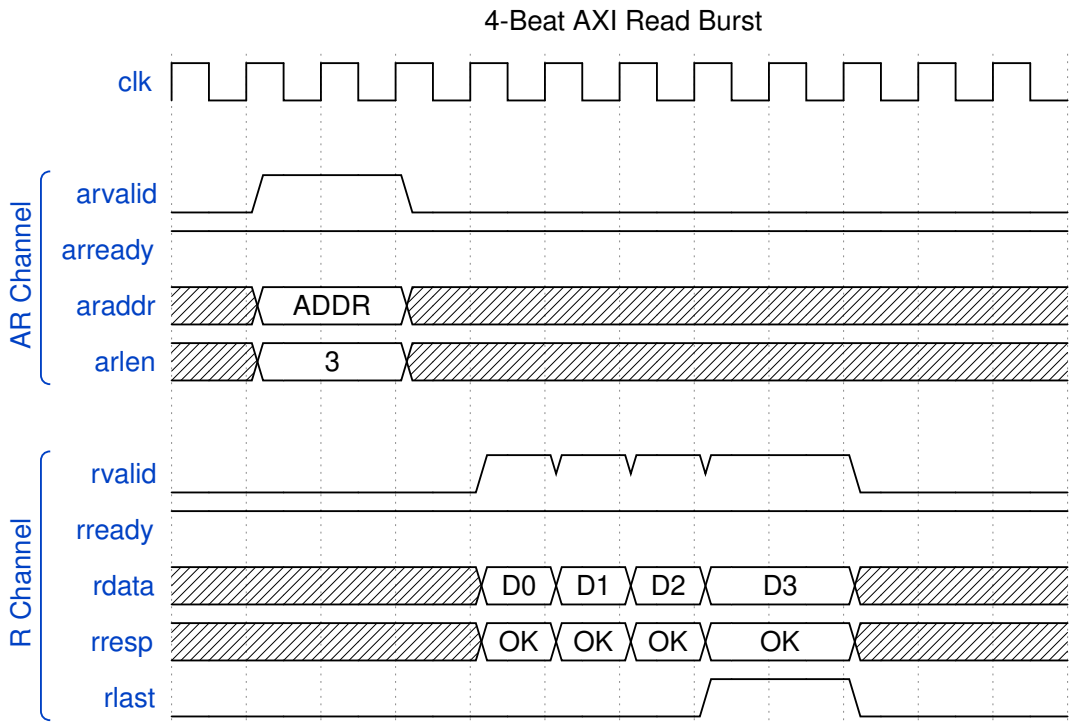
Parameter	Default	Range	Description
Data Width	512-bit	Configurable	Match SRAM width
Address Width	64-bit	Configurable	System address space
ID Width	8-bit	Configurable	Transaction

Parameter	Default	Range	Description
			tracking
Max Burst	256	1-256	AXI4 maximum
Outstanding	8	1-16	Concurrent AR requests

12.4.3 Signal List

Source AXI Signals

Signal	Width	Direction	Description
`m_axi_src_arid`	8	output	Read ID
`m_axi_src_araddr`	64	output	Read address
`m_axi_src_arlen`	8	output	Burst length - 1
`m_axi_src_arsize`	3	output	Beat size (6 = 64 bytes)
`m_axi_src_arburst`	2	output	Burst type (INCR)
`m_axi_src_arvalid`	1	output	Address valid
`m_axi_src_arready`	1	input	Address ready
`m_axi_src_rid`	8	input	Response ID
`m_axi_src_rdata`	512	input	Read data
`m_axi_src_rresp`	2	input	Read response
`m_axi_src_rlast`	1	input	Last beat
`m_axi_src_rvalid`	1	input	Data valid
`m_axi_src_rready`	1	output	Data ready



Timing Diagram

12.5 AXI Protocol Constraints

12.5.1 Address Alignment

Address Alignment Requirements

Transfer Type	Alignment Requirement
Descriptor	32-byte aligned
Sink Data	64-byte aligned
Source Data	64-byte aligned

12.5.2 4KB Boundary Handling

AXI4 requires burst transactions not to cross 4KB boundaries. RAPIDS automatically splits bursts at 4KB boundaries:

```
Example: 8-beat burst starting at 0x0FE0 (64 bytes/beat)
  Beat 0-1: 0x0FE0 - 0x0FFF (to 4KB boundary)
  Beat 2-7: 0x1000 - 0x117F (after boundary)
```

```
Split into two bursts:
  Burst 1: ARLEN=1, ARADDR=0x0FE0
  Burst 2: ARLEN=5, ARADDR=0x1000
```

12.5.3 Response Handling

AXI Response Handling

Response	Code	RAPIDS Behavior
OKAY	2'b00	Normal completion
EXOKAY	2'b01	Treated as OKAY
SLVERR	2'b10	Error, transfer aborted
DECERR	2'b11	Error, transfer aborted

13 AXIS Interface Specification

13.1 Overview

RAPIDS Beats uses AXI-Stream interfaces for network data transfer:

1. **Sink AXIS Slave** - Receives data from network (ingress)
2. **Source AXIS Master** - Sends data to network (egress)

These are the ports of the AXIS tops (`rapids_beats_top`, `rapids_top`). The same AXIS stream, with `tid` = channel, is also what every network plane presents to the core, so the contract below holds inside a CXS top too: the CXS planes convert it to and from CXS flits (see the CXS Network Interface chapter).

13.2 Sink AXIS Slave

13.2.1 Purpose

Receives streaming data from external network interface and writes to SRAM buffer.

13.2.2 Configuration

Sink AXIS Configuration

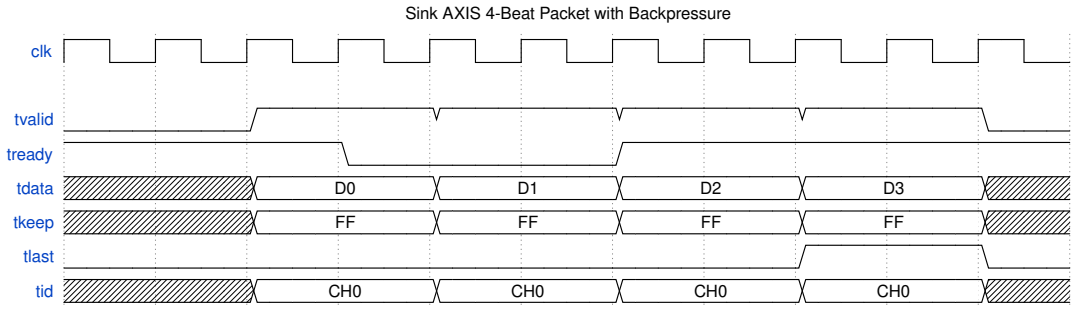
Parameter	Default	Range	Description
Data Width	512-bit	Configurable	TDATA width
ID Width	8-bit	0-8	TID width (optional)
DEST Width	8-bit	0-8	TDEST width (optional)
USER Width	1-bit	0-16	TUSER width (optional)

13.2.3 Signal List

Sink AXIS Signals

Signal	Width	Direction	Description
`s_axis_sink_tdata`	512	input	Data payload
`s_axis_sink_tkeep`	64	input	Byte valid mask
`s_axis_sink_t`	64	input	Byte position

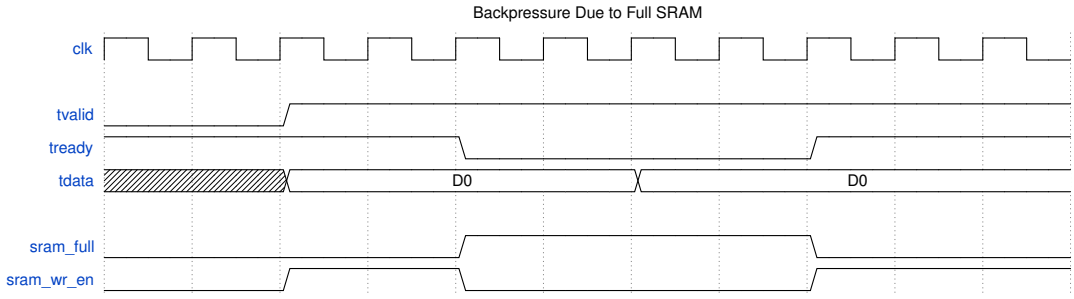
Signal	Width	Direction	Description
`strb`			(optional)
`s_axis_sink_tlast`	1	input	End of packet
`s_axis_sink_tid`	8	input	Transaction ID (channel select)
`s_axis_sink_tdest`	8	input	Destination (optional)
`s_axis_sink_tuser`	1	input	User sideband (optional)
`s_axis_sink_tvalid`	1	input	Data valid
`s_axis_sink_tready`	1	output	Ready to accept



Timing Diagram

13.2.4 Backpressure Behavior

When SRAM buffer is full, `tready` deasserts:



Waveform 10

13.3 Source AXIS Master

13.3.1 Purpose

Sends streaming data from SRAM buffer to external network interface.

13.3.2 Configuration

Source AXIS Configuration

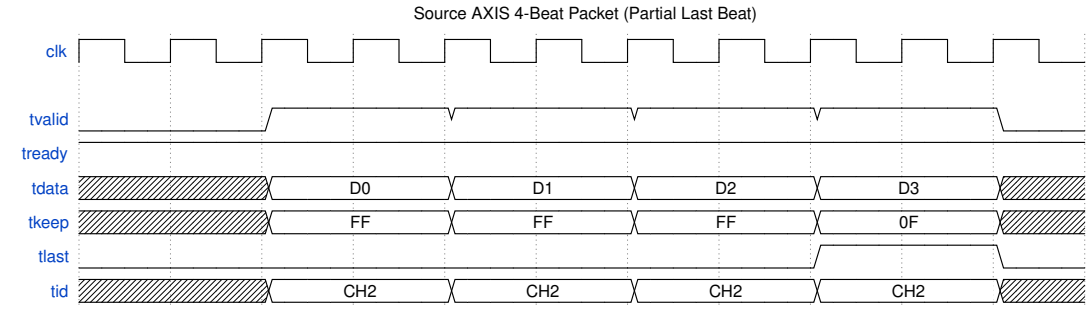
Parameter	Default	Range	Description
Data Width	512-bit	Configurable	TDATA width
ID Width	8-bit	0-8	TID width (optional)
DEST Width	8-bit	0-8	TDEST width (optional)
USER Width	1-bit	0-16	TUSER width (optional)

13.3.3 Signal List

Source AXIS Signals

Signal	Width	Direction	Description
`m_axis_src_tdata`	512	output	Data payload
`m_axis_src_tkeep`	64	output	Byte valid mask
`m_axis_src_tstrb`	64	output	Byte position (optional)
`m_axis_src_tlast`	1	output	End of packet
`m_axis_src_tid`	8	output	Transaction ID (channel)
`m_axis_src_tdest`	8	output	Destination (optional)
`m_axis_src_tuser`	1	output	User sideband (optional)

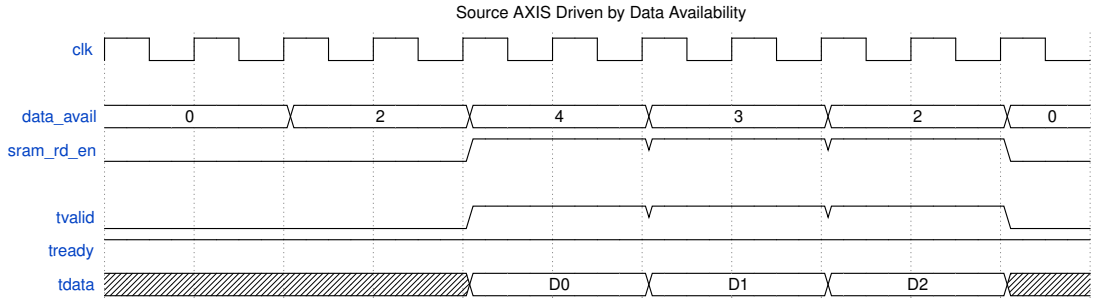
Signal	Width	Direction	Description
`m_axis_src_tvalid`	1	output	Data valid
`m_axis_src_tready`	1	input	Downstream ready



Timing Diagram

13.3.4 Data Availability

Source AXIS only asserts `tvalid` when data is available in SRAM:



Waveform 12

13.4 AXIS Protocol Notes

13.4.1 Handshake Rules

Standard AXI-Stream handshake applies:

1. `tvalid` asserts when data is available
2. `tready` asserts when receiver can accept
3. Transfer occurs on clock edge when both are high
4. `tvalid` SHALL NOT depend on `tready`

5. `tready` MAY depend on `tvalid`

13.4.2 TLAST Semantics

TLAST Usage

Scenario	TLAST Behavior
Descriptor transfer complete	Assert on last beat
Packet boundary	Assert on last beat of packet
Continuous stream	Not used (always 0)

13.4.3 TKEEP Usage

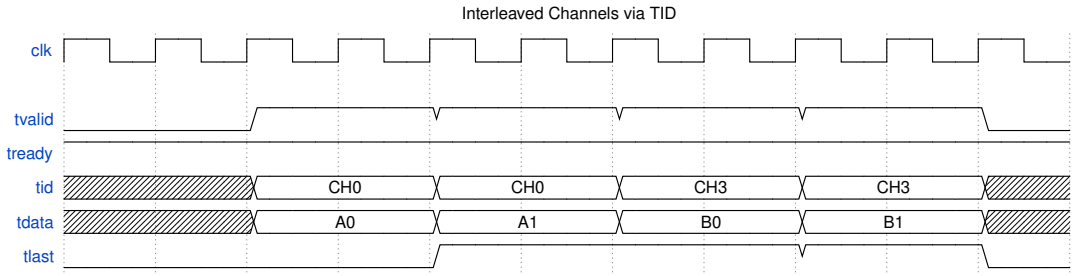
TKEEP indicates valid bytes within each beat:

TKEEP Examples

TKEEP Value	Meaning
64'hFFFF_FFFF_FFFF_FFFF	All 64 bytes valid
64'h0000_FFFF_FFFF_FFFF	Upper 16 bytes invalid
64'h0000_0000_0000_000F	Only lower 4 bytes valid

13.4.4 Channel Multiplexing via TID

Multiple channels can share a single AXIS interface using TID:



Waveform 13

14 APB Configuration Interface

14.1 Overview

At the top level (`rapids_beats_top`), RAPIDS Beats exposes a single **APB4 slave** (`s_apb_*`) for all software access. This interface allows software to:

1. Configure channel and monitor parameters (via the register block)
2. Initiate descriptor processing (kick-off)
3. Read status and error information

14.2 APB Slave and Address Routing

The APB slave is converted to an internal command/response transaction and routed by address to two targets:

APB Address Routing

Address Range	Target	Purpose
0x000-0x03F	`rapids_regs` (staged descriptor addresses)	Per-channel `CH*_DESC_ADDR_{LOW,HIGH}`, stride 0x8
0x040	`rapids_regs` (`KICK_ENABLE`)	Per-channel launch bits (singlepulse)
0x100-0x3FF	`rapids_regs` (base regfile)	Channel/scheduler/descriptor configuration and status
0x1000+	`rapids_regs` (monitor regfile)	AXI-monitor configuration and performance counters

Because the monitor regfile is located at 0x1000, the APB address bus must be at least 13 bits wide to reach it. Configuration that was formerly imagined as discrete `cfg_*` ports is now driven internally by the register block through `rapids_config_block`; see the Register Map for the full address map, and the descriptor address ranges (`DESCENG_ADDR0/1_*` at 0x224-0x230).

14.2.1 APB Slave Signal List

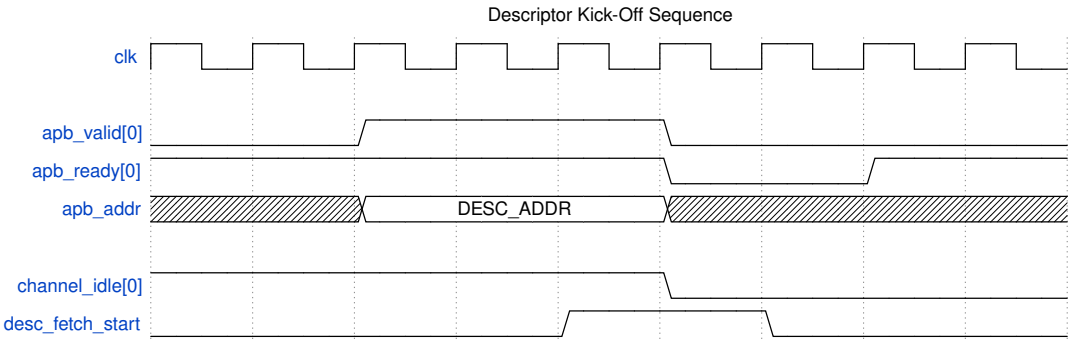
APB Slave Signals

Signal	Width	Direction	Description
`s_apb_paddr`	APB_ADDR_WIDTH (>=13)	input	Address
`s_apb_psel`	1	input	Select
`s_apb_penable`	1	input	Enable phase
`s_apb_pwrite`	1	input	Write / read
`s_apb_pwdata`	32	input	Write data
`s_apb_pstrb`	4	input	Write byte strobes
`s_apb_prdata`	32	output	Read data
`s_apb_pready`	1	output	Ready
`s_apb_pslverr`	1	output	Slave error

14.2.2 Descriptor Kick-Off

To start descriptor processing on a channel:

1. Write descriptor to memory
2. Assert `apb_valid[ch]` with descriptor address on `apb_addr`
3. Wait for `apb_ready[ch]` assertion
4. Deassert `apb_valid[ch]`



Timing Diagram

14.3 Status Interface

14.3.1 Per-Channel Status Signals

Status Signals

Signal	Width	Direction	Description
`channel_idle`	NC	output	Channel idle status
`channel_error`	NC	output	Channel error flag
`scheduler_state`	4*NC	output	FSM state per channel

14.3.2 Channel State Encoding

Scheduler State Encoding

State	Value	Description
IDLE	4'h0	Waiting for kick-off
WAIT_DESC	4'h1	Waiting for descriptor
PARSE_DESC	4'h2	Parsing descriptor
CH_XFER_DATA	4'h3	Transfer in progress
CHECK_NEXT	4'h4	Checking next descriptor
COMPLETE	4'h5	Transfer complete
ERROR	4'hF	Error state

14.4 Configuration Registers

14.4.1 Address Range Validation

RAPIDS validates descriptor addresses against configurable ranges:

```
Valid if:
  (addr >= cfg_addr0_base && addr < cfg_addr0_limit) ||
  (addr >= cfg_addr1_base && addr < cfg_addr1_limit)
```

14.4.2 Configuration Registers

Configuration and status registers are provided by the `rapids_regs` register block. See the Register Map for the complete, RTL-accurate address map. The registers most relevant to bring-up are:

Key Configuration Registers

Register	Offset	Description
GLOBAL_CTRL	0x100	GLOBAL_EN, GLOBAL_RST (level; write 0 to release)
CHANNEL_ENABLE	0x120	Per-channel enable bitmap
SCHED_TIMEOUT_CYCLES	0x200	Write-progress timeout window
SCHED_TIMEOUT_LIMIT	0x208	Consecutive-timeout escalation limit (0 = never)
DESCENG_CONFIG	0x220	Descriptor-engine enable / prefetch / FIFO threshold
DESCENG_ADDR0_BASE/ _LIMIT	0x224/0x228	Descriptor address range 0

14.5 Programming Sequence

14.5.1 Initialization

```
// 1. Optional global reset: GLOBAL_RST is a level, not self-clearing
GLOBAL_CTRL = 0x2;           // assert
GLOBAL_CTRL = 0x0;           // release

// 2. Configure descriptor address range(s)
DESCENG_ADDR0_BASE  = 0x8000_0000;
DESCENG_ADDR0_LIMIT = 0x9000_0000;

// 3. Scheduler timeout policy
SCHED_TIMEOUT_CYCLES = 1000;
SCHED_TIMEOUT_LIMIT  = 4;    // escalate after 4 stalled windows; 0 =
never

// 4. Enable channels and global enable
CHANNEL_ENABLE = 0xFF;       // enable all 8 channels
GLOBAL_CTRL    = 0x1;        // GLOBAL_EN
```

14.5.2 Descriptor Kick-Off

```
// Prepare descriptor in memory
desc->src_addr = src;
desc->dst_addr = dst;
desc->length = beats;
desc->last = 1;
desc->valid = 1;

// Kick off channel 0: stage the address, then launch
CH0_DESC_ADDR_LOW  = (uint32_t)desc_addr;
CH0_DESC_ADDR_HIGH = (uint32_t)(desc_addr >> 32);
KICK_ENABLE        = 0x01;   // KICK0, self-clearing

// Wait for completion: idle, or faulted (CH_ERROR is never idle)
while (!(SCHEDULER_IDLE & 0x01) && !(SCHED_ERROR & 0x01));

// Check for errors
if (SCHED_ERROR & 0x01) {
    handle_error(0);
}
```


---

15 Monitor Bus Interface

15.1 Overview

RAPIDS Beats outputs a 64-bit Monitor Bus (MonBus) for real-time event reporting. The MonBus provides visibility into:

- State machine transitions
- Descriptor processing events
- Error conditions
- Performance metrics

15.2 MonBus Packet Format

15.2.1 64-bit Packet Structure

```
Bit Field           Width   Description
──────────────────────────────────────────────────────
[63:60]             4-bit   packet_type (error, completion, etc.)
[59:57]             3-bit   protocol (CORE for RAPIDS)
[56:53]             4-bit   event_code (specific event)
[52:47]             6-bit   channel_id
[46:43]             4-bit   unit_id (subsystem)
[42:35]             8-bit   agent_id (module)
[34:0]              35-bit  event_data (event-specific)
```

15.2.2 Signal List

MonBus Signals

Signal	Width	Direction	Description
`monbus_pkt_valid`	1	output	Packet valid
`monbus_pkt_data`	64	output	Packet data
`monbus_pkt_ready`	1	input	Downstream ready

15.3 Packet Types

MonBus Packet Types

Type	Code	Description
ERROR	4'h0	Error event

Type	Code	Description
COMPLETION	4'h1	Transfer/operation complete
THRESHOLD	4'h2	Threshold crossed
TIMEOUT	4'h3	Timeout event
PERF	4'h4	Performance metric
DEBUG	4'hF	Debug/trace event

15.4 Agent IDs

RAPIDS modules use the following Agent IDs:

RAPIDS Agent IDs

Agent ID	Module	Description
0x08	Descriptor AXI monitor	`axi4_master_rd_monlite` on the descriptor read master (per half, `USE_AXI_MONITORS`)
0x09	Sink-ingress network monitor	The sink RX plane's monitor-lite: `axis4_slave_monlite` on `s_axis_*` (AXIS top) or `cxs_monitor_lite` in `cxs_slave_monlite` on `s_cxs_*` (CXS top) (`USE_AXI_MONITORS`; config `SNK.MON.WRMON_*`)
0x0A	Source-egress network monitor	The source TX plane's monitor-lite: `axis4_master_monlite` on `m_axis_*` (AXIS top) or `cxs_monitor_lite` in `cxs_master_monlite` on `m_cxs_*` (CXS top) (`USE_AXI_MONITORS`; config `SRC.MON.RDMON_*`)
0x10-0x17	Descriptor Engine	Channels 0-7
0x20	Control Read Engine	ctrlrd (one per half)
0x21	Control Write	ctrlwr (one per half)

Agent ID	Module	Description
	Engine	
0x30-0x37	Scheduler	Channels 0-7

These are the values the scheduler group array and the halves pass down (`DESC_AXI_MON_AGENT_ID`, `DESC_MON_BASE_AGENT_ID`, `CTRLRD_MON_AGENT_ID`, `CTRLWR_MON_AGENT_ID`, `SCHED_MON_BASE_AGENT_ID`); the module defaults inside the engines differ and are overridden at instantiation. The data paths carry no MonBus emitters (an earlier revision listed data-path agents at 0x40-0x57, which nothing emits). Agents 0x09 and 0x0A live in the network planes, outside the core; their packets enter each half's 3:1 `monbus_arbiter` as client 1 on `snk_plane_mon_*` / `src_plane_mon_*`. A CXS plane's packets carry protocol `PROTOCOL_CXS`, which the top's monitor group filters through its AXIS mask slot. A CXS PACKET_END completion carries the channel (`CXSSRCID`) in `event_data[55:48]`; its `channel_id` field holds `{PRCLTYPE[2:0], TGTID[5:0]}`.

15.5 Event Codes

15.5.1 Scheduler Events (Agent 0x30-0x37)

Scheduler Event Codes

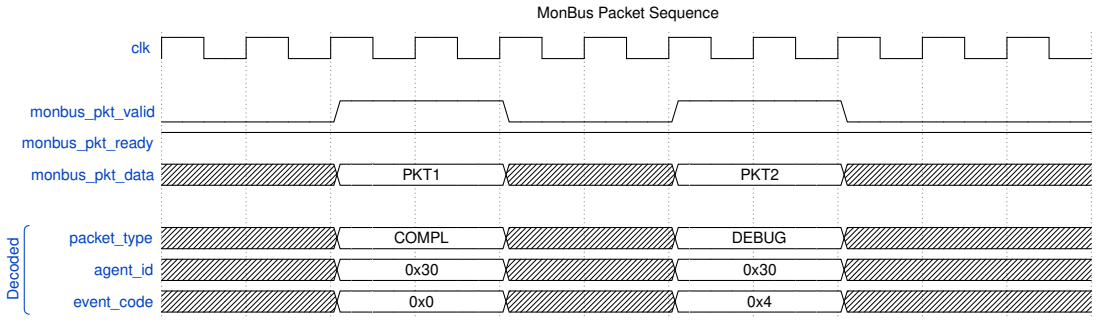
Event Code	Type	Description
0x0	COMPLETION	Descriptor complete
0x1	COMPLETION	Chain complete
0x2	ERROR	Timeout
0x3	ERROR	AXI error
0x4	DEBUG	State transition

The scheduler's and the descriptor engine's COMPLETION packets are reported only while the half's `SCHED_CONFIG.COMPL_EN` is set (reset value 1). With the bit clear the scheduler group drops them before its monitor-bus arbiter and still acknowledges the emitters, so a channel runs identically either way; ERROR packets are reported regardless of `ERR_EN`.

15.5.2 Descriptor Engine Events (Agent 0x10-0x17)

Descriptor Engine Event Codes

Event Code	Type	Description
0x0	COMPLETION	Descriptor fetched
0x1	ERROR	Address range error
0x2	ERROR	AXI read error
0x3	DEBUG	Fetch started



Timing Diagram

15.6 Event Data Encoding

15.6.1 Completion Event Data

```
[34:32] - Reserved
[31:0]  - Transfer length (beats completed)
```

15.6.2 Error Event Data

```
[34:32] - Error type
[31:16] - Reserved
[15:0]  - Error details
```

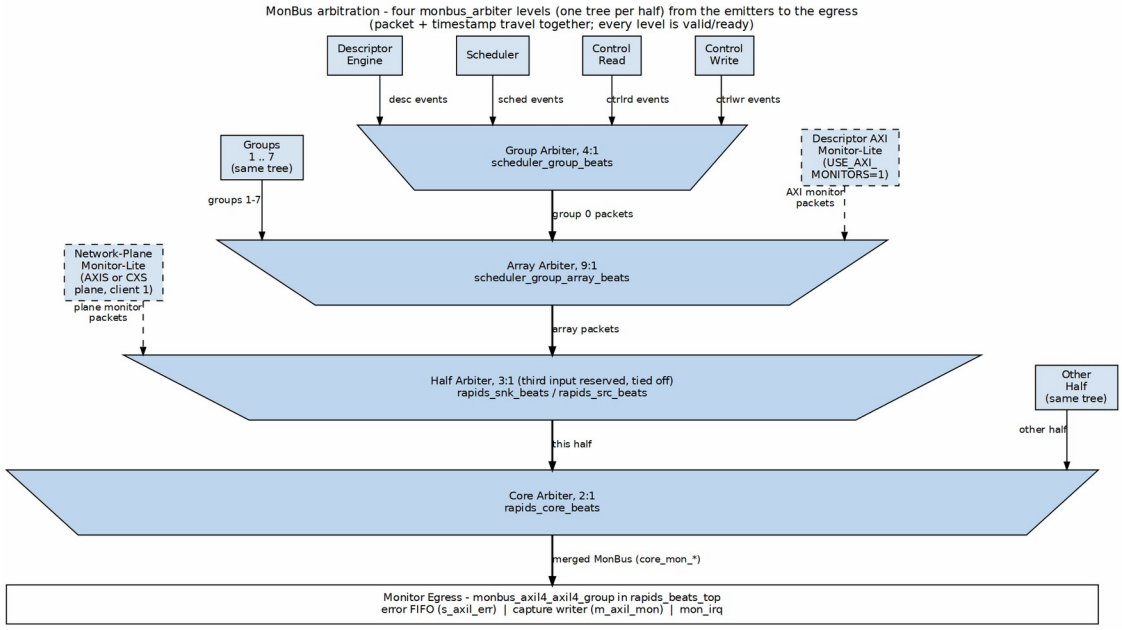
15.6.3 State Transition Event Data

```
[34:32] - Reserved
[31:28] - Previous state
[27:24] - New state
[23:0]  - Timestamp (lower bits)
```

15.7 MonBus Arbitration

Multiple RAPIDS modules generate MonBus packets. Internal arbitration ensures ordered delivery:

15.7.1 Figure 3.5.1: MonBus Arbitration Tree



MonBus Arbitration Tree

Source: ch03_monbus_arbitration.dot

15.7.2 Arbitration Policy

- Round-robin across modules
- No packet loss (backpressure if output blocked)
- Priority boosting for ERROR packets

15.8 Top-Level MonBus Delivery (monbus_axil4_axil4_group)

At the top level (`rapids_beats_top`, `rapids_beats_cxs_top`), the internal MonBus is not exposed as a raw 64-bit port. Instead, when `USE_AXI_MONITORS = 1`, the core's single merged stream (`core_mon_*`, which already carries the descriptor-monitor and network-plane packets) feeds a `monbus_axil4_axil4_group`. The data masters `m_axi_rd` and `m_axi_wr` carry no monitors. When `USE_AXI_MONITORS = 0`, the group is not built and its outputs are tied off (`mon_irq = 0`). The group presents three external interfaces:

Top-Level MonBus AXI-Lite Group

Interface	Signals	Purpose
Error-drain slave	`s_axil_err_ar*`, `s_axil_err_r*` (AXI-Lite, 32-bit read)	CPU reads captured error events from the error FIFO

Interface	Signals	Purpose
Capture master	`m_axil_mon_aw*`, `m_axil_mon_w*` (64-bit `wdata`), `m_axil_mon_b*`	Bulk-writes the MonBus trace to system memory
Interrupt	`mon_irq`	Asserts on error/threshold events

The capture region and flush behavior are configured by `cfg_mon_base_addr`, `cfg_mon_limit_addr`, and `cfg_mon_flush_watermark`.

15.9 Integration Notes

15.9.1 Downstream Connection

Internally, MonBus output connects to:

1. **MonBus Arbiter** - Combines core + rd/wr monitor sources
2. **monbus_axil4_axil4_group** - Error-drain slave, capture master, and `mon_irq`
3. **Debug FIFO** - Buffered access for debug tools

15.9.2 Packet Rate

MonBus Packet Rates

Scenario	Approximate Rate
Idle	0 packets/cycle
Single channel active	~1 packet/100 cycles
All channels active	~1 packet/20 cycles
Error storm	Up to 1 packet/cycle

16 CXS Network Interface

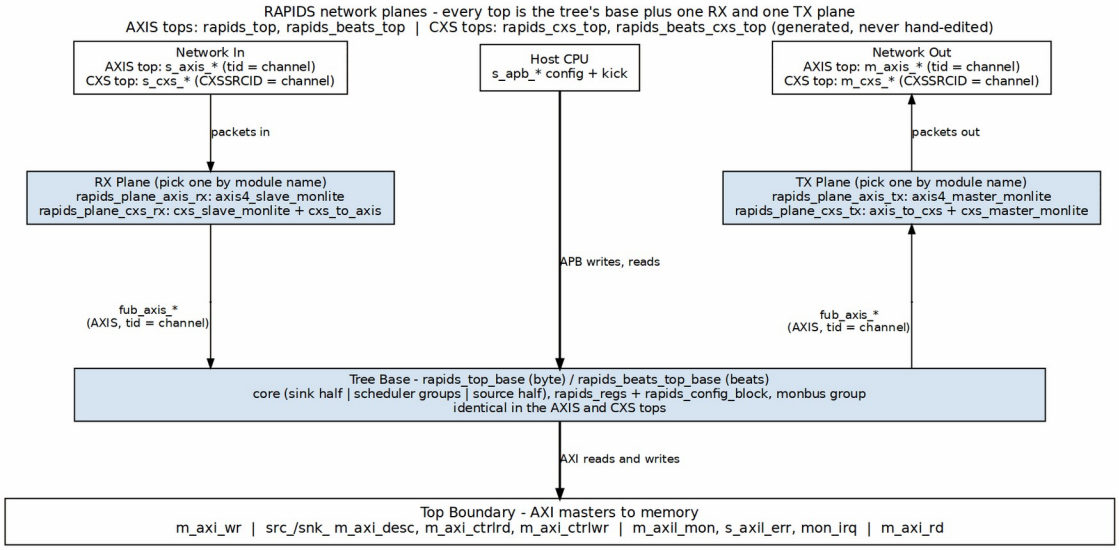
16.1 Overview

The network side of RAPIDS is a swappable **network plane**. Each top is the tree's base (core, registers, monitor group) plus one receive (RX) plane and one transmit (TX) plane, and the plane is chosen by the top's module name:

- `rapids_beats_top`, `rapids_top`: AXI-Stream planes; the `s_axis_*` and `m_axis_*` ports described in the AXIS Interface chapter.
- `rapids_beats_cxs_top`, `rapids_cxs_top`: CXS planes; an `s_cxs_*` link into the sink half and an `m_cxs_*` link out of the source half.

The core never sees CXS. Every plane presents the same AXIS stream (`tid` = channel) to the core, so the descriptor engines, schedulers, SRAM buffers and AXI engines are identical in the AXIS and CXS tops. All four tops are generated from the tree's base by `bin/gen_plane_tops.py`.

16.1.1 Figure 3.6.1: Network Planes



Network Planes

Source: ch03_network_planes.dot

16.2 Link Profile

The CXS planes use one fixed profile of Arm IHI 0079 (CXS) Issue D. Issue D is the first issue with the SRCID, TGTID and USER sidebands, which RAPIDS needs to carry the channel and the byte strobes.

CXS Link Profile

Property	Value
Links	One per direction: `s_cxs_*` into the sink half, `m_cxs_*` out of the source half
Channel	`CXSSRCID` carries the RAPIDS channel (the AXIS `tid`)
`CXSTGTID`	The AXIS `tdest`
`CXSUSER`	`{tuser, tstrb}`; `tuser` is dropped when `DATA_WIDTH/8 + AXIS_USER_WIDTH > 128`
Packets per flit / start alignment	`CXSMAXPKTPERFLIT` = 2, 16-byte alignment; one AXIS packet is one CXS packet
Stream framing	Per stream, keyed on the channel bits of `CXSSRCID`: channels may interleave mid-packet
Link control	Explicit credit return, activation and deactivation; idle deactivation after `CXS_IDLE_CYCLES`
Parity	Off
Data width	`DATA_WIDTH` up to 1024 (`CXSUSER` is at most 128 bits and carries `tstrb`)

Channels interleave on one link. The byte-granular source arbitrates per drain block while a packet spans its whole descriptor, so with two or more source channels active the egress switches channel in the middle of a packet. That is legal CXS (streams may interleave, IHI 0079D 2.5.2) because every block on the link frames packets per stream rather than per link.

16.3 Credits

One credit permits one flit (IHI 0079D 2.1). Credits are a single pool for the whole link: a credit grant carries no ID, so it cannot be reserved for one channel.

CXS Top Parameters

Parameter	Default	Meaning
CXS_MAX_CREDIT	32	Credits the receiver grants. The grant-to-flit-to-free round trip is about 3 cycles, so 32 sustains one flit per cycle with margin for link pipelining or a clock crossing
CXS_RX_FIFO_DEPTH	CXS_MAX_CREDIT	The sink plane's receive FIFO; it is the credit pool, so every granted credit is backed by space
CXS_CREDIT_BEYOND_SPEC	CXS_MAX_CREDIT > 63	Above the spec's limit of 63 the link is built beyond spec, and the far end must be built the same way
CXS_IDLE_CYCLES	32	Idle cycles before the TX plane deactivates the link

The credit count is not tied to SRAM_DEPTH. SRAM_DEPTH is the beats each channel buffers in each direction; the credit pool only has to cover the link's round trip.

16.4 Signal List

The sink link (s_cxs_*) is a CXS receiver; the source link (m_cxs_*) is a CXS transmitter. Direction is given at the RAPIDS top.

CXS Link Signals

Signal	Sink (s_cxs_*)	Source (m_cxs_*)	Width	Description
*_cxs_valid	input	output	1	Flit valid
*_cxs_data	input	output	DATA_WIDTH	Flit data

Signal	Sink (s_cxs_*)	Source (m_cxs_*)	Width	Description
*_cxs_cntl	input	output	CXS_CNTL_PW	Packet start/end control (CXSCNTL)
*_cxs_srcid	input	output	AXIS_ID_WIDTH	Source ID; the channel
*_cxs_tgtid	input	output	AXIS_DEST_WIDTH	Target ID; the AXIS tdest
*_cxs_user	input	output	CXS_UW	{tuser, tstrb}
*_cxs_crdgnt	output	input	1	Credit grant
*_cxs_crdrtn	input	output	1	Credit return
*_cxs_activereq	input	output	1	Link activation request
*_cxs_activeack	output	input	1	Link activation acknowledge
*_cxs_deacthint	output	input	1	Deactivation hint

CXS Status Signals

Signal	Direction	Description
snk_cxs_link_state, src_cxs_link_state	output [1:0]	{ACTIVEREQ, ACTIVEACK} of each link
snk_cxs_protocol_error, src_cxs_protocol_error	output	Sticky: the endpoint saw a CXS protocol violation
snk_cxs_unsupported	output	Sticky: a received flit that AXIS framing cannot represent

16.5 Flow Control and the Sink Contract

The byte-granular sink holds `s_axis_tready` low on an AXIS top until the channel's descriptor has been fetched. On a CXS top the same backpressure appears as credits: the receive FIFO fills and the sink plane stops granting credits until the core drains it.

16.6 Monitoring

Each CXS plane carries a `cxs_monitor_lite` in place of the AXIS monitor-lite. It is programmed from the same CSRs (`SNK.WRMON_*` for the sink plane, `SRC.RDMON_*` for the source plane) and reports with the same agent IDs (0x09 sink, 0x0A source). Its packets are tagged `PROTOCOL_CXS` and join the half's 3:1 `monbus_arbiter` as client 1, exactly where the AXIS plane's packets go. See the Monitor Bus Interface chapter.

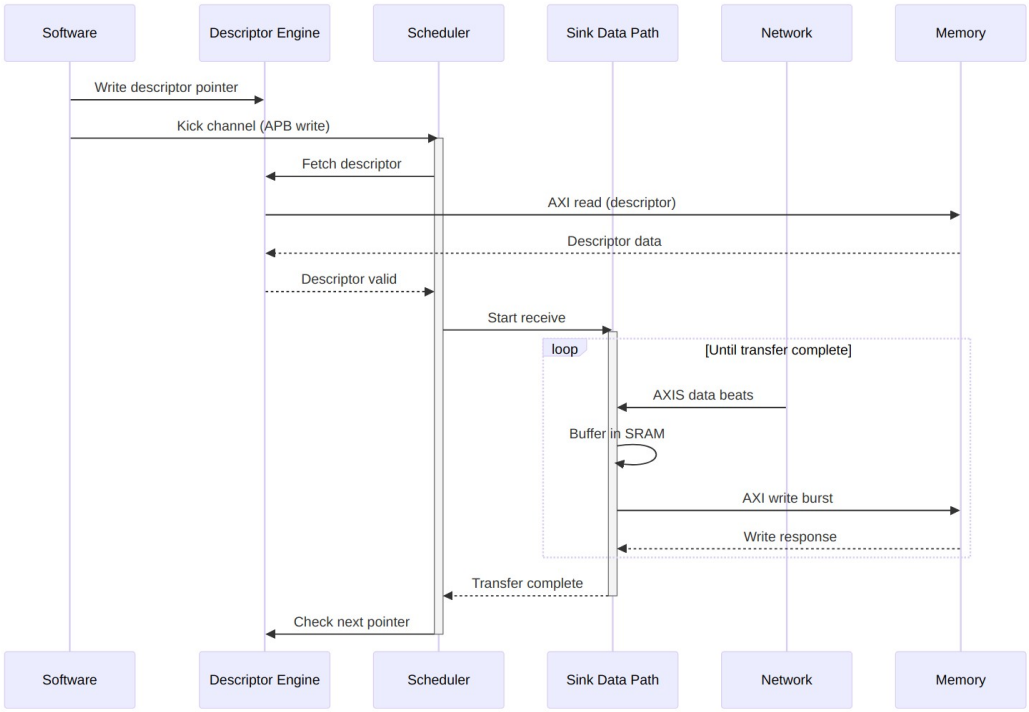
16.7 Routing and Payload Type (Proposed)

A proposal (2026-10-16, not implemented) gives every packet a header carrying its destination coordinate, source coordinate, payload type (`cfg` / `prog` / `mem`), length and target address. The header belongs to the tile network, not to CXS, and is described once in the compute-engine tile's `TILE_SPEC.md` Section 11. CXS only transports it, as the first flit of the packet. Nothing on this page changes: `CXSSRCID` keeps the channel, so per-stream framing is the same. The descriptor fields that supply the header are in the Descriptor Format chapter.

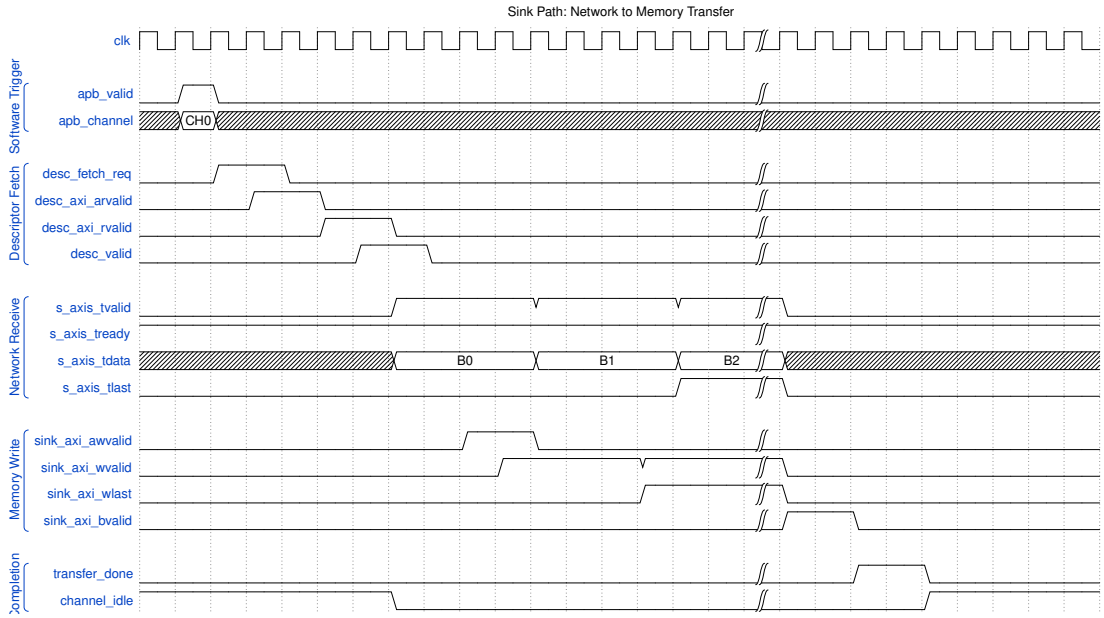
17 Use Case: Network to Memory (Sink Path)

17.1 Overview

The Sink Path transfers data from a network interface (AXI-Stream slave) to system memory (AXI4 write master). This is the primary receive data path for network applications.



Operation Flow



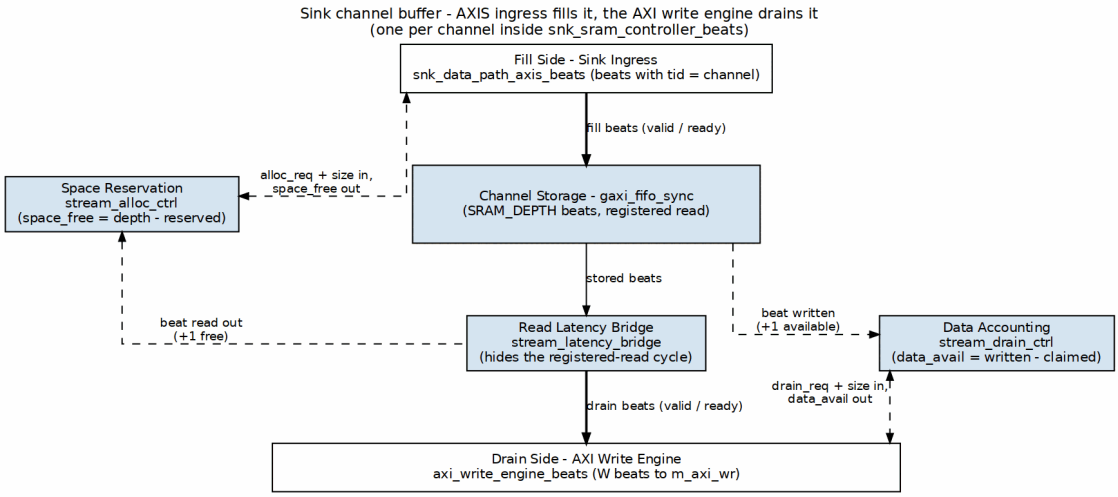
Timing Diagram

17.2 Buffer Management

17.2.1 SRAM Buffering Strategy

The sink path uses internal SRAM to decouple network timing from memory timing:

17.2.2 Figure 4.1.1: Sink Channel Buffer



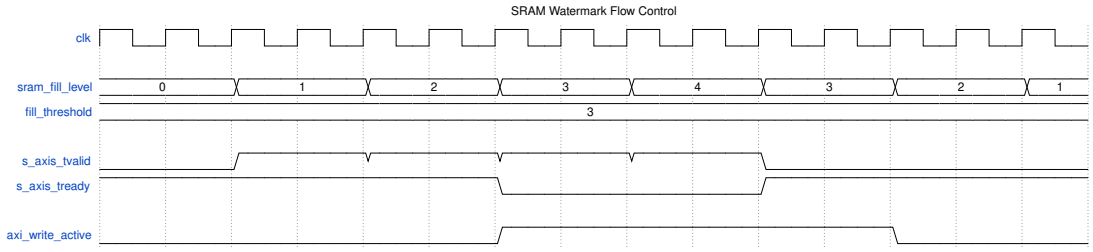
Sink Channel Buffer

Source: ch04_sink_buffering.dot

17.2.3 Flow Control

Sink Path Flow Control

Condition	Behavior
SRAM full	Backpressure network (TREADY=0)
SRAM empty	Pause AXI writes
SRAM threshold	Optional early write start



Watermark Timing

17.3 Performance Considerations

17.3.1 Throughput

Sink Path Throughput Factors

Factor	Impact
Network bandwidth	Upper bound on receive rate
Memory bandwidth	Must match or exceed network
SRAM depth	Burst absorption capability
AXI outstanding	Overlapped write latency hiding

17.3.2 Latency

Sink Path Latency Breakdown

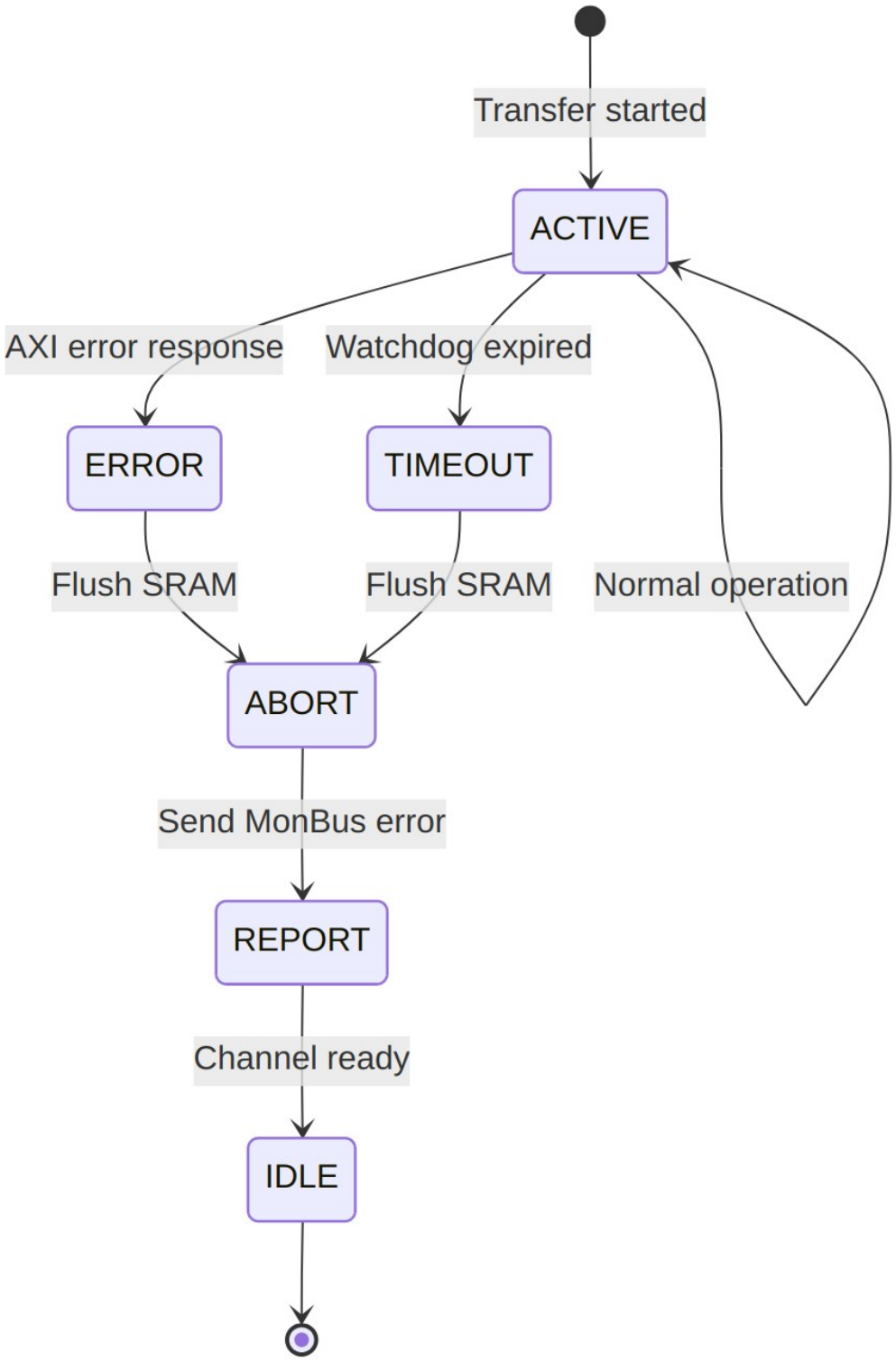
Phase	Typical Cycles
Descriptor fetch	10-50 (memory dependent)
First beat to SRAM	1
SRAM to AXI write	2-4
AXI write completion	10-100 (memory dependent)

17.4 Error Handling

17.4.1 Error Sources

Sink Path Error Handling

Error	Detection	Response
AXI write error	BRESP != OKAY	Stop transfer, report via MonBus
SRAM overflow	Fill pointer catches drain	Should never occur (TREADY control)
Timeout	Watchdog timer	Abort transfer, report error



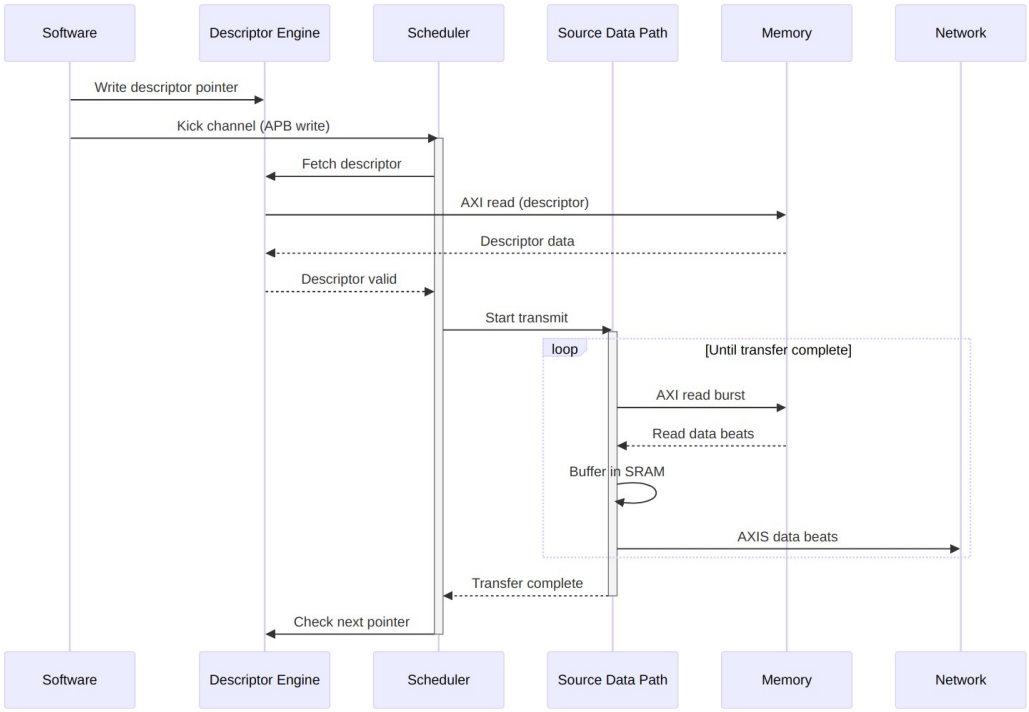
Error Recovery Sequence

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

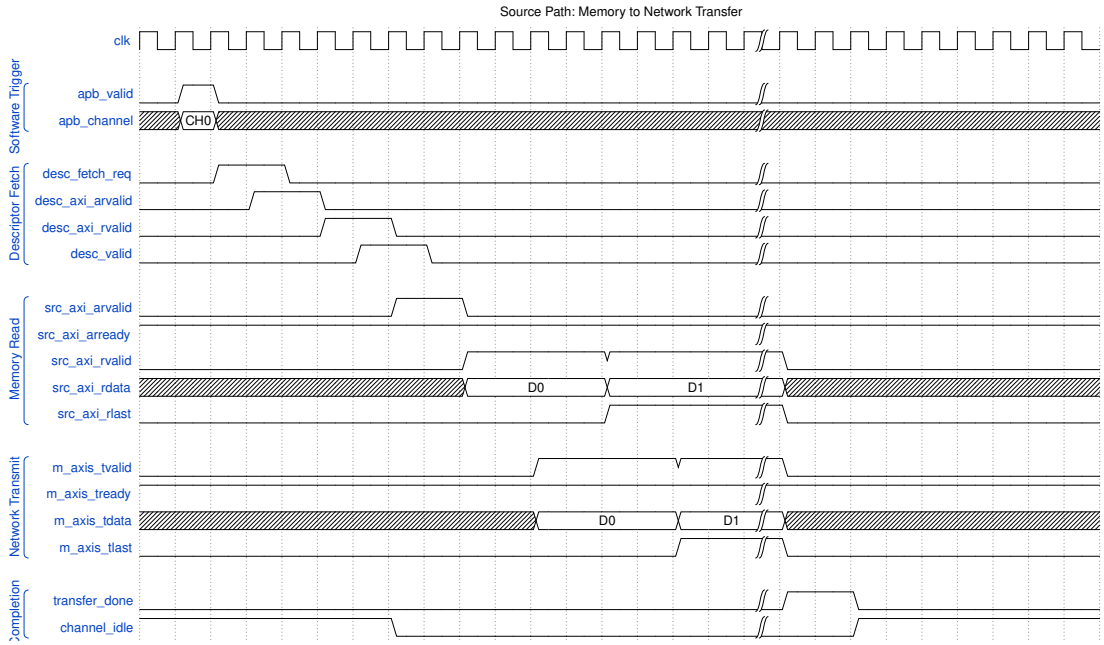
18 Use Case: Memory to Network (Source Path)

18.1 Overview

The Source Path transfers data from system memory (AXI4 read master) to a network interface (AXI-Stream master). This is the primary transmit data path for network applications.



Operation Flow



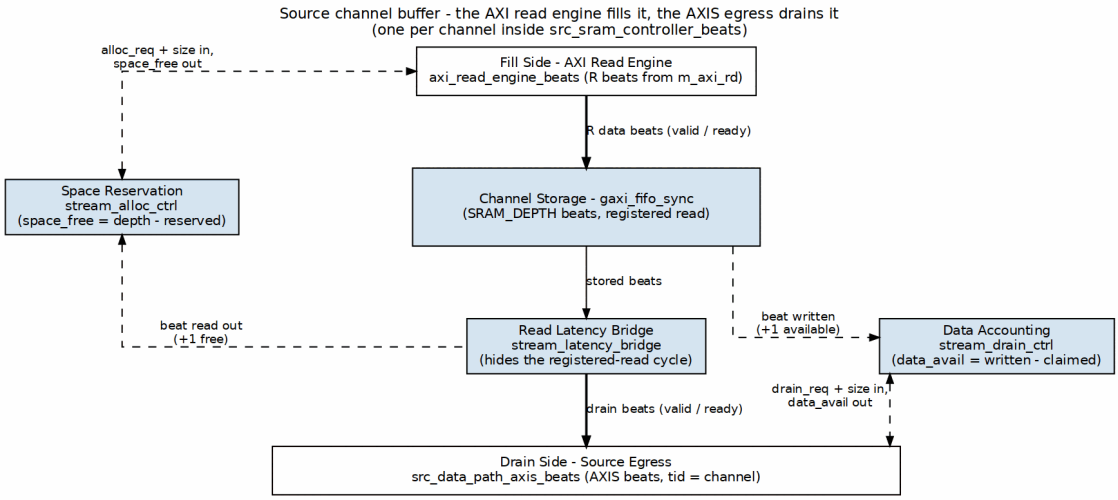
Timing Diagram

18.2 Buffer Management

18.2.1 SRAM Buffering Strategy

The source path uses internal SRAM to prefetch data and maintain network throughput:

18.2.2 Figure 4.2.1: Source Channel Buffer



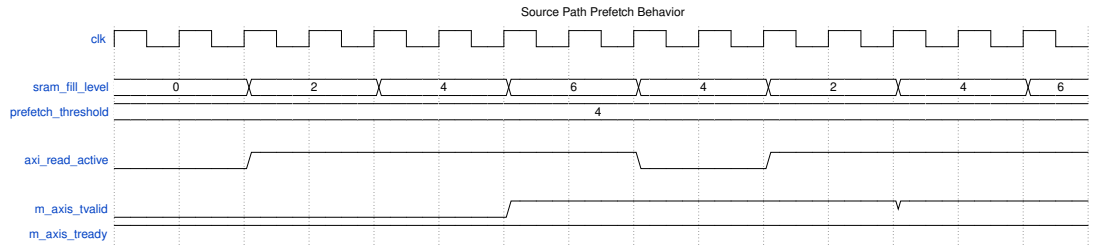
Source Channel Buffer

Source: ch04_source_buffering.dot

18.2.3 Flow Control

Source Path Flow Control

Condition	Behavior
SRAM full	Pause AXI reads
SRAM empty	Hold TVALID low
Network backpressure	Stop draining SRAM

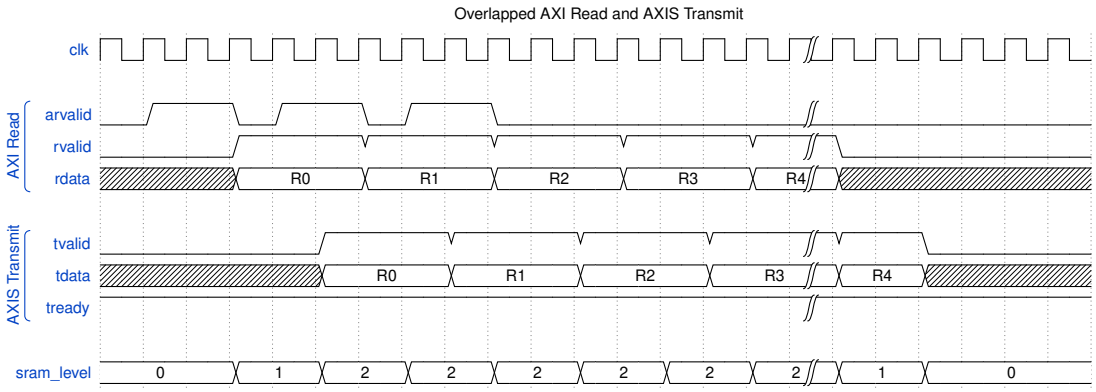


Prefetch Strategy

18.3 Concurrent Operation

18.3.1 Read-Ahead Pipeline

The source path can issue AXI reads ahead of network consumption:



Waveform 20

18.4 Performance Considerations

18.4.1 Throughput

Source Path Throughput Factors

Factor	Impact
Memory bandwidth	Upper bound on transmit rate
Network acceptance	Must keep pace with reads
SRAM depth	Read-ahead capability
AXI outstanding	Overlapped read latency hiding

18.4.2 Latency

Source Path Latency Breakdown

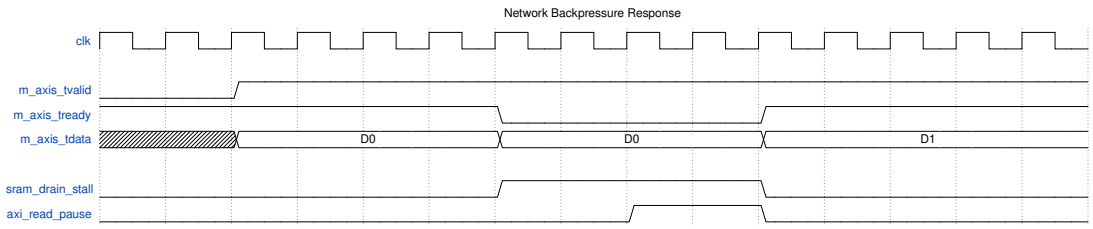
Phase	Typical Cycles
Descriptor fetch	10-50 (memory dependent)
AXI read issue	1-2
Memory read latency	10-100 (memory dependent)
SRAM to network	1-2

18.5 Error Handling

18.5.1 Error Sources

Source Path Error Handling

Error	Detection	Response
AXI read error	RRESP != OKAY	Stop transfer, report via MonBus
Network timeout	Watchdog timer	Abort transfer, report error
Address range	Descriptor validation	Reject descriptor



Backpressure Handling

When the network deasserts TREADY: 1. SRAM drain stalls immediately 2. SRAM continues filling until threshold 3. AXI reads pause when SRAM approaches full 4. Resume automatically when TREADY returns

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

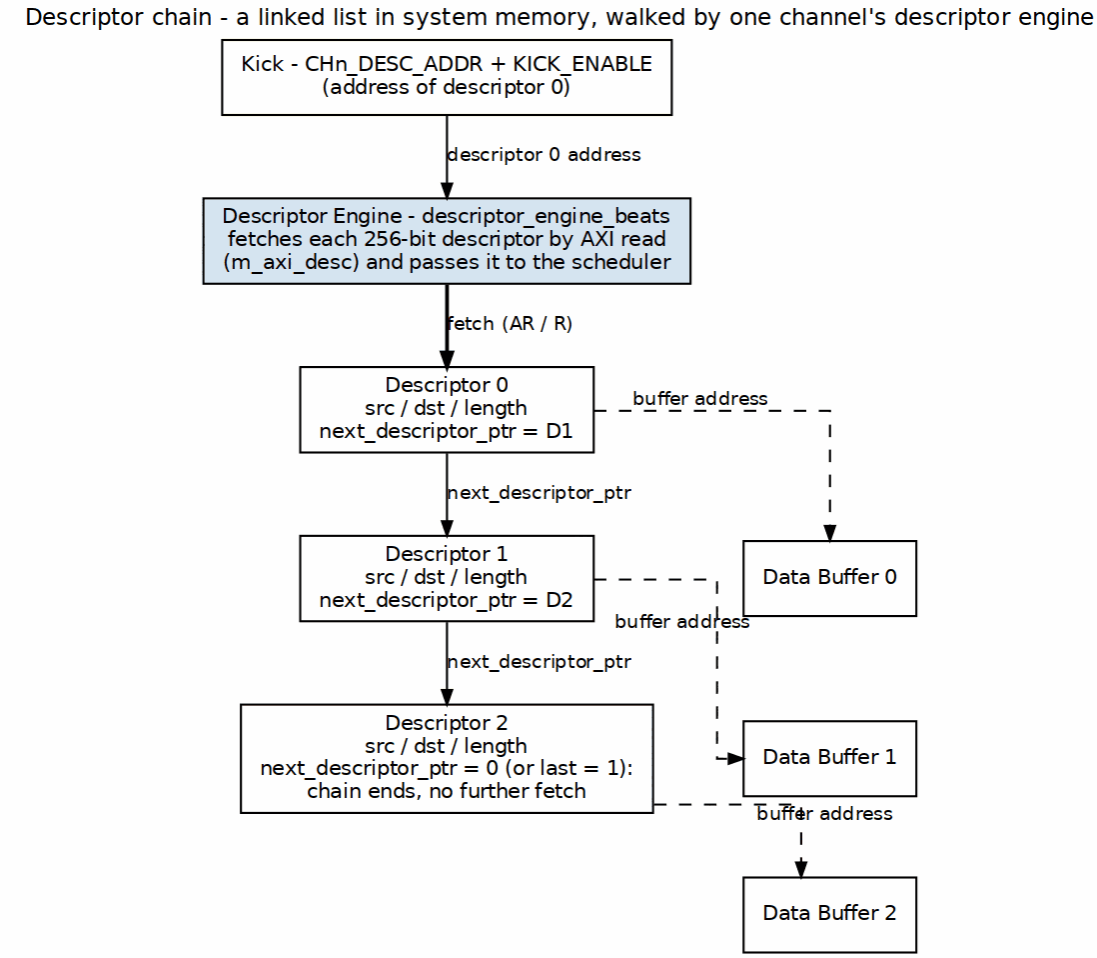
19 Use Case: Descriptor Chaining

19.1 Overview

Descriptor chaining enables multiple transfers to execute sequentially without software intervention. Each descriptor contains a pointer to the next descriptor, forming a linked list that RAPIDS processes automatically.

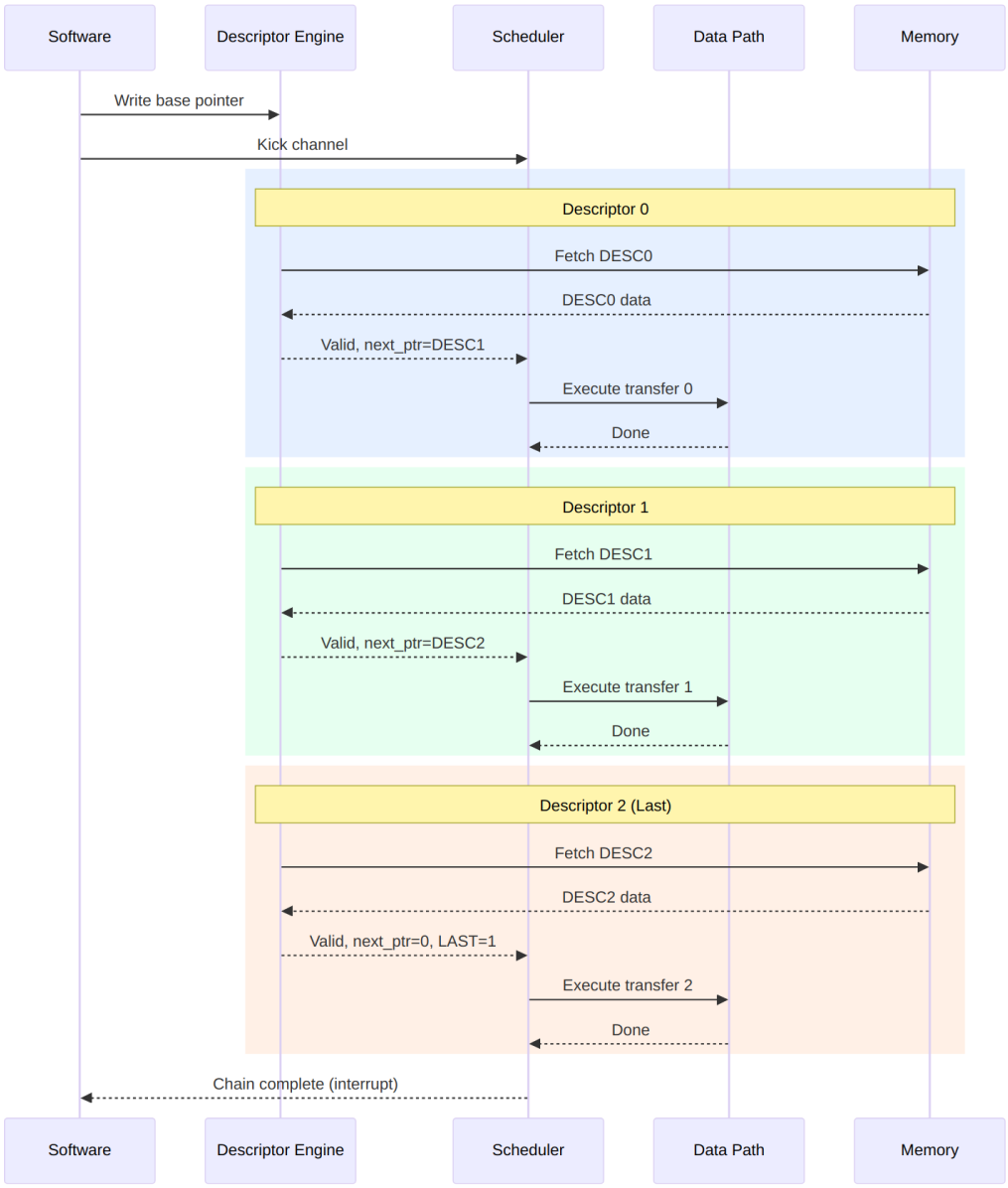
19.2 Chain Structure

19.2.1 Figure 4.3.1: Descriptor Chain Structure

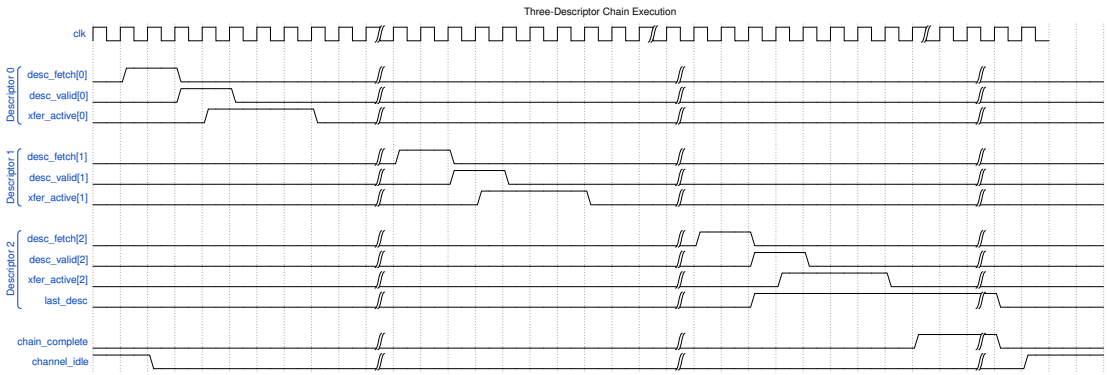


Descriptor Chain Structure

Source: ch04_chain_structure.dot



Chain Execution Flow



Timing Diagram

19.3 Descriptor Fields for Chaining

19.3.1 Relevant Fields

Chaining-Related Descriptor Fields

Field	Bits	Description
`next_ptr`	[191:128]	64-bit pointer to next descriptor
`last`	[255]	Last descriptor in chain flag
`irq_en`	[254]	Generate interrupt on completion

19.3.2 Chain Termination

A chain terminates when ANY of these conditions is true:

1. `next_ptr == 0` (null pointer)
2. `last == 1` (explicit last flag)
3. Error during transfer

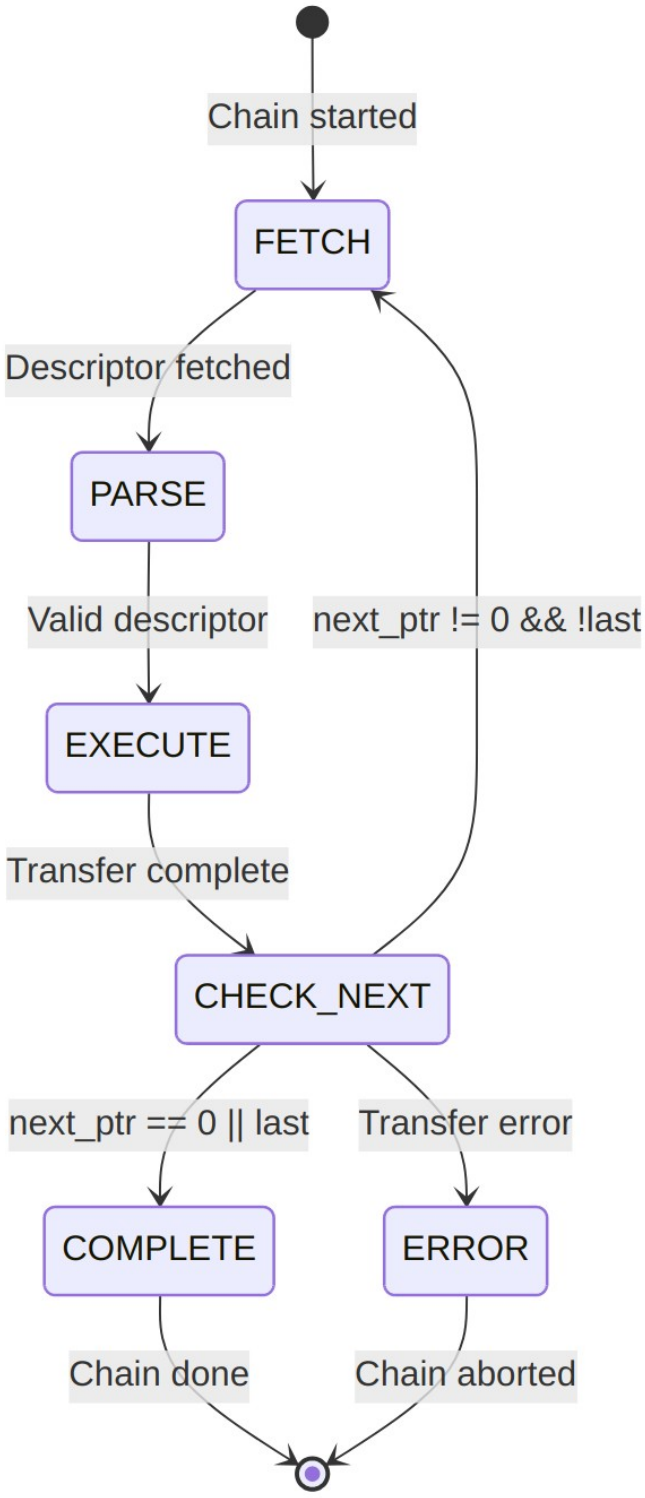
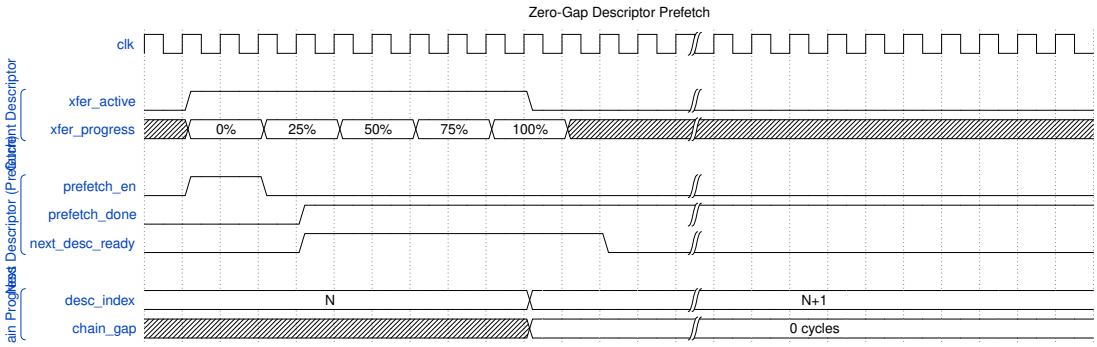

Chain started
FETCH
Descriptor fetched
PARSE
Valid descriptor
next_ptr != 0 && !last
EXECUTE
Transfer complete
CHECK_NEXT
next_ptr == 0 || last
Transfer error
COMPLETE
ERROR
Chain done
Chain aborted

Diagram 10

19.4 Prefetch Optimization

19.4.1 Descriptor Prefetching

RAPIDS can prefetch the next descriptor while the current transfer executes:



Waveform 23

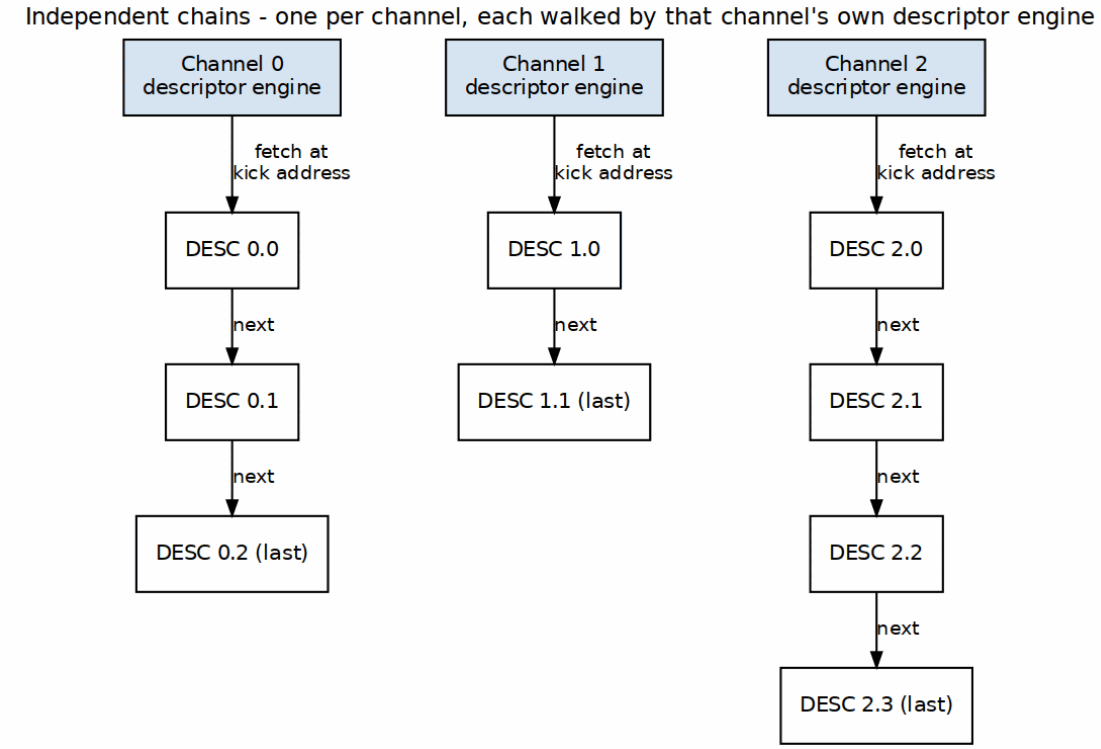
Benefit: Zero-cycle gap between chained transfers when prefetch completes before current transfer.

Prefetch depth is controlled by `DESCENG_CONFIG.PREFETCH_EN` and `DESCENG_CONFIG.FIFO_THRESH` (register block, offset 0x220). With prefetch disabled the descriptor engine fetches on demand (roughly one descriptor ahead of the scheduler); with prefetch enabled it buffers up to `FIFO_THRESH` descriptors ahead. A chain fetch that is throttled (output FIFO at the threshold) is deferred and reissued automatically once the FIFO drains, so no chain link is lost.

19.5 Multi-Channel Chaining

Each channel maintains independent chains:

19.5.1 Figure 4.3.2: Independent Per-Channel Chains



Independent Per-Channel Chains

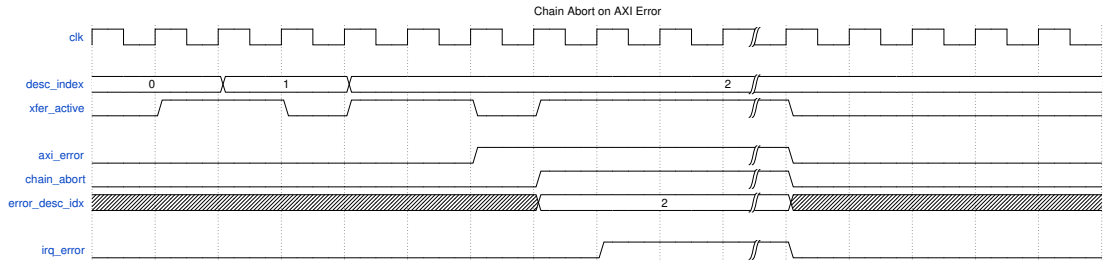
Source: ch04_multichannel_chains.dot

Channels execute their chains concurrently and independently.

19.6 Error During Chain

19.6.1 Error Handling

When an error occurs mid-chain:



Waveform 24

Error Response: 1. Current transfer aborts 2. Chain processing stops 3. Error status captures failing descriptor index 4. MonBus reports error event 5. Interrupt generated (if enabled)

19.6.2 Recovery

Software must: 1. Read error status to identify failing descriptor 2. Fix the issue (descriptor content, memory mapping, etc.) 3. Restart chain from failing descriptor or beginning

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

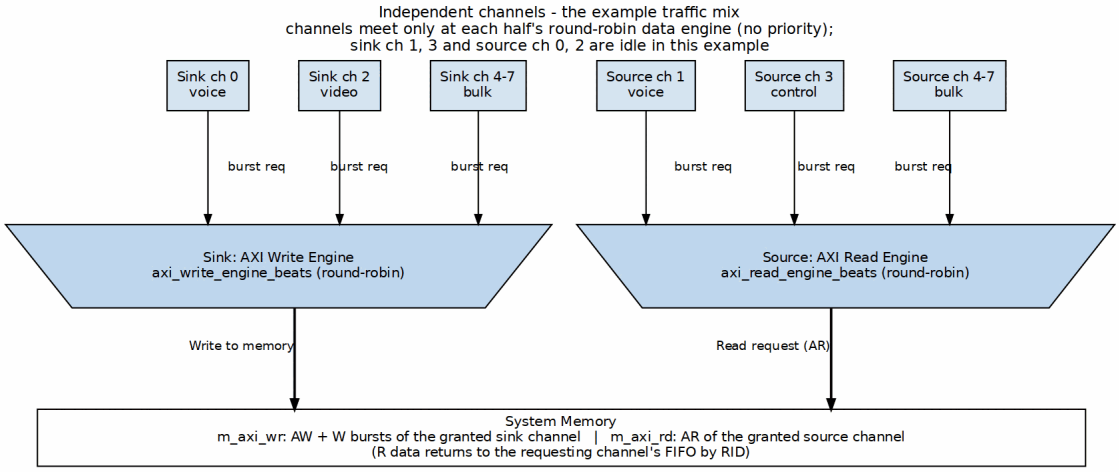
20 Use Case: Multi-Channel Operation

20.1 Overview

RAPIDS Beats supports 8 independent DMA channels that can operate concurrently. This enables parallel data transfers for multiple network connections, priority-based scheduling, and efficient resource utilization.

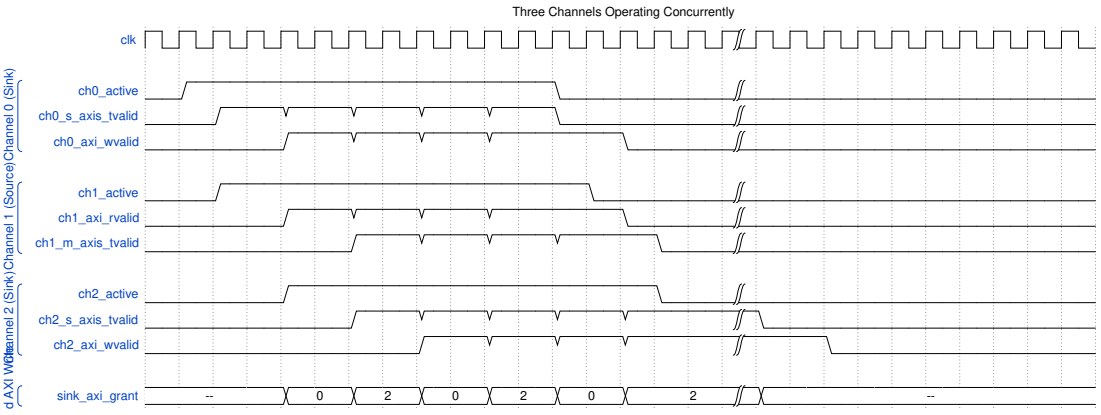
20.2 Channel Independence

20.2.1 Figure 4.4.1: Independent Channels



Independent Channels

Source: ch04_channel_independence.dot

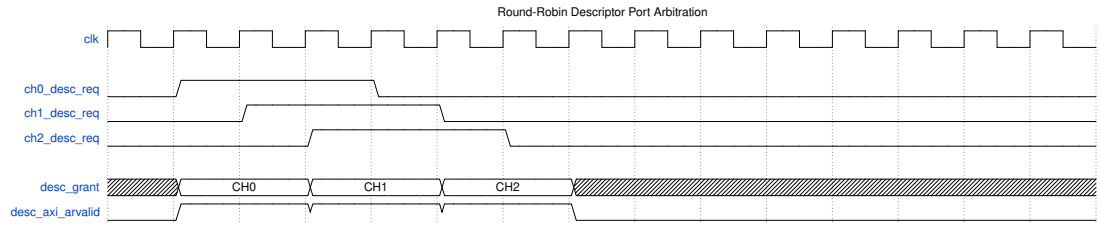


Concurrent Operation Timing

20.3 Resource Arbitration

20.3.1 Descriptor Port Arbitration

All channels share a single AXI port for descriptor fetches:

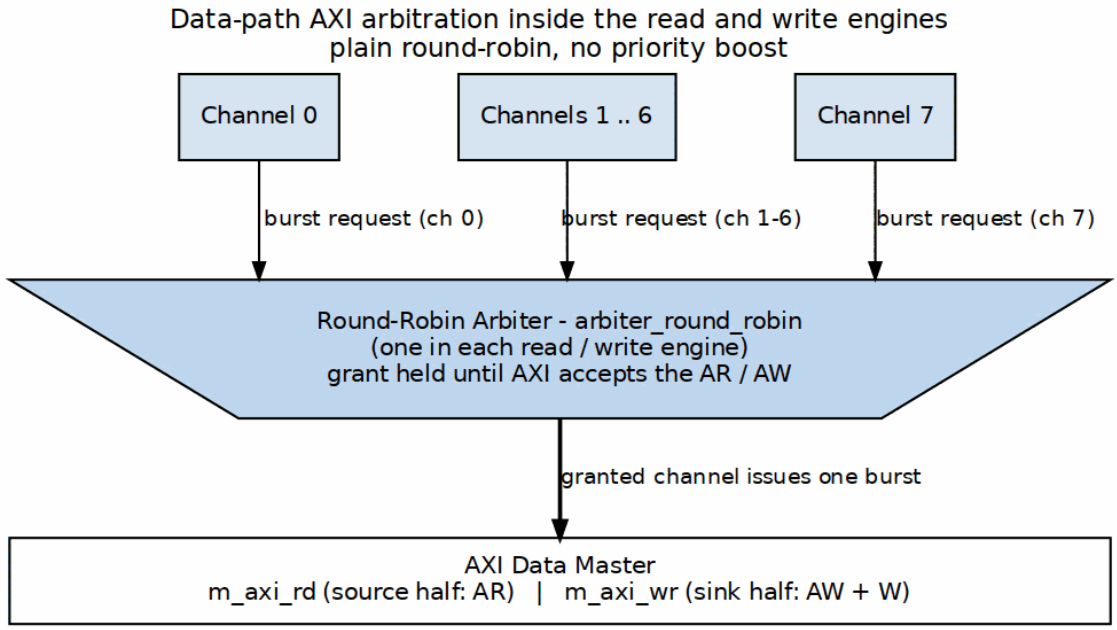


Waveform 26

20.3.2 Sink/Source Arbitration

Data path AXI ports use plain round-robin arbitration (`arbiter_round_robin` inside each engine). There is no priority boost: the descriptor's priority field is carried but not used for arbitration.

20.3.3 Figure 4.4.2: Data-Path AXI Arbitration



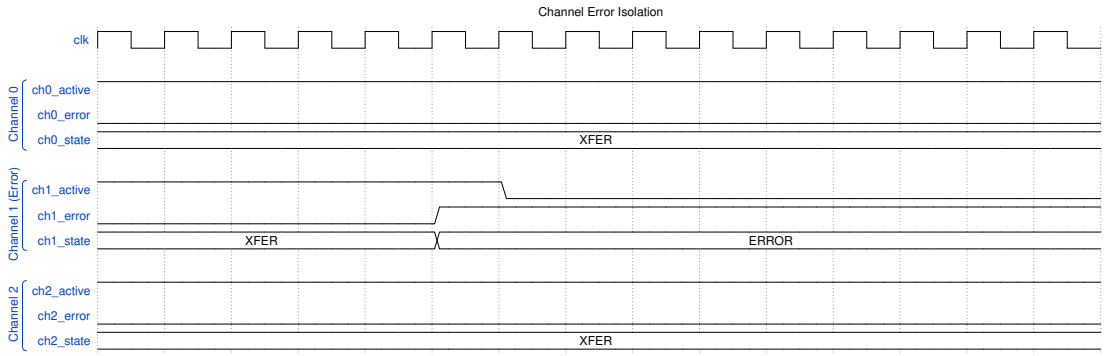
Data-Path AXI Arbitration

Source: ch04_datapath_arbitration.dot

20.4 Channel Isolation

20.4.1 Error Isolation

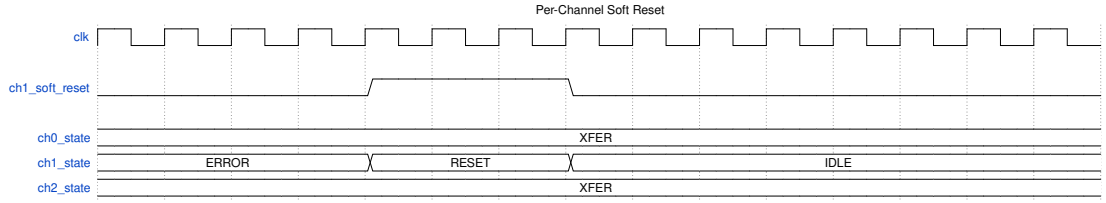
Errors in one channel do not affect other channels:



Waveform 27

20.4.2 Per-Channel Soft Reset

Individual channels can be reset without affecting others:



Waveform 28

20.5 Use Case Examples

20.5.1 Mixed Traffic Pattern

Typical network application with different traffic types:

Mixed Traffic Channel Assignment

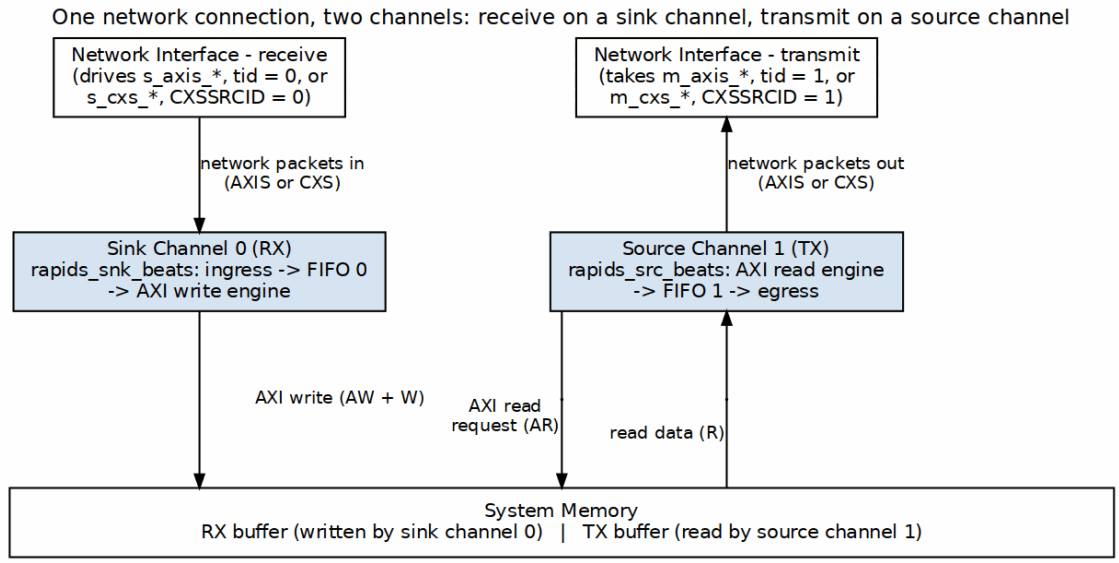
Channel	Traffic Type	Direction	Priority	Notes
0	Voice/RTP	Sink	High	Low latency required
1	Voice/RTP	Source	High	Low

Channel	Traffic Type	Direction	Priority	Notes
				latency required
2	Video	Sink	Medium	High bandwidth
3	Control	Source	Medium	Small packets
4-7	Bulk data	Both	Low	Background transfers

20.5.2 Bidirectional Transfer

Single network connection using paired channels:

20.5.3 Figure 4.4.3: Bidirectional Transfer



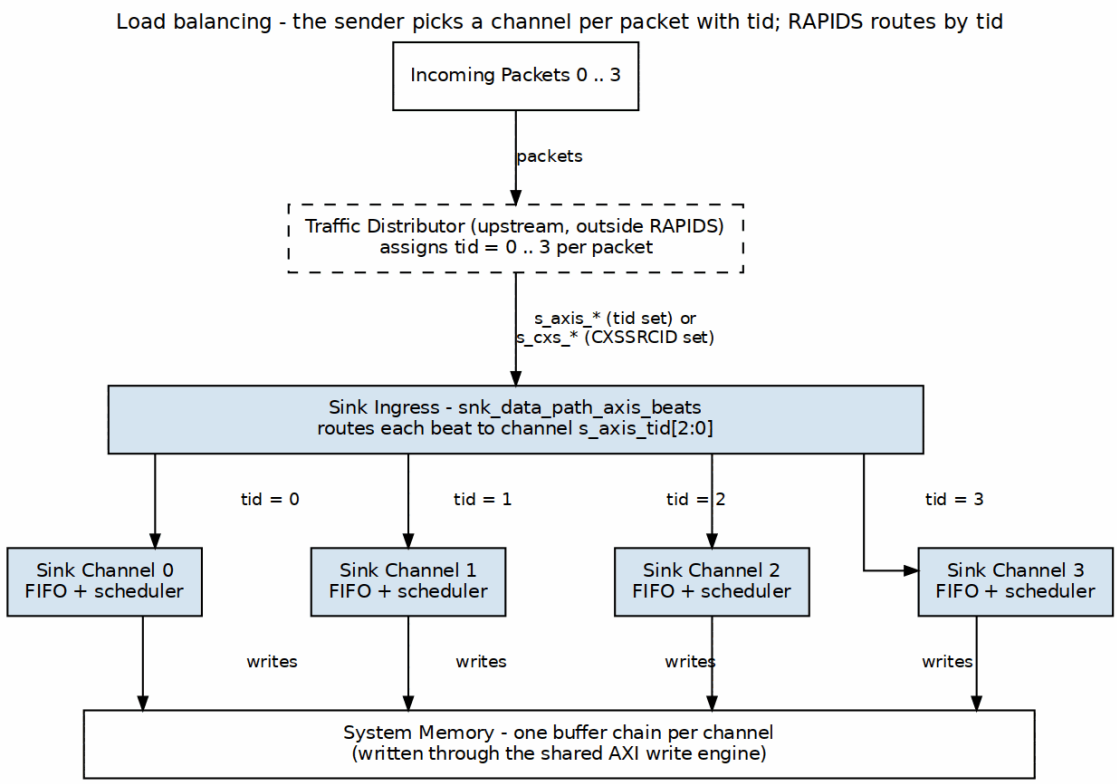
Bidirectional Transfer

Source: ch04_bidirectional.dot

20.5.4 Load Balancing

Multiple channels processing same traffic type:

20.5.5 Figure 4.4.4: Load Balancing by tid



Load Balancing by tid

Source: ch04_load_balancing.dot

20.6 Performance Considerations

20.6.1 Bandwidth Sharing

Bandwidth Distribution

Active Channels	Per-Channel BW (% of max)	Notes
1	100%	Full bandwidth available
2	~50% each	Round-robin sharing
4	~25% each	Fair distribution
8	~12.5% each	All channels active

Note: Actual distribution depends on transfer sizes and memory latency.

20.6.2 Latency Impact

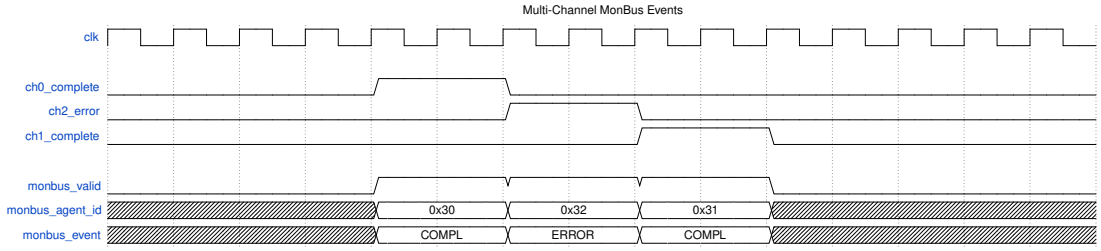
Multi-channel operation adds arbitration latency:

Arbitration Latency

Scenario	Additional Latency
Single channel	0 cycles
2 channels contending	1-2 cycles
8 channels contending	Up to 7 cycles

20.7 MonBus Events

Each channel generates independent MonBus events:



Waveform 29

Events are arbitrated and serialized on the shared MonBus output.

21 Descriptor Format

21.1 Overview

RAPIDS Beats uses 256-bit (32-byte) descriptors to define DMA transfers. Descriptors are stored in system memory and fetched by the Descriptor Engine when a channel is kicked.

21.2 Descriptor Layout

21.2.1 256-Bit Structure

Bit Range	Field Name	Width	Description
[255:213]	reserved	43	Reserved (write 0)
[212:210]	desc_type	3	0 = LEGACY, 1 = EXTENDED (see below)
[209:208]	opcode	2	0 = DATA, 1 = CTRL_READ, 2 = CTRL_WRITE
[207:200]	priority	8	Transfer priority (informational)
[199:196]	channel_id	4	Channel ID (informational)
[195]	error	1	Error flag (written by hardware)
[194]	last	1	Last descriptor in chain
[193]	gen_irq	1	Generate interrupt on completion
[192]	valid	1	Descriptor valid flag
[191:160]	next_descriptor_ptr	32	Address of next descriptor (0 = last)
[159:128]	length	32	Transfer length in BEATS (not bytes)
[127:64]	dst_addr	64	Destination address
[63:0]	src_addr	64	Source address

: 256-Bit Descriptor Layout (`rapids_pkg::descriptor_t`)

Proposed (2026-10-16, not implemented): three descriptor fields would supply the tile packet header defined in the compute-engine tile's `TILE_SPEC.md` Section 11. They use bits that are unused today.

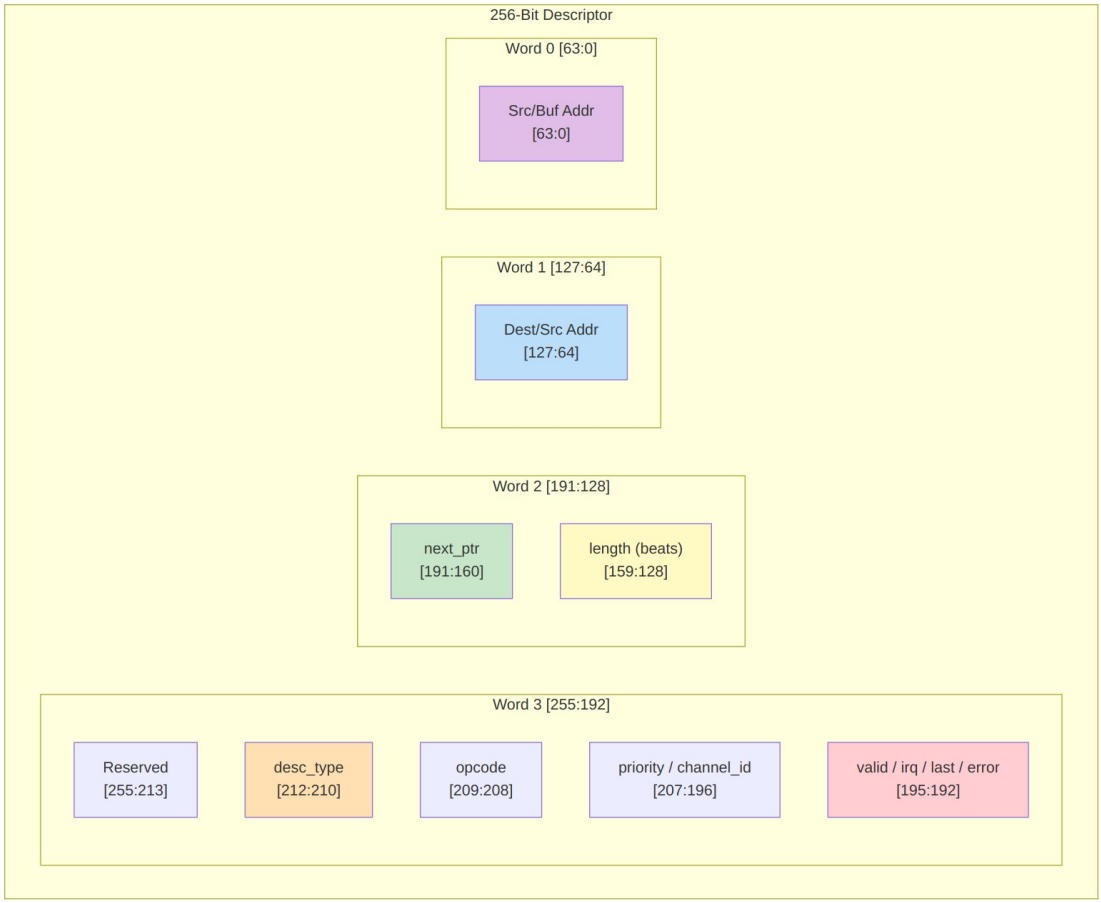
Proposed Descriptor Fields

Field	Descriptor bits	Meaning
`ptype`	`[214:213]` (from the reserved `[255:213]`)	0 = `cfg`, 1 = `prog`, 2 = `mem`, 3 = reserved

Field	Descriptor bits	Meaning
Destination coordinate	`dst_addr[63:50]` = descriptor `[127:114]`	`{level, X, Y, Z}` (TILE_SPEC Section 11.3)
Target address	`dst_addr[49:0]` = descriptor `[113:64]`	Byte address inside the addressed node or cell

The coordinate and target address apply to SOURCE descriptors, whose `dst_addr` is unused today. The source half would build the header from these fields and send it ahead of the payload. A source descriptor that pairs a cell's X and Y with a `level` other than 0 would be rejected with a descriptor error.

The authoritative definition is `rapids_pkg::descriptor_t` in `rtl/includes/rapids_pkg.sv`; the same layout is restated in the scheduler's header comment. Note the struct is declared 272 bits wide with a 16-bit pad above bit 255 – only `[255:0]` is fetched and used.



Visual Layout

21.3 Field Descriptions

21.3.1 Control Bits [195:192]

Control Bits

Bit	Field	Description
192	`valid`	Must be 1; a descriptor with valid=0 raises a descriptor error
193	`gen_irq`	Set to 1 to generate a completion interrupt
194	`last`	Set to 1 for the last descriptor in a chain
195	`error`	Error flag; written by hardware, write 0

21.3.2 Direction

There is no direction field in the descriptor. Direction is a property of the **half** that owns the channel: the SOURCE half (memory -> AXIS) reads from `src_addr`, the SINK half (AXIS -> memory) writes to `dst_addr`. A descriptor is delivered to one half's descriptor engine by the address it was staged at, so the same layout serves both.

21.3.3 Transfer Length [159:128]

- 32-bit field specifying transfer size in BEATS, not bytes
- 1 beat = DATA_WIDTH bits (512 bits = 64 bytes in the default build)
- Length of 0 is reserved (no operation)

21.3.4 Next Pointer [191:160]

- **32-bit** address of the next descriptor (zero-extended to the engine's 64-bit address width)
- Value of 0 terminates the chain, as does `last` = 1
- Autonomous chaining also requires the address to fall inside one of the two configured descriptor address ranges

21.3.5 Address Fields [127:0]

For SINK transfers (network to memory): | Field | Usage | |———-|———-| | `dst_addr` [127:64] | Memory write destination address | | `src_addr` [63:0] | Not used (write 0) |

For SOURCE transfers (memory to network): | Field | Usage | |———-|———-| | `dst_addr` [127:64] | Not used (write 0) | | `src_addr` [63:0] | Memory read source address |

: Address Field Usage

21.4 Descriptor Opcodes (DATA / CTRL_READ / CTRL_WRITE)

Beyond plain DATA transfers, a descriptor carries a 2-bit opcode (`rapids_pkg desc_op_e`) that selects one of three behaviors. DATA descriptors move payload through the concurrent read/write engines (above); the two control opcodes let a channel synchronize with a producer/consumer in memory without moving payload – the basis of the RAPIDS producer/consumer flow.

Opcode	Value	Behavior
DATA	2'b00	Concurrent read/write payload transfer (fields as above)
CTRL_READ	2'b01	**Consumer gate** – poll a memory location until `(read & mask) == expected`, then release the descriptor/chain
CTRL_WRITE	2'b10	**Producer doorbell** – single 32-bit write of a value to a memory location, then continue

CTRL_READ (consumer gate) re-interprets the descriptor fields as a poll specification. The scheduler drives the per-channel control-read engine, which reads the gate address once per retry and compares the masked value; the descriptor completes when the gate is satisfied. The retry budget is bounded by `CTRL_CONFIG.CTRLRD_MAX_TRY` (0-511, reset 16) so a never-satisfied gate cannot hang the channel – exhaustion raises an error instead.

Field	Bits	Usage
poll_addr	[63:0]	Gate address to poll
expected	[95:64]	Expected value after masking
mask	[127:96]	Compare mask (`(read & mask) == expected`)
max_try	[143:128]	Per-descriptor retry hint (capped by `CTRLRD_MAX_TRY`)

CTRL_WRITE (producer doorbell) performs one single-beat 32-bit AXI write, after which the chain continues. It has no poll/retry – it is an unconditional write used to signal a consumer.

Field	Bits	Usage
wr_addr	[63:0]	Doorbell address
wr_data	[95:64]	32-bit value to write

The control opcodes are handled by dedicated engines (control-read / control-write), one pair per channel, driven by the scheduler's control interface. See the MAS Scheduler (Section 2.1) and Control-Read / Control-Write Engine specifications (Sections 2.8 / 2.9).

21.5 Extended Descriptors (row/col-major addressing)

desc_type = 1 (EXTENDED) selects strided / 2-D / circular addressing instead of linear accumulation. It is gated at build time by USE_ROW_COL_MAJOR_ADDRESSING; when that parameter is 0 the second-chunk fetch is unreachable and synthesizes away, and an EXT descriptor is treated as legacy.

An extended descriptor is **512 bits in two 256-bit chunks**. Chunk 0 is the layout above with desc_type = 1. Chunk 1 holds the address-generator configuration and is fetched by a second single-beat read at descriptor_addr + 0x20, so the two chunks must be contiguous in memory.

```
Chunk 1 (at descriptor_addr + 0x20)
Bit Range    Field Name        Width  Description
-------------------------------------------------------------------------
[255:192]    reserved_hi       64     Reserved (future 3rd dimension)
[191:188]    wr_reserved       4      Reserved
[187:182]    wr_wrap1_log2     6      Write outer wrap window, log2 (0
= off)
[181:176]    wr_wrap0_log2     6      Write inner wrap window, log2 (0
= off)
[175:160]    wr_inner_count    16     Write beats per contiguous run
[159:128]    wr_stride_1       32     Write outer stride, SIGNED bytes
[127:96]     wr_stride_0       32     Write inner stride, SIGNED bytes
[95:92]      rd_reserved       4      Reserved
[91:86]      rd_wrap1_log2     6      Read outer wrap window, log2 (0
= off)
[85:80]      rd_wrap0_log2     6      Read inner wrap window, log2 (0
= off)
[79:64]      rd_inner_count    16     Read beats per contiguous run
[63:32]      rd_stride_1       32     Read outer stride, SIGNED bytes
[31:0]       rd_stride_0       32     Read inner stride, SIGNED bytes
```

: Extended Descriptor Chunk 1 (rapids_pkg::descriptor_ext_t)

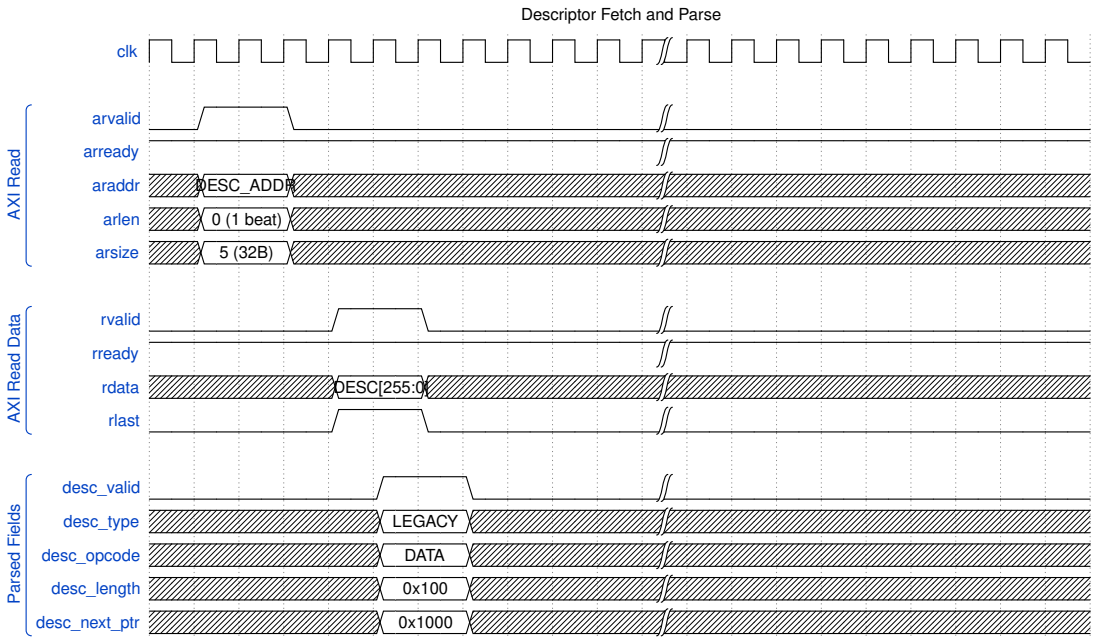
Read and write are configured independently, which is what makes transpose possible: one side bursts while the other issues single beats.

Mode selection is implicit. The hardware compares stride_0 against the beat size (DATA_WIDTH/8):

Condition	Mode	Behaviour
`stride_0 == beat_size`	run-contiguous	bursts `inner_count` beats, then jumps to the next run base
`stride_0 != beat_size`	per-beat 2-D	one beat per address; transpose, gather, scatter

Addresses follow `addr = base + (i0 * stride_0) + (i1 * stride_1)`, with each offset masked by `(1 << wrap_log2) - 1` when that wrap field is non-zero, giving circular buffers. `i0` is the fast index and runs to `inner_count`. Strides are signed, so negative values walk backwards.

The layout is byte-compatible with STREAM's `descriptor_ext_t`, so one descriptor builder serves both engines.



Descriptor Fetch Timing

21.6 Alignment Requirements

Address Alignment Requirements

Field	Alignment	Notes
Descriptor address	32-byte	One descriptor is 32 bytes; an EXTENDED descriptor's chunk 1 is fetched at `+0x20`

Field	Alignment	Notes
`next_ptr` [191:160]	32-byte	32-bit descriptor address, zero-extended by the engine
`dst_addr` [127:64]	DATA_WIDTH/8	64-byte for 512-bit data
`src_addr` [63:0]	DATA_WIDTH/8	64-byte for 512-bit data

Note: Unaligned addresses may cause unpredictable behavior or AXI protocol violations.

21.7 Descriptor Examples

21.7.1 Sink Descriptor (Network to Memory)

A 256-beat sink transfer to `0x1_0000_0000`, chained to a descriptor at `0x2000`, raising an interrupt on completion.

```
64-bit words, LSB word first:
[63:0]    = 0x0000_0000_0000_0000  // src_addr  (unused by the SINK
half)
[127:64]  = 0x0000_0001_0000_0000  // dst_addr  = 0x1_0000_0000
[191:128] = 0x0000_2000_0000_0100  // next_ptr[191:160]=0x2000,
length[159:128]=256
[255:192] = 0x0000_0000_0000_0007  // valid=1, gen_irq=1, last=1

Word 3 detail:
  [192] = 1        // valid
  [193] = 1        // gen_irq
  [194] = 1        // last
  [195] = 0        // error (hardware-written)
  [199:196] = 0    // channel_id (informational)
  [207:200] = 0    // priority
  [209:208] = 00   // opcode = DATA
  [212:210] = 000  // desc_type = LEGACY
```

21.7.2 Source Descriptor (Memory to Network)

A 128-beat source transfer from `0x2_0000_0000`, terminated by `next_ptr = 0`.

```
64-bit words, LSB word first:
[63:0]    = 0x0000_0002_0000_0000  // src_addr = 0x2_0000_0000
[127:64]  = 0x0000_0000_0000_0000  // dst_addr (unused by the SOURCE
half)
```

```
[191:128] = 0x0000_0000_0000_0080  // next_ptr = 0 (terminates),
length = 128
[255:192] = 0x0000_0000_0000_0003  // valid=1, gen_irq=1, last=0

Word 3 detail:
  [192] = 1        // valid
  [193] = 1        // gen_irq
  [194] = 0        // not last -- next_ptr = 0 terminates the chain
  [209:208] = 00   // opcode = DATA
  [212:210] = 000  // desc_type = LEGACY
```

21.8 Software Construction

21.8.1 C Structure Example

```c
typedef struct __attribute__((packed, aligned(32))) {
    uint64_t src_addr;      // [63:0]
    uint64_t dst_addr;      // [127:64]
    uint32_t length;        // [159:128]  transfer length in BEATS
    uint32_t next_ptr;      // [191:160]  32-bit, 0 terminates the
chain
    uint32_t flags;         // [223:192]  see macros below
    uint32_t reserved[2];   // [255:224]
} rapids_descriptor_t;

// flags bit positions are relative to descriptor bit 192
#define DESC_VALID          (1u << 0)   // [192]
#define DESC_GEN_IRQ        (1u << 1)   // [193]
#define DESC_LAST           (1u << 2)   // [194]
#define DESC_ERROR          (1u << 3)   // [195] hardware-written
#define DESC_CHANNEL_ID(n)  (((n) & 0xFu) << 4)    // [199:196]
#define DESC_PRIORITY(n)    (((n) & 0xFFu) << 8)   // [207:200]
#define DESC_OPCODE(n)      (((n) & 0x3u) << 16)   // [209:208]
#define DESC_TYPE(n)        (((n) & 0x7u) << 18)   // [212:210]

#define DESC_OP_DATA        0x0
#define DESC_OP_CTRL_READ   0x1
#define DESC_OP_CTRL_WRITE  0x2

#define DESC_TYPE_LEGACY    0x0
#define DESC_TYPE_EXT       0x1
```

21.8.2 Descriptor Ring Setup

```c
// Allocate descriptor ring (must be 32-byte aligned)
rapids_descriptor_t *desc_ring = aligned_alloc(32, NUM_DESC *
sizeof(rapids_descriptor_t));
```

```
// Initialize chain
for (int i = 0; i < NUM_DESC - 1; i++) {
    desc_ring[i].next_ptr = (uint64_t)&desc_ring[i + 1];
    desc_ring[i].control = DESC_CTRL_IRQ_EN;
}

// Last descriptor
desc_ring[NUM_DESC - 1].next_ptr = 0;
desc_ring[NUM_DESC - 1].control = DESC_CTRL_LAST | DESC_CTRL_IRQ_EN;
```


---

22 Register Map

22.1 Overview

RAPIDS Beats provides a memory-mapped register interface for configuration and status. All registers are accessed through the single top-level APB slave (`s_apb_*`) and are implemented by the PeakRDL-generated `rapids_regs` register block. One addrmap holds two independent halves of the same layout (`rapids_engine_regs`): **SRC at 0x0000** and **SNK at 0x1000**, 4 KB each. Within a half, the staged descriptor addresses occupy +0x000-0x03F, the launch register `KICK_ENABLE` sits at +0x040, configuration and status at +0x100-0x3FF, and the nested monitor regfile (`rapids_mon_regs`) at +0x800. Every offset below is relative to its half; add 0x1000 for the SNK copy.

Because the SNK half lives at 0x1000, the APB address bus must be at least 13 bits wide; `paddr[12]` selects the half and `paddr[11]` the monitor window.

22.2 Address Space Layout

Range (per half)	Target	Purpose
+0x000-0x03F	`rapids_engine_regs`	Per-channel staged descriptor addresses
+0x040	`rapids_engine_regs`	`KICK_ENABLE` per-channel launch bits
+0x100-0x3FF	`rapids_engine_regs`	Configuration and status
+0x800-0x9FF	`rapids_mon_regs`	AXI-monitor config and performance
0x0000 / 0x1000	half base	SRC half / SNK half

The three `*_PKT_MASK` registers are masks: bit[type] = 1 masks (drops) that packet type, at the monitor-lite and again at the monbus group. They reset to all-masked (0xFFFF), so nothing reaches the group until the host clears the bits of the classes it wants to see. (rapids BUG-008: the description used to read “1 = enable”; the polarity and the reset are as intended.) `*_TIMEOUT` counts MICROSECONDS in its low 16 bits.

On a build with `USE_MON_REGS = 0` (the default when `USE_AXI_MONITORS = 0`) the two monitor windows keep their addresses but answer every access with `PSLVERR` and zero read data: the config blocks drive nothing from them, and a read-back that succeeded would tell the host a monitor is configured that does not exist.

: Address Space Layout

22.3 Base Registers (0x100-0x3FF)

Base Registers

Offset	Name	Access	Description
0x100	GLOBAL_CTRL	RW	GLOBAL_EN[0], GLOBAL_RST[1] (level; write 0 to release)
0x104	GLOBAL_STATUS	RO	SYSTEM_IDLE[0]
0x108	VERSION	RO	MINOR[7:0], MAJOR[15:8], NUM_CHANNELS[23:16]=8
0x120	CHANNEL_ENABLE	RW	CH_EN[7:0] per-channel enable
0x124	CHANNEL_RESET	RW	CH_RST[7:0] per-channel reset (level; write 0 to release)
0x140	CHANNEL_IDLE	RO	CH_IDLE[7:0] – not driven at the top level, reads 0; use SCHEDULER_IDLE
0x144	DESC_ENGINE_IDLE	RO	Per-channel descriptor-engine idle
0x148	SCHEDULER_IDLE	RO	Per-channel scheduler idle (excludes CH_ERROR)
0x150-0x16C	CH_STATE[0..7]	RO	Per-channel FSM state (stride 0x4)
0x170	SCHED_ERROR	RO	Per-channel sticky scheduler error
0x174	AXI_RD_COMPLETE	RO	Per-channel read-complete (SRC half only; reads 0 in SNK)
0x178	AXI_WR_COMPLETE	RO	Per-channel write-complete – not driven, reads 0
0x200	SCHED_TIMEOUT_CYCLES	RW	Write-progress timeout window (cycles, reset 1000)
0x204	SCHED_CONFIG	RW	SCHED_EN, TIMEOUT_EN, ERR_EN, COMPL_EN, PERF_EN

Offset	Name	Access	Description
0x208	SCHED_TIMEOUT_LIMIT	RW	LIMIT[7:0] escalation limit (reset 4, 0 = never)
0x220	DESCENG_CONFIG	RW	DESCENG_EN[0], PREFETCH_EN[1], FIFO_THRESH[5:2] (reset 8)
0x224	DESCENG_ADDR0_BASE	RW	Descriptor address range 0 base [31:0]
0x228	DESCENG_ADDR0_LIMIT	RW	Descriptor address range 0 limit [31:0]
0x22C	DESCENG_ADDR1_BASE	RW	Descriptor address range 1 base [31:0]
0x230	DESCENG_ADDR1_LIMIT	RW	Descriptor address range 1 limit [31:0]
0x240	CTRL_CONFIG	RW	CTRLRD_MAX_TRY[8:0] control-read poll retry budget (0-511, reset 16)
0x2A0	AXI_XFER_CONFIG	RW	AXI transfer sizing configuration
0x2B0	PERF_CONFIG	RW	Performance profiler configuration
0x2C0-0x2CC	OBS_CTRL/ OBS_FLAGS/ OBS_DATA0/ OBS_DATA1	RW/RO	Observation mux
0x35C	PERF_CH_SEL	RW	Performance channel select
0x378-0x380	HIST_SEL/ HIST_DATA/ HIST_TOTAL	RW/RO	Latency histogram

22.4 Monitor Registers (+0x800 within each half)

Monitor Registers (+0x800 within each half; SRC at 0x0800, SNK at 0x1800)

Offset	Name	Access	Description
+0x800	MON_FIFO_STATUS	RO	MonBus capture/error FIFO

Offset	Name	Access	Description
			status
+0x804	MON_FIFO_COUNT	RO	MonBus FIFO occupancy
+0x8C0-0x8DC	DAXMON_*	RW	Descriptor-monitor config (enable/timeout/latency/masks)
+0x8E0-0x8FC	RDMON_*	RW	Source-egress AXIS monitor-lite config (same layout; live in the SRC half, rapids TASK-015)
+0x900-0x91C	WRMON_*	RW	Sink-ingress AXIS monitor-lite config (same layout; live in the SNK half, rapids TASK-015; the SRC copy's COMPRESS_EN drives the group)
+0x950-0x978	DAXMON_PERF_*	RO/RW	Descriptor-monitor performance counters
+0x980-0x9A8	RDMON_PERF_*	RO/RW	Read-monitor performance counters
+0x9B0-0x9D8	WRMON_PERF_*	RO/RW	Write-monitor performance counters
+0x9E0-0x9F4	{RD,WR}MON_PERF_CH_*	RO	Per-channel producer/backpressure/starve/idle/overflow

22.5 Key Register Fields

22.5.1 GLOBAL_CTRL (0x100) - Read/Write

GLOBAL_CTRL Register

Bits	Field	Reset	Description
[0]	GLOBAL_EN	0	Master enable for the entire engine
[1]	GLOBAL_RST	0	Global reset: holds every

Bits	Field	Reset	Description
			channel of the half in reset while 1. Not self-clearing – write 0 to release
[31:2]	Reserved	0	Reserved

22.5.2 SCHED_TIMEOUT_LIMIT (0x208) - Read/Write

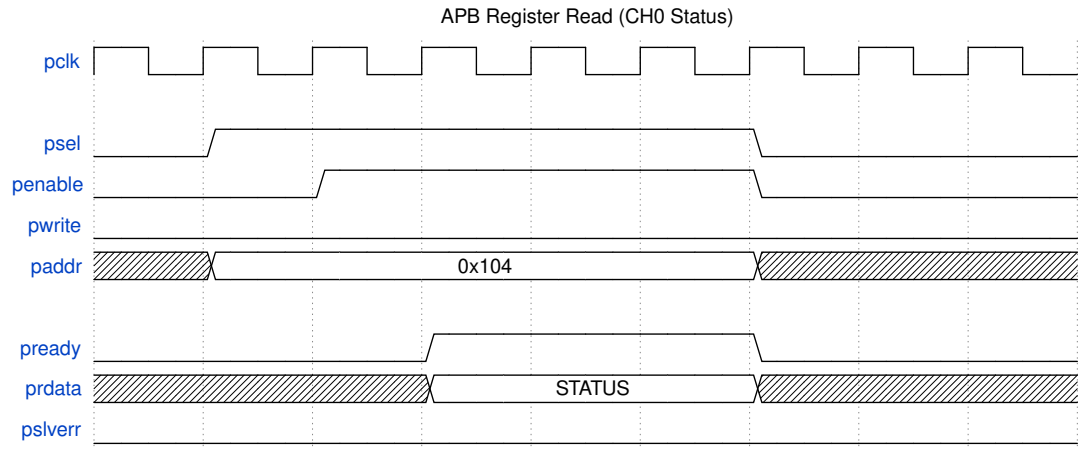
SCHED_TIMEOUT_LIMIT Register

Bits	Field	Reset	Description
[7:0]	LIMIT	4	Consecutive write-progress timeout windows before escalating to a fatal, sticky CH_ERROR. 0 = never escalate. Total escalation time ~= LIMIT x SCHED_TIMEOUT_CYCLES.

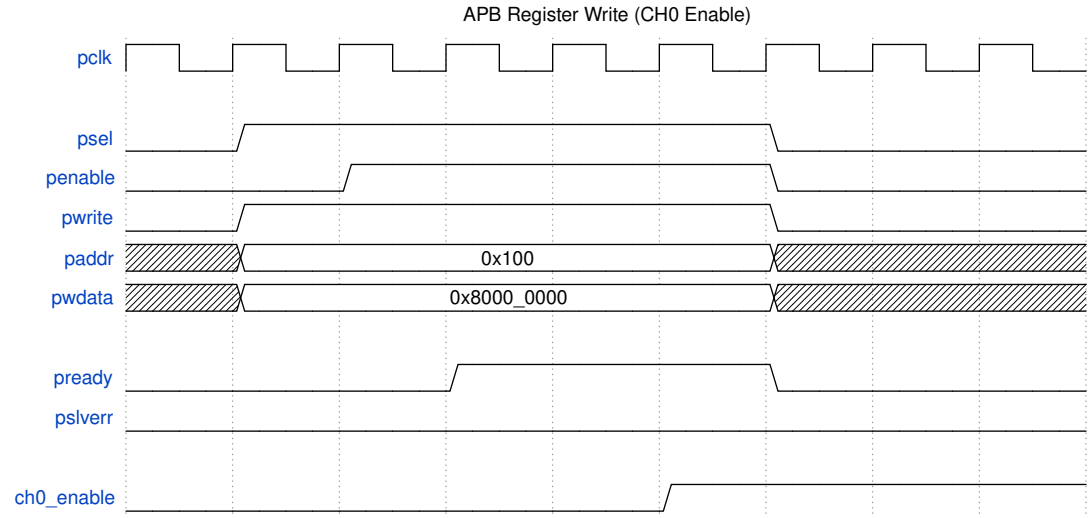
22.5.3 DESCENG_CONFIG (0x220) - Read/Write

DESCENG_CONFIG Register

Bits	Field	Reset	Description
[0]	DESCENG_EN	1	Descriptor engine master enable
[1]	PREFETCH_EN	0	Prefetch chaining (0 = on-demand, ~1 ahead)
[5:2]	FIFO_THRESH	8	Descriptors buffered ahead when prefetch enabled
[31:6]	Reserved	0	Reserved



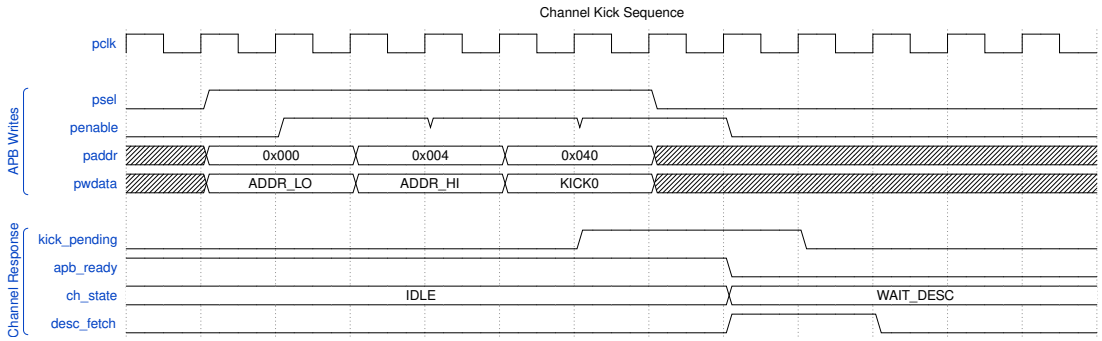
Register Access Timing



Waveform 32

22.6 Channel Kick Sequence

Staging a channel's descriptor address, then writing that channel's bit in `KICK_ENABLE`, triggers descriptor processing:



Waveform 33

22.7 Address Decoding

Registers are global (not per-channel windows); per-channel fields are packed as bit lanes within a single 32-bit register (e.g. `CHANNEL_ENABLE.CH_EN[7:0]`, `SCHEDULER_IDLE.SCHED_IDLE[7:0]`). The only per-channel array is `CH_STATE[0..7]` at 0x150-0x16C (stride 0x4). Descriptor kick-off is part of this register block: staged addresses at 0x000-0x03F (`CH*_DESC_ADDR_{LOW,HIGH}`, stride 0x8) and the `KICK_ENABLE` launch register at 0x040.

```
+0x000 - 0x03F: Staged descriptor addresses (CH*_DESC_ADDR_{LOW,HIGH})
+0x040        : KICK_ENABLE (per-channel launch, singlepulse)
+0x100 - 0x3FF: Base configuration / status registers
+0x800 - 0x9FF: Monitor configuration / performance registers
SRC half at 0x0000, SNK half at 0x1000 (paddr[12]); PSLVERR on the
monitor
window when USE_MON_REGS = 0.
```

23 Initialization Sequence

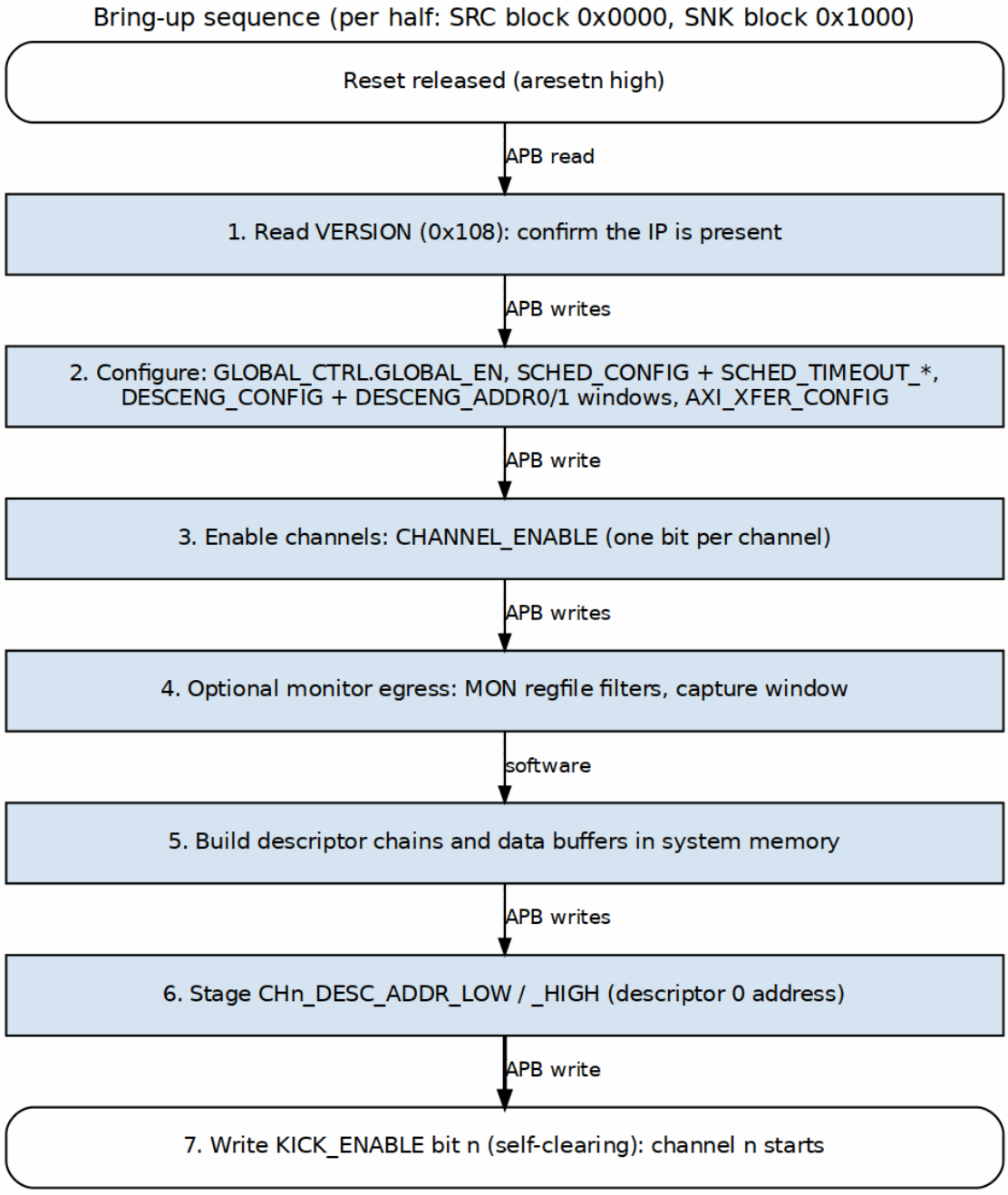
23.1 Overview

This section describes how software brings up RAPIDS Beats and starts a transfer. Every register named here is a register of the PeakRDL `rapids_engine_regs` register file (`rtl/macro_beats/rapids_engine_regs.rdl`), using the names in the generated per-instance map `rapids_regmap.py`. The two halves share one layout: **SRC at 0x0000** and **SNK at 0x1000**. Each half is initialized on its own, and the offsets in the Register Map are relative to the half.

The code below uses `reg_wr(half, NAME, value)` and `reg_rd(half, NAME)` as shorthand for an APB access to register `NAME` in the SRC or SNK half. Resolve the names through the generated map instead of hard-coding offsets.

23.2 Boot Sequence

23.2.1 Figure 5.3.1: Bring-Up Sequence



Bring-Up Sequence

Source: ch05_boot_sequence.dot

23.3 What the Hardware Requires

A channel runs a descriptor only when every one of these terms is true. They are ANDed in `rapids_config_block`, so a missing one leaves the channel idle without raising an error:

Conditions for a Channel to Run

Term	Register.field	Reset	Effect when 0
Global enable	`GLOBAL_CTRL.GLOBAL_EN`	0	Masks `CH_EN`, `SCHED_EN`, `DESCENG_EN` and every monitor enable in the half
Channel enable	`CHANNEL_ENABLE.CH_EN[n]`	0	The scheduler stays in `CH_IDLE` with a descriptor waiting
Scheduler enable	`SCHED_CONFIG.SCHED_EN`	1	All schedulers in the half are disabled
Descriptor engine enable	`DESCENG_CONFIG.DESCENG_EN`	1	No descriptor is fetched
Descriptor window	`DESCENG_ADDR0_BASE/_LIMIT`, `DESCENG_ADDR1_BASE/_LIMIT`	0 / 0xFFFFFFFF	A descriptor address outside both windows is refused and the channel errors
Kick	`KICK_ENABLE.KICKn`	–	Nothing is fetched until the bit is written

`GLOBAL_EN` and `CH_EN` both reset to 0. The other enables reset to 1, and both descriptor windows reset to the full 32-bit range, so a minimal bring-up only has to write `CHANNEL_ENABLE`, `GLOBAL_CTRL`, the staged address and `KICK_ENABLE`.

23.4 Detailed Steps

23.4.1 Step 1: Verify Hardware Presence

```
// VERSION (0x108): MINOR[7:0], MAJOR[15:8], NUM_CHANNELS[23:16]
uint32_t version = reg_rd(SRC, VERSION);
uint8_t minor    = (version >>  0) & 0xFF;   // 0x5A (90) in this
release
uint8_t major    = (version >>  8) & 0xFF;   // 0x00: pre-release,
```

```
version 0.90
uint8_t num_ch   = (version >> 16) & 0xFF;   // 8

if (num_ch == 0) {
    printf("ERROR: RAPIDS not detected\n");
    return -1;
}
```

There is no separate configuration register. The channel count is the only build parameter that software can read back. Data width and SRAM depth are not readable from the register file.

23.4.2 Step 2: Optional Reset

`GLOBAL_CTRL.GLOBAL_RST` and `CHANNEL_RESET.CH_RST[n]` are ordinary read/write bits. They do **not** self-clear. A channel is held in reset (scheduler forced to `CH_IDLE`, descriptor engine reset) for as long as its bit, or `GLOBAL_RST`, reads 1. Software must write the bit back to 0 to release it.

```
// Pulse the global reset: set, then clear. Keep GLOBAL_EN at 0
meanwhile.
reg_wr(half, GLOBAL_CTRL, GLOBAL_CTRL_GLOBAL_RST);   // bit 1
reg_wr(half, GLOBAL_CTRL, 0);

// Every scheduler should now report idle (SYSTEM_IDLE = AND of
SCHED_IDLE).
while (!(reg_rd(half, GLOBAL_STATUS) & 1)) { }
```

23.4.3 Step 3: Configure the Engine

```
// Scheduler: SCHED_EN[0] TIMEOUT_EN[1] ERR_EN[2] COMPL_EN[3]
PERF_EN[4]
reg_wr(half, SCHED_TIMEOUT_CYCLES, 1000);   // write-progress window,
in clocks
reg_wr(half, SCHED_TIMEOUT_LIMIT,  4);      // fatal after 4 stalled
windows; 0 = never
reg_wr(half, SCHED_CONFIG, SCHED_EN | TIMEOUT_EN | ERR_EN | COMPL_EN);

// Descriptor engine: DESCENG_EN[0] PREFETCH_EN[1] FIFO_THRESH[5:2]
reg_wr(half, DESCENG_CONFIG, DESCENG_EN | PREFETCH_EN | (4 << 2));

// Descriptor address windows (lower 32 bits). A fetch address outside
// both windows is refused and the channel enters CH_ERROR.
reg_wr(half, DESCENG_ADDR0_BASE,  desc_region_base);
reg_wr(half, DESCENG_ADDR0_LIMIT, desc_region_base + desc_region_size
- 1);

// Burst sizing: RD_XFER_BEATS[7:0] WR_XFER_BEATS[15:8]
//               ALLOC_SIZE[23:16]  DRAIN_SIZE[31:24]
reg_wr(half, AXI_XFER_CONFIG, (16 << 24) | (16 << 16) | (8 << 8) | 8);
```

The SRC half uses the read-side fields and the SNK half uses the write-side fields. Writing the whole word in both halves does no harm.

23.4.4 Step 4: Optional Monitor and Interrupt Egress

The only interrupt output is `mon_irq`. It comes from the top-level `monbus_axil4_axil4_group` and is asserted while that group's error FIFO is not empty. The scheduler error and completion packets travel on the CORE protocol. For both halves, the group takes its CORE filter from the **SRC** half's `WRMON_*` registers:

```
// CORE packets: PKT_MASK bit[type] = 1 DROPS that type; reset is
0xFFFF.
reg_wr(SRC, WRMON_PKT_MASK, ~((1u << 0) | (1u << 1)) & 0xFFFF);  //
pass Error, Completion


// ERR_SELECT[3:0] bit[type] = 1 routes that type to the error FIFO (-
> mon_irq).
// Bit 0 = Error.  Bit 1 = Completion, which includes the gen_irq
descriptor event.
reg_wr(SRC, WRMON_ERR_CFG, (0xFF << 8) | (1u << 0));             //
keep ERR_MASK reset value
```

Packets that are passed but not selected for the error FIFO go to the capture path (`m_axil_mon_*`). The group, `mon_irq` and the `s_axil_err_*` drain exist only when `USE_AXI_MONITORS = 1`. With `USE_AXI_MONITORS = 0`, `mon_irq` is tied to 0, and software must poll `SCHED_ERROR` and `SCHEDULER_IDLE` instead.

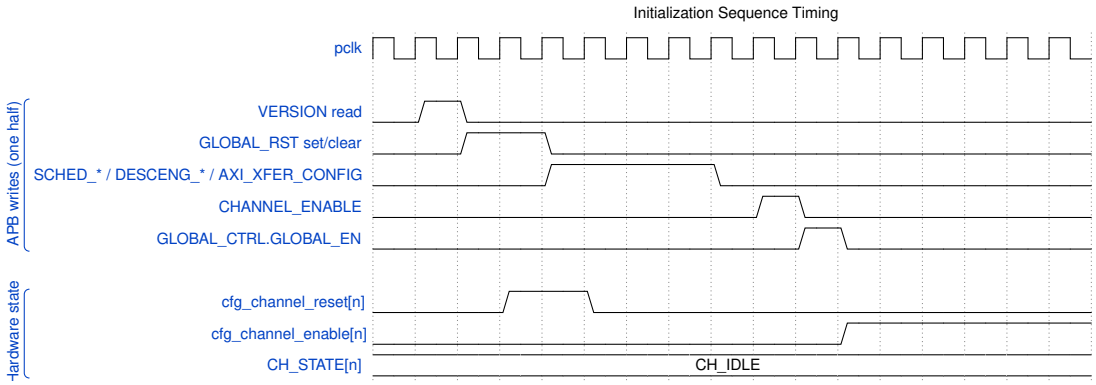
23.4.5 Step 5: Build Descriptors

```
// rapids_pkg::descriptor_t, 256 bits (see Descriptor Format)
for (int i = 0; i < NUM_RX_DESC; i++) {
    desc[i].src_addr           = 0;                          // unused
by a sink transfer
    desc[i].dst_addr           = rx_buffer + i * FRAME_SIZE;
    desc[i].length             = FRAME_LEN;   // BEATS in RAPIDS
Beats, BYTES in byte-granular RAPIDS
    desc[i].next_descriptor_ptr = (i < NUM_RX_DESC - 1) ?
(uint32_t)&desc[i + 1] : 0;
    desc[i].valid              = 1;                          // 0 ->
CH_ERROR
    desc[i].last               = (i == NUM_RX_DESC - 1);
    desc[i].gen_irq            = 1;                          //
completion event RAPIDS_EVENT_IRQ
}
__sync_synchronize();   // descriptors visible before the kick
```

The descriptors must lie inside a `DESCENG_ADDR0`/`DESCENG_ADDR1` window.

23.4.6 Step 6: Enable

```
reg_wr(half, CHANNEL_ENABLE, 1u << ch);          // CH_EN[n]
reg_wr(half, GLOBAL_CTRL,    GLOBAL_CTRL_GLOBAL_EN);   // bit 0;
GLOBAL_RST stays 0
```



Initialization Timing

`cfg_channel_enable[n]` is `CH_EN[n] & GLOBAL_EN`, so it asserts only after the last of the two writes.

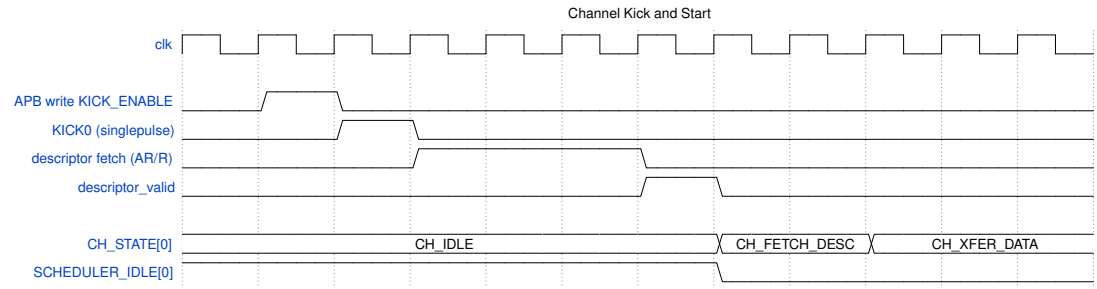
23.5 Starting Transfers

23.5.1 Kick a Channel

Stage the descriptor address, then launch it with a single write to `KICK_ENABLE`. The address registers only store the value, so writing them does not start anything. Each `KICKn` bit is a singlepulse field and clears itself. One 32-bit write can launch several channels on the same clock.

```
reg_wr(half, CH0_DESC_ADDR_LOW,  (uint32_t)((uint64_t)&desc[0]));
reg_wr(half, CH0_DESC_ADDR_HIGH, (uint32_t)((uint64_t)&desc[0] >>
32));
reg_wr(half, KICK_ENABLE, 1u << 0);              // KICK0
```

The APB write completes as soon as the request is raised. The RTL holds the request until the descriptor engine accepts it, so a completed write does not mean the engine has accepted the kick.



Waveform 35

23.6 Completion and Interrupt Handling

There is no interrupt status register and no per-channel interrupt clear. Software sees completion in one of two ways:

- **Polling.** `SCHEDULER_IDLE.SCHED_IDLE[n]` returns to 1 when the scheduler is back in `CH_IDLE` after the last descriptor of a chain. A channel in `CH_ERROR` never reports idle, and `SCHED_ERROR.SCHED_ERR[n]` reads 1 for it instead. Poll both registers, or the loop will hang on a faulted channel. Do not poll `CHANNEL_IDLE`, `AXI_WR_COMPLETE`, or the SNK half's `AXI_RD_COMPLETE`. Those fields are not driven at the top level and always read 0 (see the Register Map).
- **mon_irq.** If Step 4 routed Completion packets to the error FIFO, every descriptor completion lands there, and so does every error. A completion is reported as `RAPIDS_EVENT_DESC_COMPLETE`, or as `RAPIDS_EVENT_IRQ` when the descriptor's `gen_irq` is set. The descriptor engine's descriptor-loaded completions land there too. `mon_irq` stays high until the records are drained over `s_axil_err_*`.

```
void rapids_irq_handler(void) {
    // mon_irq == error FIFO not empty: drain every record
    while (mon_irq_pending()) {
        mon_record_t r = err_fifo_read();        // s_axil_err_* reads
        if (r.pkt_type == PKT_TYPE_ERROR) {
            handle_channel_error(r.half, r.channel_id);   // see Error
Handling
        } else if (r.pkt_type == PKT_TYPE_COMPLETION) {
            process_completed_buffer(r.half, r.channel_id,
r.event_data);
        }
    }
}
```

To run the channel again, stage the next chain's first descriptor address and write its KICK_ENABLE bit. No re-enable is needed, because CH_EN stays set.

23.7 Reset and Recovery

23.7.1 Per-Channel Reset

```
void reset_channel(int half, int ch) {
    // Assert, then release: CH_RST is NOT self-clearing.
    reg_wr(half, CHANNEL_RESET, 1u << ch);
    reg_wr(half, CHANNEL_RESET, 0);

    while (!(reg_rd(half, SCHEDULER_IDLE) & (1u << ch))) { }
    // SCHED_ERROR[ch] is the CH_ERROR state, so it reads 0 again
here.
}
```

The scheduler's error state is sticky. The only ways out of CH_ERROR are a channel reset (CH_RST[n]) or a global reset (GLOBAL_RST). The reset leaves CH_EN[n] and the staged descriptor address unchanged. After the reset, re-stage the address if it should change, then kick.

23.7.2 Global Reset

```
void reset_rapids_half(int half) {
    reg_wr(half, GLOBAL_CTRL, GLOBAL_CTRL_GLOBAL_RST);   // GLOBAL_EN
= 0, RST = 1
    reg_wr(half, GLOBAL_CTRL, 0);                        // release
    while (!(reg_rd(half, GLOBAL_STATUS) & 1)) { }       //
SYSTEM_IDLE
    rapids_init_half(half);                              // Steps 3-6
again
}
```

GLOBAL_RST drives every channel's reset in that half only. The other half, and the shared monitor group's error FIFO, are not affected.

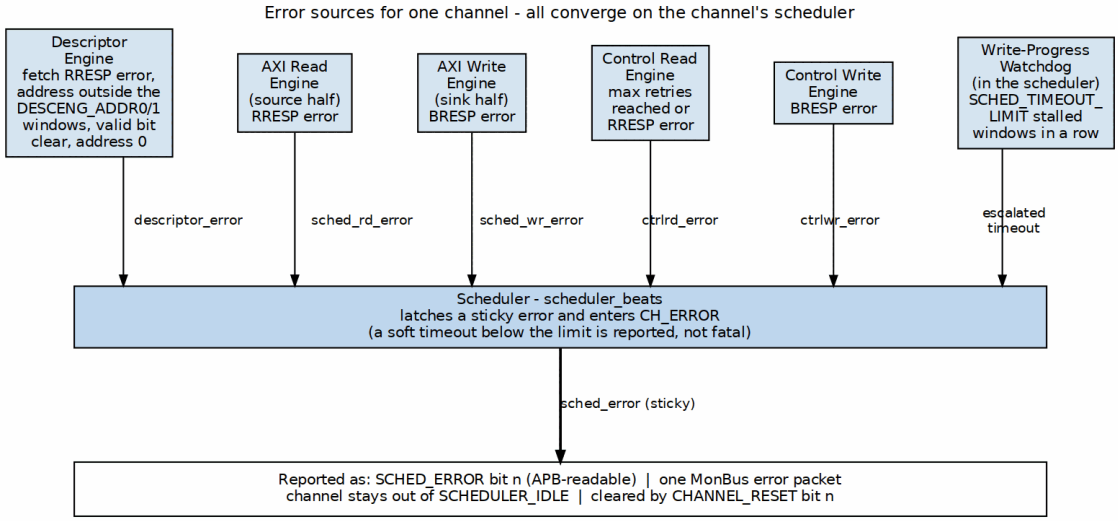
24 Error Handling

24.1 Overview

This section covers how RAPIDS Beats detects an error, how it reports one, and how software recovers. Every fatal error in a channel ends the same way. The channel's scheduler latches it and moves to the sticky CH_ERROR state, and it stays there until software resets the channel. No per-channel error-status register holds an error code. Software has three sources of information: the SCHED_ERROR bit, the CH_STATE readback, and the MonBus error packet.

24.2 Error Categories

24.2.1 Figure 5.4.1: Error Sources



Error Sources

Source: ch05_error_sources.dot

24.3 Error Detection

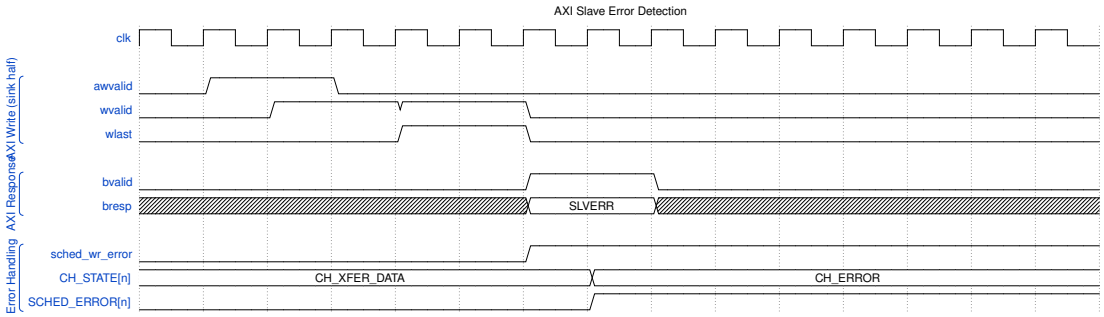
24.3.1 AXI Response Errors

RAPIDS checks the response on every AXI transaction it issues: descriptor fetches, source-half data reads, sink-half data writes, and the control read/write engines.

AXI Response Codes

Response	Code	Description	Typical Cause
OKAY	2'b00	Normal	–

Response	Code	Description	Typical Cause
		success	
EXOKAY	2’b01	Exclusive OK	–
SLVERR	2’b10	Slave error	Memory protection, invalid access
DECERR	2’b11	Decode error	Address not mapped



Waveform 36

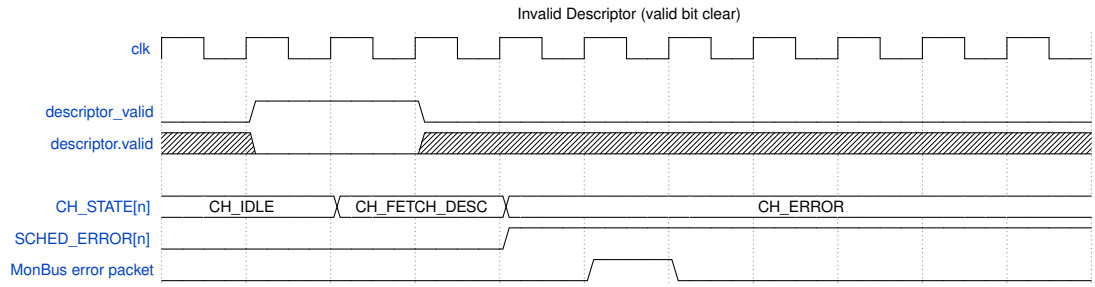
24.3.2 Descriptor Errors

The descriptor engine and the scheduler apply a small, fixed set of checks. None of them has its own status code. Each one sets the channel’s `SCHED_ERROR` bit, and the descriptor-engine checks also emit a MonBus error packet.

Descriptor Checks

Check	Where	Error condition
Address window	Descriptor engine	The fetch address is outside both `DESCENG_ADDR0_BASE..DESCENG_ADDR0_LIMIT` and `DESCENG_ADDR1_BASE..DESCENG_ADDR1_LIMIT`. The fetch is refused and never issued.
Null pointer	Descriptor engine	The descriptor address is 0
Fetch response	Descriptor engine	`RRESP` is not OKAY on the descriptor read
Valid bit	Scheduler (`CH_FETCH_DESC`)	The descriptor’s `valid` bit is 0

The hardware does not check length, alignment, or direction. A descriptor with a zero or oversized length is software's responsibility. The software validation example at the end of this section shows one way to catch it.



Waveform 37

24.3.3 Timeout Detection

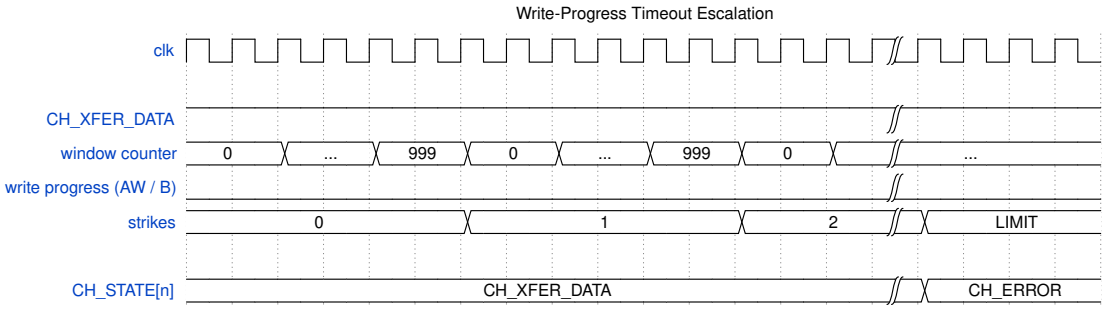
Only the scheduler's write-progress watchdog can put a channel into `CH_ERROR` by itself. The control-read retry budget has a similar effect. The AXI monitors' timeouts only report.

Timeout Mechanisms

Mechanism	Register	Reset value	Result
Write-progress watchdog	`SCHED_TIMEOUT_CYCLES.TIMEOUT_CYCLES` (clocks per window), `SCHED_TIMEOUT_LIMIT.LIMIT` (windows)	1000, 4	Fatal `CH_ERROR` after `LIMIT` consecutive stalled windows. With `LIMIT = 0` it never escalates. Enabled by `SCHED_CONFIG.TIMEOUT_EN`.
Control-read poll budget	`CTRL_CONFIG.CTRLRD_MAX_TRY`	16	A `CTRL_READ` descriptor that does not match within the budget raises `ctrlrd_erro`

Mechanism	Register	Reset value	Result
			r, and the channel goes to CH_ERROR
Monitor transaction timeout	DAXMON_TIMEOUT, RDMON_TIMEOUT, WRMON_TIMEOUT	–	Emits a MonBus Timeout packet when the monitor's TIMEOUT_EN is set. The channel state does not change.

Recoverable write-progress timeout. Each window of SCHED_TIMEOUT_CYCLES clocks that expires without write progress adds one strike to a counter. The channel goes to the fatal, sticky CH_ERROR only after SCHED_TIMEOUT_LIMIT (register 0x208) consecutive stalled windows. Any write progress, either an AW issue or a B-response commit, clears the strike counter. A limit of 0 disables escalation entirely, which makes the timeout soft. A channel in CH_ERROR is reported through SCHED_ERROR and is deliberately left out of SCHEDULER_IDLE.



Waveform 38

24.4 Error Reporting

24.4.1 Status Registers

Error-Related Status Registers

Register	Meaning on error
SCHED_ERROR.SCHED_ERR[n]	1 while channel n is in CH_ERROR. The bit reads the

Register	Meaning on error
	state directly and clears only on channel reset.
CH_STATE[n].STATE	One-hot scheduler state. CH_ERROR = 7'b0100000.
SCHEDULER_IDLE.SCHED_IDLE[n]	0 for a faulted channel, because CH_ERROR is not idle
GLOBAL_STATUS.SYSTEM_IDLE	0 while any channel in the half is faulted

The register file does not record which response code or which descriptor caused the error. That detail travels only in the MonBus error packet.

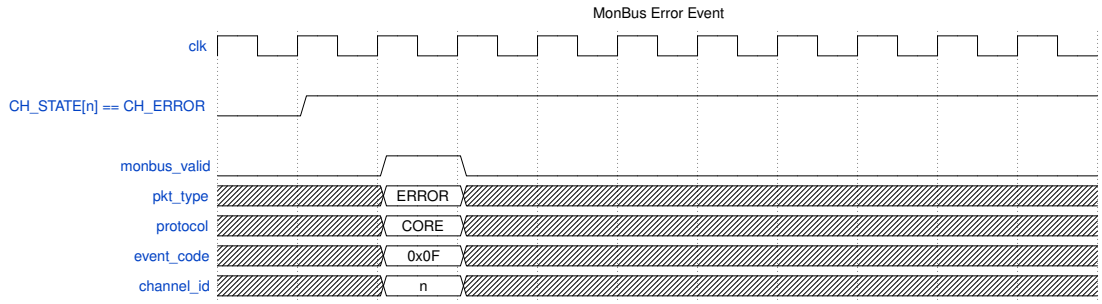
24.4.2 MonBus Error Events

Each error episode emits one error packet (PktTypeError, CORE protocol). The packet carries the half's agent ID and the channel ID:

MonBus Error Packets

Emitter	Event code	Event data
Scheduler (on entering CH_ERROR)	RAPIDS_EVENT_ERROR (0x0F)	Read-side and write-side sticky error flags. Both clear means the cause was the descriptor engine, a control engine, or the timeout.
Descriptor engine	CORE_ERR_DESCRIPTOR_ENGINE	The descriptor fetch RRESP

SCHED_CONFIG.ERR_EN and COMPL_EN do not gate error packets. The scheduler re-arms its one-shot error packet when the channel returns to CH_IDLE.

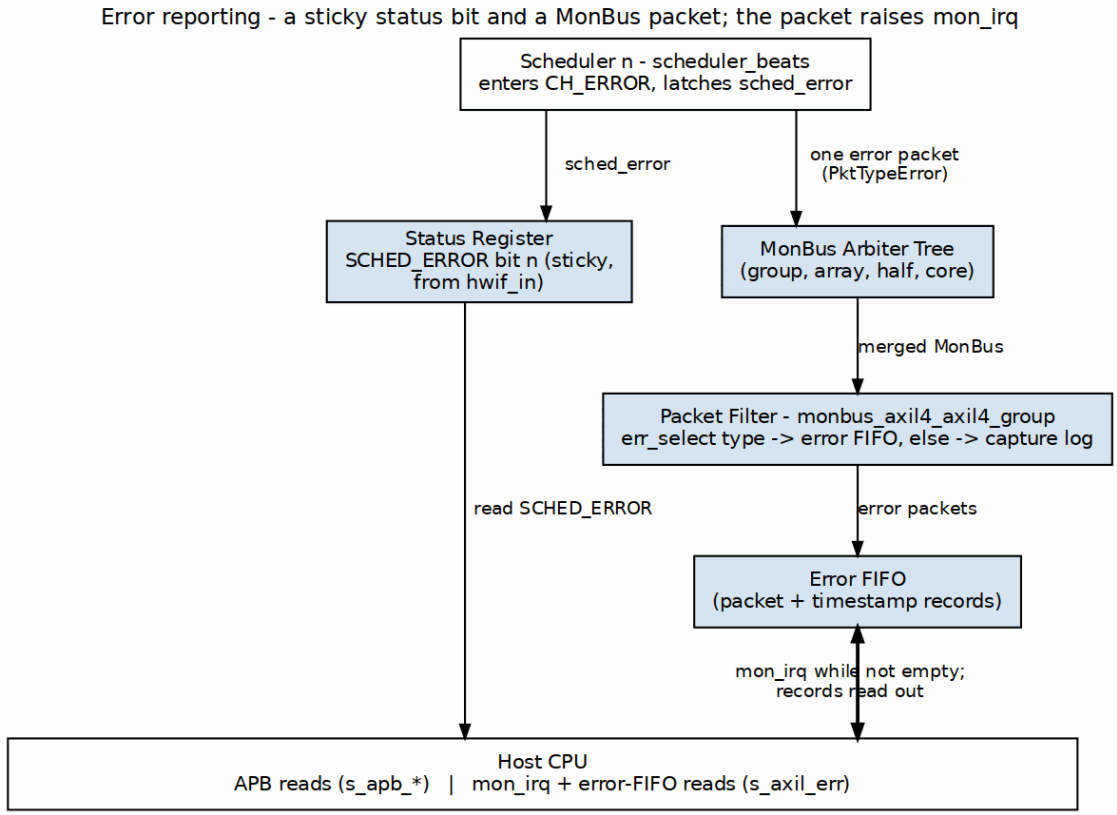


Waveform 39

24.4.3 Interrupt Generation

There is no separate interrupt status register. A channel error sets that channel's sticky `SCHED_ERROR` bit and emits one MonBus error packet. The top-level monitor group routes the packet types that its `err_select` configuration selects into its error FIFO. For CORE packets that configuration is `SRC.MON.WRMON_ERR_CFG.ERR_SELECT`, and bit 0 selects Error. `SRC.MON.WRMON_PKT_MASK` bit 0 must also be clear, because the reset value masks every type. `mon_irq` is asserted while the error FIFO is not empty. Software reads the records out over `s_axil_err`:

24.4.4 Figure 5.4.2: Error Reporting



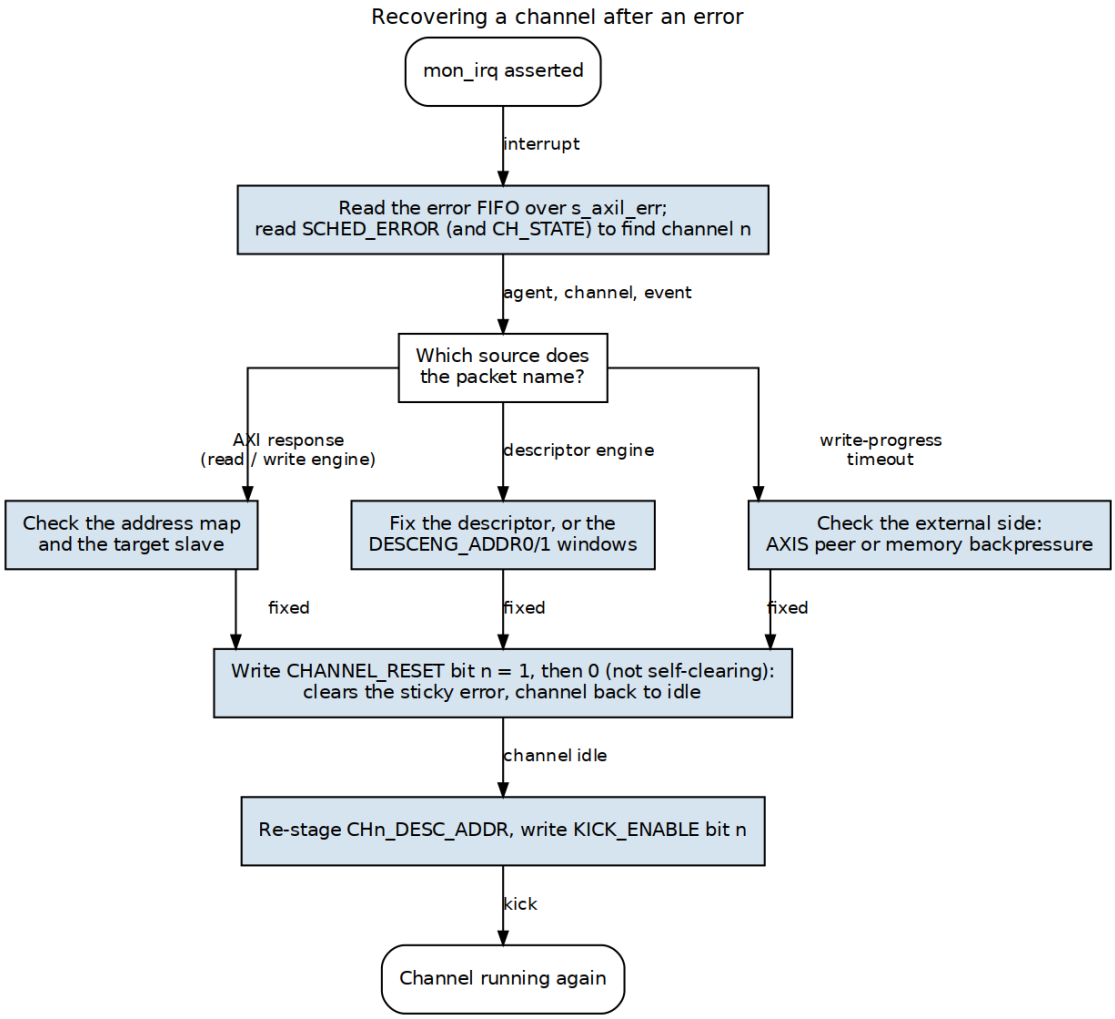
Error Reporting

Source: ch05_error_reporting.dot

24.5 Error Recovery

24.5.1 Recovery Flow

24.5.2 Figure 5.4.3: Error Recovery



Error Recovery

Source: ch05_error_recovery.dot

Every recovery ends the same way. Fix the cause, then reset the channel. Set its `CHANNEL_RESET` bit and clear it again, because the bit is not self-clearing. Re-stage the descriptor address and kick. `reset_channel()` is defined in Initialization.

24.5.3 Finding the Faulted Channel

```
void handle_errors(int half) {
    uint32_t faulted = reg_rd(half, SCHED_ERROR) & 0xFF;    //
```

```
SCHED_ERR[7:0]

    for (int ch = 0; ch < 8; ch++) {
        if (!(faulted & (1u << ch)))
            continue;
        mon_record_t r = find_error_record(half, ch);   // from the
s_axil_err drain
        recover_channel(half, ch, &r);
    }
}
```

24.5.4 AXI Error Recovery

```
void recover_axi_error(int half, int ch, const mon_record_t *r) {
    // The scheduler packet says which side failed; the address is not
latched.
    if (r->rd_err)
        printf("%s ch%d: read-side AXI error\n", half_name(half), ch);
    if (r->wr_err)
        printf("%s ch%d: write-side AXI error\n", half_name(half),
ch);
    // Check the address map and the target slave, then:
    reset_channel(half, ch);
    restage_and_kick(half, ch, first_desc_addr[half][ch]);
}
```

24.5.5 Descriptor Error Recovery

```
void recover_desc_error(int half, int ch, uint64_t bad_desc) {
    // Window refusal, null pointer, fetch RRESP error, or valid bit
clear.
    // Fix the descriptor (or widen DESCENG_ADDR0/1) before re-
kicking.
    fix_descriptor(bad_desc);
    reset_channel(half, ch);
    restage_and_kick(half, ch, bad_desc);
}

void restage_and_kick(int half, int ch, uint64_t addr) {
    reg_wr(half, CH_DESC_ADDR_LOW(ch),  (uint32_t)addr);          //
CHn_DESC_ADDR_LOW
    reg_wr(half, CH_DESC_ADDR_HIGH(ch), (uint32_t)(addr >> 32));  //
CHn_DESC_ADDR_HIGH
    reg_wr(half, KICK_ENABLE, 1u << ch);                          //
KICKn
}
```

24.5.6 Timeout Recovery

```
void recover_timeout(int half, int ch) {
    // Escalated write-progress timeout: the write side stopped making
progress
    // for SCHED_TIMEOUT_LIMIT windows. In the SNK half that usually
means the
    // AXIS peer stopped sending or memory stopped accepting writes.
In the SRC
    // half, check m_axis_tready.
    printf("%s ch%d: write-progress timeout\n", half_name(half), ch);
    reset_channel(half, ch);
    restage_and_kick(half, ch, first_desc_addr[half][ch]);
}
```

To ride out long stalls without a fatal error, raise `SCHED_TIMEOUT_CYCLES` or `SCHED_TIMEOUT_LIMIT`, or set the limit to 0. The timeout is then reported but never escalates.

24.6 Error Prevention

24.6.1 Best Practices

Error Prevention Best Practices

Practice	Rationale
Validate descriptors before submission	The hardware checks only the window, a null pointer, and the `valid` bit
Set `DESCENG_ADDR0/1` to the real descriptor region	A stray `next_descriptor_ptr` is refused rather than fetched
Size `SCHED_TIMEOUT_CYCLES` x `SCHED_TIMEOUT_LIMIT` above the worst legitimate stall	Avoids false fatal timeouts
Route Error packets to the error FIFO	`mon_irq` reports errors without polling

24.6.2 Descriptor Validation Example

```
int validate_descriptor(const rapids_descriptor_t *desc) {
    if (!desc->valid)
        return ERR_NOT_VALID;           // the scheduler would fault
the channel
    if (desc->length == 0 || desc->length > MAX_BEATS)
        return ERR_LENGTH;              // not checked by hardware
    if (desc->next_descriptor_ptr &&
        !in_desc_window(desc->next_descriptor_ptr))
        return ERR_ADDRESS;             // the descriptor engine
```

```
would refuse it
    return 0;
}
```

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

25 Throughput Targets

RAPIDS Beats is a network-to-memory accelerator: the **sink** path moves an AXIS ingress stream into system memory, and the **source** path reads system memory and drives an AXIS egress stream. The two paths are **independent engines** (with their own SRAM buffers, schedulers, and AXI masters), so a sink transfer and a source transfer run concurrently and their bandwidths add.

25.1 Theoretical Maximum

A beat is `DATA_WIDTH = 512` bits = 64 bytes. Per-direction bandwidth is `DATA_WIDTH x f_aclk / 8`.

25.1.1 Per-Interface Bandwidth

Interface	Data Width	Frequency	Bandwidth
Descriptor Fetch	256 bits	100 MHz	3.2 GB/s
Sink AXI Write (to memory)	512 bits	100 MHz	6.4 GB/s
Source AXI Read (from memory)	512 bits	100 MHz	6.4 GB/s
AXIS ingress / egress (network)	512 bits/beat	100 MHz	6.4 GB/s

The table uses the Nexys A7 characterization clock (`CLK100MHZ`, 100 MHz). The design targets 100-500 MHz; scale linearly (e.g. 12.8 GB/s per direction at 200 MHz, 32 GB/s at 500 MHz).

25.1.2 Aggregate Bandwidth

- **Concurrent sink + source:** 12.8 GB/s at 100 MHz (6.4 sink + 6.4 source), since the two directions are separate engines against separate AXI masters.
- **Descriptor Overhead:** ~0.1% for large transfers; the descriptor engine prefetches (`DESCENG_CONFIG.PREFETCH_EN`) so chained fetches overlap the current transfer.

25.2 Practical Throughput

A transfer is bounded by the slower of its two ends: a sink transfer by `min(AXIS ingress, AXI write)`, a source transfer by `min(AXI read, AXIS egress)`. The SRAM buffer decouples the two ends so a transient stall on one side does not immediately stall the other.

25.2.1 Single-Channel Performance

Scenario	Expected Efficiency	Effective Bandwidth (100 MHz)
Large sequential (>1MB)	>95%	>6.1 GB/s per direction
Medium sequential (64KB-1MB)	85-95%	5.4-6.1 GB/s
Small sequential (4KB-64KB)	70-85%	4.5-5.4 GB/s
Very small (<4KB)	40-70%	2.6-4.5 GB/s

25.2.2 Multi-Channel Performance

All channels of one direction share that direction's AXI master, so aggregate is bounded by the shared master while per-channel throughput scales inversely with active channel count.

Active Channels (per direction)	Per-Channel BW	Aggregate BW (per direction)
1	6.4 GB/s	6.4 GB/s
2	3.2 GB/s	6.4 GB/s
4	1.6 GB/s	6.4 GB/s
8	0.8 GB/s	6.4 GB/s

Note: figures are at 100 MHz for a single direction. Sink and source run concurrently, so full-duplex aggregate is twice the per-direction column.

25.3 Throughput Limiting Factors

25.3.1 Memory / Network System Factors

Factor	Impact	Mitigation
Memory latency	Reduces efficiency for small transfers	Deeper outstanding / larger SRAM

Factor	Impact	Mitigation
Memory or network bandwidth	Hard limit on the slower end	Match RAPIDS to the memory/network capability
AXIS backpressure	Stalls the network end (`s_axis_tready`/`m_axis_tready`)	SRAM buffering absorbs transient stalls
Interconnect contention	Variable impact	Priority / channel configuration

25.3.2 RAPIDS Internal Factors

Factor	Impact	Mitigation
Descriptor fetch overhead	Fixed per descriptor	Longer transfers; descriptor prefetch (`PREFETCH_EN`)
SRAM depth	Limits outstanding beats	Configure adequate `SRAM_DEPTH`
Arbitration overhead	Multi-channel penalty on the shared AXI master	Reduce active channels per direction
AXI transfer sizing	Small bursts lose efficiency	`AXI_XFER_CONFIG` RD/WR beats, ALLOC/DRAIN sizing
Control descriptors	`CTRL_READ`/ `CTRL_WRITE` are synchronization, not payload	Use only for producer/consumer handshakes; see below

Control descriptors do not carry throughput. A `CTRL_READ` (consumer gate) polls a memory location until it matches; a `CTRL_WRITE` (producer doorbell) issues one 32-bit write. They synchronize a channel with a producer/consumer and occupy the channel while active, but move no payload – budget them as latency, not bandwidth (see Latency, Section 6.2).

25.4 Performance Targets

25.4.1 Design Targets

Metric	Target	Condition
Peak throughput (per direction)	6.4 GB/s	Single channel, 100 MHz, ideal
Full-duplex aggregate	12.8 GB/s	Sink + source concurrent, 100 MHz
Sustained throughput	>6.1 GB/s	Large transfers, single channel
Small transfer efficiency	>50%	4KB transfers

25.4.2 Verification Criteria

Scenario	Pass Criteria
Sink stream (large)	>95% of AXI write bandwidth
Source stream (large)	>95% of AXI read bandwidth
Multi-channel (per direction)	Aggregate within 5% of the shared master limit
Data integrity (all cases)	`wr_crc`/`rd_crc`/`chk_crc` == golden model

25.5 On-Silicon Validation

The characterization harness (`rapids_char_top` on the Nexys A7-100T, 100 MHz) validates **data integrity** end to end via golden-CRC self-checks on both paths: sink `wr_crc == golden`, source `rd_crc == chk_crc == golden`. The current harness measures correctness and beat counts.

Per-direction bus utilization is measured IN THE DESIGN, not by an external observer. `rapids_beats_top` instantiates the shared `axi_bus_meter` (`rtl/amba/shared/`) twice – `u_rd_bus_meter` on the read master and `u_wr_bus_meter` on the write master – giving the true per-direction split (STREAM's shared master is aggregate-only). Aggregate PROD/BP/STARV/IDLE plus beat/byte/burst counts surface through the `RDMON_PERF_*` / `WRMON_PERF_*` CSRs, and per-channel buckets through `SRC.PERF_CH_SEL` / `SNK.PERF_CH_SEL` with `RDMON_PERF_CH_*` / `WRMON_PERF_CH_*`.

Adopting `axi4_intf_master_observer` here was considered and rejected (RAPIDS TASK-001, 2026-09-24). The observer is a packaging of that same `axi_bus_meter` plus a latency histogram and monbus taps; since the meters are already the shared primitive, it buys no measurement RAPIDS lacks while costing +208% LUTs at its smallest legal config – the same measured ground on which pumice rejected it (PUMICE-016). The one thing it would add is

`axi_perf_latency_hist`, which RAPIDS does not instantiate; that is a separate question from this one.

As of 2026-08-05 the observer carries its OWN APB config regblock (`obs_regs`) instead of exporting 29 `cfg_*` ports for the instantiating harness to tie off, and it moved to `projects/components/utility-ip/misc/rtl/` so any board flow can reach it:

```
-f $MISC_ROOT/rtl/filelists/axi4_intf_master_observer.f
```

It was renamed from `axi4_dma_observer` at the same time – the header always said "DMA-agnostic", and the DMA in the name read wrong for a block a memory controller or a beats DMA would share. `dma_slave_monitors` is **retired**: module, filelist and its `slvmon_regs` regblock were deleted 2026-09-20 (STREAM TASK-073). The slave-side equivalent is `axi4_intf_slave_observer` + `obs_regs`.

Latest on-silicon result (full characterization suite): **48 / 48 configurations pass** across channels {1, 2, 4} x beats {1, 4, 8, 16} x backpressure {off, on} x seeds {default, alternate}, sink and source, all golden-validated – including under injected AXIS backpressure.

Last Updated: 2026-07-13

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

26 Latency Characteristics

Latency is dominated by the one-time pipeline fill from a channel kick to the first data beat. Once running, both paths sustain the throughput of Section 6.1; control descriptors add a bounded synchronization delay.

26.1 Latency Components

26.1.1 Kick-to-First-Beat Breakdown

Stage	Typical Cycles (100 MHz)	Notes
Kick decode (APB write to `KICK_ENABLE`)	TBD – re-measure	Address is staged beforehand; the launch request is then held until the descriptor engine accepts it
Descriptor fetch (AXI read, 256-bit)	AXI read latency + 1	Overlapped by prefetch on chains
Descriptor parse	1-2	Opcode + fields decoded
SRAM fill (sink) / AXI read issue (source)	AXI/AXIS latency	First beat into the buffer
First beat out	1-2	Drain to memory (sink) / AXIS (source)

26.1.2 Total Latency Formula

```
L_first_beat  =  L_kick + L_desc_fetch + L_parse + L_pipeline_fill
```

Note: L_kick predates the staged-address/KICK_ENABLE kick and has not been re-measured. The previous figure came from a path that stalled APB until the descriptor engine accepted the kick; acceptance is now asynchronous, so the term is no longer bounded by an APB stall.

L_desc_fetch is hidden on chained descriptors when the descriptor engine has prefetched the next descriptor (DESCENG_CONFIG.PREFETCH_EN).

26.2 First-Beat Latency

The dominant term is the memory/network round trip to fill the pipeline. For a sink transfer the first AXIS beat must arrive and be buffered before the first AXI write can drain; for a source transfer the first AXI read must return before the first AXIS egress beat. After the first beat the pipeline is full and throughput is bandwidth-bound, not latency-bound.

26.3 Control-Descriptor Latency

Control descriptors are latency, not throughput (Section 6.1):

Opcode	Latency
CTRL_WRITE (doorbell)	One single-beat AXI write: AW->W->B round trip
CTRL_READ (gate)	One poll per retry (AXI read + compare); retries paced by tick_1us, up to CTRL_CONFIG.CTRLRD_MAX_TRY (default 16, max 511)

A CTRL_READ gate blocks its descriptor (and the chain) until the polled value matches. The retry budget bounds the worst case: an unsatisfied gate cannot hang the channel – after CTRLRD_MAX_TRY polls the engine raises ctrlrd_error instead of waiting forever.

26.4 Latency Variability

26.4.1 Sources of Variability

- Memory read latency (descriptor fetch, source reads) and write-response timing.

- AXIS backpressure (`s_axis_tready` / `m_axis_tready`) on the network end.
- Multi-channel arbitration for the shared AXI master.
- `CTRL_READ` retry count when a gate is not yet satisfied.

26.4.2 Worst-Case Latency

Bounded by: memory timeout (`SCHED_TIMEOUT_CYCLES` / `SCHED_TIMEOUT_LIMIT` escalation on the scheduler side) and the control-read retry budget (`CTRL_CONFIG.CTRLRD_MAX_TRY`). Both convert an otherwise-unbounded wait into a reported error so a channel is never permanently stalled.

26.5 Latency Optimization

26.5.1 Software Recommendations

- Chain descriptors so the descriptor engine prefetches the next while the current transfers (hides `L_desc_fetch`).
- Keep the AXIS consumer/producer ready to avoid backpressure stalls.
- Size `CTRL_READ` gates so the expected value is already (or soon) present to minimize poll retries.

26.5.2 Hardware Configuration

- Adequate `SRAM_DEPTH` so the buffer covers memory/network latency without stalling.
- Tune `AXI_XFER_CONFIG` (RD/WR beats, ALLOC/DRAIN) so bursts amortize AXI setup.

Last Updated: 2026-07-13

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

27 Resource Estimates

Unlike the theoretical throughput and latency numbers above, the resource figures here are **measured** from the actual place-and-route of the characterization build (`rapids_char_harness` wrapping `rapids_beats_top`) on the Nexys A7-100T.

27.1 FPGA Resource Summary

Measured, post-implementation (Vivado 2025.1, `xc7a100t-csg324-1`, 100 MHz, characterization build: `NUM_CHANNELS = 4`, `DATA_WIDTH = 512`, `ADDR_WIDTH = 64`, `APB_ADDR_WIDTH = 13`):

Resource	Used	Available	Utilization
Slice LUTs	37,555	63,400	59.24%
– LUT as Logic	30,205	63,400	47.64%
– LUT as Memory	7,350	19,000	38.68%
Slice Registers (FF)	28,683	126,800	22.62%
Block RAM Tile	22	135	16.30%
– RAMB36	20	135	–
– RAMB18	4	270	–
DSP48E1	0	240	0%

Timing: all user-specified timing constraints met at 100 MHz (positive slack) – the harness plus DUT close cleanly on the -1 speed grade part.

The characterization build uses 4 channels (sink + source) to fit the harness (BRAM SRAM buffers, golden-CRC checkers, UART bridge) comfortably on the 100T. The design parameter default is `NUM_CHANNELS = 8`; an 8-channel build roughly scales the per-channel SRAM/scheduler logic and would target a larger part.

27.2 Resource Breakdown by Block

Approximate distribution of the DUT logic (excludes the char harness's UART/CRC scaffolding). Both a sink and a source instance are present; they do not share logic.

27.2.1 APB Configuration Slave + Register Block

- LUTs: ~1.5K
- FFs: ~2K

- Function: APB decode, PeakRDL `rapids_regs` block (config + monitor regfile @ 0x1000), `rapids_config_block` hwif fan-out.

27.2.2 Scheduler Group (per direction)

- LUTs: ~3-4K per direction
- FFs: ~3K per direction
- Function: `scheduler_beats` (three-opcode: DATA / CTRL_READ / CTRL_WRITE), `descriptor_engine_beats` (256-bit fetch/parse + prefetch), channel arbitration.

27.2.3 Control Engines (per direction)

- LUTs: ~0.5K per direction
- FFs: ~0.5K per direction
- Function: `ctrlrd_engine` (poll-until-match, retry budget) and `ctrlwr_engine` (single-beat doorbell write). Small – one AXI read/write state machine each.

27.2.4 AXI Read / Write Engines

- LUTs: ~2-3K each
- FFs: ~2-3K each
- Function: 512-bit streaming AXI masters (source read, sink write) with beat-level completion.

27.2.5 SRAM Buffers (sink + source)

- BRAM: dominant BRAM consumer (20x RAMB36 + 4x RAMB18)
- Function: per-channel sink and source data buffers decoupling network and memory ends. BRAM scales with `SRAM_DEPTH` x `NUM_CHANNELS` x 2 directions.

27.2.6 MonBus / AXI Monitors (USE_AXI_MONITORS)

- LUTs: ~1-2K
- FFs: ~1-2K
- Function: rd/wr AXI monitors feeding the MonBus AXI-Lite group (error-drain + capture master), monitor perf windows @ 0x1000.

27.3 SRAM Sizing

BRAM is the primary scaling resource. Each direction has its own per-channel buffer, so total BRAM ~= `2 x NUM_CHANNELS x ceil(SRAM_DEPTH x DATA_WIDTH / BRAM_bits)`. For the 4-channel char build this lands at 22 BRAM tiles (16.30% of the 100T). Deeper buffers improve small-transfer efficiency (Section 6.1) at the cost of BRAM; wider `NUM_CHANNELS` multiplies both scheduler logic and BRAM.

27.4 Scaling Notes

Change	LUT/FF impact	BRAM impact
`NUM_CHANNELS` 4 -> 8	~+scheduler/arbiter per channel	~2x buffer tiles
`SRAM_DEPTH` deeper	minimal	+tiles per direction
`USE_AXI_MONITORS` off	-1-2K LUT/FF	none
`DATA_WIDTH` 512 -> 256	~half datapath LUT/FF	~half buffer width

DSP usage is zero – RAPIDS Beats is pure control + datapath movement, no arithmetic multiply/accumulate.

Last Updated: 2026-07-13

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

28 RAPIDS Beats Architecture MAS

Version: 0.7 **Date:** 2026-07-13 **Purpose:** Module Architecture Specification for RAPIDS Phase 1 "Beats" Architecture

---

28.1 Document Organization

Note: All chapters linked below for automated document generation.

28.1.1 Front Matter

- Document Information

28.1.2 Chapter 1: Overview

- Architecture Overview
- Top-Level Port List
- Clocks and Reset

28.1.3 Chapter 2: FUB (Functional Unit Blocks)

Control Path: - Scheduler - Descriptor Engine

AXI Engines: - AXI Read Engine - AXI Write Engine

Flow Control: - Beats Alloc Control - Beats Drain Control - Beats Latency Bridge

Control Engines: - Control-Read Engine - Control-Write Engine

28.1.4 Chapter 3: Macro (Integration Blocks)

Scheduler Integration: - Beats Scheduler Group - Beats Scheduler Group Array

Sink Data Path (Network to Memory): - Sink Data Path - Sink Data Path AXIS - Sink SRAM Controller

Source Data Path (Memory to Network): - Source Data Path - Source Data Path AXIS - Source SRAM Controller

Top-Level Integration: - RAPIDS Core Beats - RAPIDS Registers - RAPIDS Config Block - RAPIDS Beats Top - Network Planes

28.1.5 Chapter 4: Interfaces

- Interface Overview

- AXI4 Interface Specification
- AXIS Interface Specification
- MonBus Interface Specification
- CXS Interface Specification

28.2 Quick Reference

28.2.1 FUB Modules (fub_beats/)

FUB Module Summary

Module	File	Purpose	Status
scheduler_beats	`scheduler_beats.sv`	Transfer coordinator, descriptor processing	Implemented
descriptor_engine_beats	`descriptor_engine_beats.sv`	Descriptor fetch/parse (256-bit)	Implemented
axi_read_engine_beats	`axi_read_engine_beats.sv`	AXI read master, streaming pipeline	Implemented
axi_write_engine_beats	`axi_write_engine_beats.sv`	AXI write master, streaming pipeline	Implemented
alloc_ctrl_beats	`alloc_ctrl_beats.sv`	Space allocation tracking (virtual FIFO); kept and tested, no longer in the SRAM path since `bdf4e0dff`	Implemented
drain_ctrl_beats	`drain_ctrl_beats.sv`	Data availability tracking (virtual FIFO); kept and tested, no longer in the SRAM path	Implemented
latency_bridge_beats	`latency_bridge_beats.sv`	Registered-FIFO to valid/ready skid bridge; kept and tested, no longer in the SRAM path	Implemented

Module	File	Purpose	Status
ctrlrd_engine	`ctrlrd_engine.sv`	Control-read consumer gate (poll-until-match, retry budget)	Implemented
ctrlwr_engine	`ctrlwr_engine.sv`	Control-write producer doorbell (single-beat write)	Implemented

28.2.2 Macro Modules (macro_beats/)

Macro Module Summary

Module	File	Purpose	Status
scheduler_group_beats	`scheduler_group_beats.sv`	Scheduler + Descriptor Engine wrapper	Implemented
scheduler_group_array_beats	`scheduler_group_array_beats.sv`	8-channel scheduler array with arbitration	Implemented
snk_data_path_beats	`snk_data_path_beats.sv`	Network-to-memory path integration	Implemented
snk_data_path_axis_beats	`snk_data_path_axis_beats.sv`	AXIS variant of sink path	Implemented
src_data_path_beats	`src_data_path_beats.sv`	Memory-to-network path integration	Implemented
src_data_path_axis_beats	`src_data_path_axis_beats.sv`	AXIS variant of source path	Implemented
snk_sram_controller_beats	`snk_sram_controller_beats.sv`	Sink SRAM: naming wrapper around STREAM's `sram_controller` (one instance, per-channel FIFOs inside it)	Implemented
src_sram_controller_beats	`src_sram_controller_beat`	Source SRAM: naming wrapper	Implemented

Module	File	Purpose	Status
	s.sv	around STREAM's sram_controller	
rapids_src_beats / rapids_snk_beats	rapids_src_beats.sv, rapids_snk_beats.sv	One half each: scheduler array + data path + monbus merge	Implemented
rapids_core_beats	rapids_core_beats.sv	Both halves + the core monbus arbiter	Implemented
rapids_regs	regs/generated/rtl/rapids_regs.sv (generated from rtl/macro_beats/rapids_regs.rdl)	PeakRDL register block (base + MON regfile @ 0x1000)	Implemented
rapids_config_block	rapids_config_block.sv	Maps register hwif_out to core/monitor cfg_*	Implemented
rapids_beats_top	rapids_beats_top.sv	Top-level: APB slave, kickoff, monitors, MonBus AXI-Lite group	Implemented

28.3 Architecture Comparison: RAPIDS vs STREAM

The "beats" architecture is a Phase 1 implementation of RAPIDS that shares concepts with STREAM but targets network-to-memory (and vice versa) transfers rather than memory-to-memory.

RAPIDS Beats vs STREAM Comparison

Feature	STREAM	RAPIDS Beats
Primary Use	Memory-to-memory DMA	Network-to-memory accelerator
Data Paths	Single	Separate sink/source paths

Feature	STREAM	RAPIDS Beats
	bidirectional	
Network Interface	None	AXIS master/slave
SRAM Buffering	Shared buffer	Separate sink/source buffers
Descriptor Format	256-bit	256-bit (compatible)
Channel Count	8	8
Flow Control	alloc_ctrl/ drain_ctrl	STREAM `stream_alloc_ctrl`/`stream_drain_ctrl` inside the shared `sram_controller` (`alloc_ctrl_beats`/`drain_ctrl_beats` kept, out of the path)

28.4 Clock and Reset Summary

28.4.1 Clock Domains

Clock Domains

Clock	Frequency	Usage
`aclk`	100-500 MHz	Primary - all RAPIDS logic, AXI/AXIS interfaces

28.4.2 Reset Signals

Reset Signals

Reset	Polarity	Type	Usage
`aresetn`	Active-low	Async assert, sync deassert	Primary - all RAPIDS logic

See: Clocks and Reset for complete timing specifications

28.5 Interface Summary

28.5.1 External Interfaces

External Interfaces

Interface	Type	Width	Purpose
AXI4 (Descriptor)	Master	256-bit	Descriptor fetch
AXI4 (Sink Write)	Master	512-bit (param)	Sink data write to memory
AXI4 (Source Read)	Master	512-bit (param)	Source data read from memory
AXIS (Sink)	Slave	512-bit (param)	Network data ingress
AXIS (Source)	Master	512-bit (param)	Network data egress
MonBus	Master	64-bit	Monitor packet output

28.5.2 Internal Buses

Internal Buses

Interface	Width	Purpose
MonBus	64-bit	Internal monitoring bus
Descriptor Bus	256-bit	Descriptor distribution
SRAM Data Bus	512-bit	SRAM read/write data

28.6 Related Documentation

- **PRD.md** - Product requirements and overview
- **CLAUDE.md** - AI development guide
- **RAPIDS Beats HAS** - Architecture specification (the original pre-beats rapids_spec/ tree was retired)

28.7 Specification Conventions

28.7.1 Signal Naming

- **Clock:** `aclk`
- **Reset:** `aresetn` (active-low)
- **Valid/Ready:** Standard AXI/AXIS handshake
- **Registers:** `r_` prefix (e.g., `r_state`, `r_counter`)
- **Wires:** `w_` prefix (e.g., `w_next_state`, `w_grant`)

28.7.2 Parameter Naming

- **Uppercase:** `NUM_CHANNELS`, `DATA_WIDTH`, `ADDR_WIDTH`
- **Derived:** `CHAN_WIDTH = $clog2(NUM_CHANNELS)`

28.7.3 Figure and Table Numbering

- **Figures:** `Figure X.Y.Z: Title` where X=chapter, Y=section, Z=figure number
- **Tables:** Caption line `: Table Title` after table markdown

Last Updated: 2026-07-13 **Maintained By:** RAPIDS Architecture Team

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

29 Product Requirements Document (PRD)

29.1 RAPIDS - Rapid AXI Programmable In-band Descriptor System

Version: 1.0 **Date:** 2025-09-30 **Status:** Active Development - Validation in Progress **Owner:** RTL Design Sherpa Project **Parent Document:** `/PRD.md`

29.2 1. Executive Summary

The Rapid AXI Programmable In-band Descriptor System (RAPIDS) is a custom hardware accelerator designed for efficient memory-to-memory data movement with network interface integration. It demonstrates complex FSM coordination, descriptor-based DMA operations, and comprehensive monitoring capabilities.

> **Status (2026-07-22):** RAPIDS was rearchitected to the "beats" design. Current RTL is in `rtl/fub_beats/`, `rtl/macro_beats/`, `rtl/top_beats/` (control engines in `rtl/fub/`); current specs are `docs/rapids_beats_has/` and `docs/rapids_beats_mas/`. The old `rapids_fub//rapids_macro/` RTL, the `docs/rapids_spec/` tree, and the program engine / network master / network slave blocks described in parts of this PRD were retired (network interfaces are now AXIS; control is APB). Sections describing the pre-beats design are kept as history and marked where practical.
>
> **Status (2026-09-29, rapids TASK-019):** RAPIDS is byte-granular. The product RTL is the un-suffixed tree `rtl/fub/`, `rtl/macro/`, `rtl/top/rapids_top.sv`: descriptor length in BYTES, byte-granular addresses, WSTRB/TSTRB carry partial beats, one AXIS packet per descriptor. The `*_beats` tree is the beat-granular stepping stone, kept runnable and green as the characterized Genesys 2 design. Tests: `dv/tests/fub`, `macro`, `top`.

29.2.1 1.1 Quick Stats

- **Modules:** ~20 SystemVerilog files (beats architecture)
- **Architecture:** 3 major groups (Scheduler group, Sink path, Source path)
- **Interfaces:** AXI4 (memory), AXIS (network), APB (control), MonBus (monitoring)
- **Test Coverage:** see `dv/tests/` beats suites (fub_beats/macro_beats/top_beats)

- **Status:** Active validation, known issues documented

29.2.2 1.2 Subsystem Goals

- **Primary:** Demonstrate complex accelerator design patterns
- **Secondary:** Provide DMA-style memory transfer capability
- **Tertiary:** Educational reference for descriptor-based engines

---

29.3 2. Documentation Structure

This PRD provides a high-level overview. **Detailed specifications are maintained separately:**

29.3.1 Complete RAPIDS Specification

> **Status (2026-07-22):** The old `docs/rapids_spec/` tree was replaced by two beats spec trees.

Byte-granular RAPIDS books (the product design): HAS Index, MAS Index. Chapters that are identical for both designs are linked from the Beats books below.

Architecture Spec (HAS), RAPIDS Beats: `docs/rapids_beats_has/`

- **HAS Index** - Complete HAS structure
- Product Overview
- Block Diagram / Data Flow / Channel Architecture
- Interfaces: Summary, AXI4, AXIS, APB, MonBus
- Use cases: `docs/rapids_beats_has/ch04_use_cases/`
- Programming: Descriptor Format, Register Map, Initialization, Error Handling
- Performance: `docs/rapids_beats_has/ch06_performance/`

Micro-Architecture Spec (MAS): `docs/rapids_beats_mas/`

- **MAS Index** - Complete MAS structure
- Overview: Architecture, Port List, Clocks and Reset

FUB blocks (`docs/rapids_beats_mas/ch02_fub_blocks/`): - Scheduler - Per-channel scheduler FSM - Descriptor Engine - Descriptor fetch/parsing - AXI Read Engine / AXI Write Engine - Alloc Ctrl / Drain Ctrl / Latency Bridge - CtrlRd Engine / CtrlWr Engine

Macro blocks (`docs/rapids_beats_mas/ch03_macro_blocks/`): - Scheduler Group / Scheduler Group Array - Sink Data Path + AXIS wrapper, SRAM controllers - Source Data Path + AXIS wrapper, SRAM controllers - RAPIDS Core / Registers / Top

Interfaces (`docs/rapids_beats_mas/ch04_interfaces/`): - AXI4 Interface - AXIS Interface - MonBus Interface

29.3.2 Known Issues

Location: `projects/components/dma-ip/rapids/known_issues/`

- **Index** - Issue tracking overview (the old scheduler.md credit-counter write-up was retired with the pre-beats RTL)
- **Sink Data Path** - RESOLVED 2026-09-25 (write-error flag wired; timeouts are the monitor's job by design)
- **Sink SRAM Control** - RESOLVED (moved to resolved/ in d587852c2)
- Plus `known_issues/active/` for current beats issues

29.3.3 Other Documentation

- **CLAUDE** - AI assistance guide for this subsystem
- **TASKS** - Work items (largely pre-beats history)
- **Validation Report** - Test results (pre-beats snapshot)

29.4 2.4 Organizational Standards - RAPIDS Code Location

MANDATORY: All RAPIDS-specific code must be in the project area

29.4.1 Code Organization Principle

"All RAPIDS-specific verification code MUST reside in `projects/components/dma-ip/rapids/dv/` for easy discovery."

This subsystem follows the repository-wide organizational standard (see `/PRD.md` Section 2.3) requiring all project-specific code to be located in the project area, NOT the framework area.

29.4.2 RAPIDS Directory Structure

```
projects/components/dma-ip/rapids/
├── rtl/                              # RTL source code
│   ├── includes/                     # RAPIDS packages
│   ├── fub/                          # Control engines (ctrlrd/ctrlwr)
│   ├── fub_beats/                    # Beats functional unit blocks
│   ├── macro_beats/                  # Beats assemblies + registers
```

```
│   ├── macro/                    # MonBus group
│   └── top_beats/                # rapids_beats_top.sv
│
└── dv/                           # Design verification (all RAPIDS-
specific)
    ├── tbclasses/                # RAPIDS TB classes HERE (not
framework!)
    │   ├── scheduler_tb.py       # Scheduler testbench class
    │   ├── descriptor_engine_tb.py
    │   └── rapids_core_beats_tb.py
    │
    ├── components/               # RAPIDS-specific BFMs
    │   └── data_mover_bfm.py
    │
    └── tests/                    # Test runners (import TB classes)
        ├── fub/                  # Control engine tests
        ├── fub_beats/            # Beats FUB tests
(test_scheduler_beats.py, ...)
        ├── macro/                # MonBus group test
        ├── macro_beats/          # Multi-block scenarios
        └── top_beats/            # Full RAPIDS operation
```

29.4.3 What Goes Where?

Code Type	CORRECT Location	WRONG Location
RAPIDS TB Classes	`projects/components/dma-ip/rapids/dv/tbclasses/`	`bin/TBClasses/` (framework area)
RAPIDS-Specific BFMs	`projects/components/dma-ip/rapids/dv/components/`	`bin/TBClasses/` (framework area)
RAPIDS Scoreboards	`projects/components/dma-ip/rapids/dv/` (project area)	`bin/TBClasses/scoreboards/`
Test Runners	`projects/components/dma-ip/rapids/dv/tests/`	Anywhere else
Shared AXI4/APB BFMs	`bin/TBClasses/{protocol}/`	Project area

29.4.4 Import Pattern for RAPIDS Tests

CORRECT - Import from Project Area:

```python
# Import framework utilities (PYTHONPATH includes bin/)
import os, sys
from TBClasses.shared.utilities import get_repo_root
from TBClasses.shared.tbbase import TBBase
```

```
# Add repo root to Python path using robust git-based method
repo_root = get_repo_root()
sys.path.insert(0, repo_root)

# Import RAPIDS TB classes from PROJECT AREA
from projects.components.dma_ip.rapids.dv.tbclasses.scheduler_tb
import SchedulerTB
from
projects.components.dma_ip.rapids.dv.tbclasses.descriptor_engine_tb
import DescriptorEngineTB

# Shared framework components
from CocoTBFramework.components.axi4.axi4_master import AXI4Master
```

WRONG - Don't Import from Framework:

```
# DON'T DO THIS!
from TBClasses.rapids.scheduler_tb import SchedulerTB  # WRONG!
```

29.4.5 Benefits of This Organization

1. **Easy Discovery** - All RAPIDS code in ONE place: `projects/components/dma-ip/rapids/`
2. **Clear Ownership** - RAPIDS team owns their `dv/` area completely
3. **No Confusion** - Never wonder "where does this TB class live?"
4. **Maintainability** - Changes isolated to RAPIDS area don't affect other projects
5. **Framework Stays Clean** - Only truly shared cross-project code in framework

29.4.6 Compliance Status

RAPIDS is now compliant - All TB classes moved to project area as of 2025-10-18

Migration History: - **Before:** TB classes incorrectly in `bin/TBClasses/rapids/` - **After:** TB classes correctly in `projects/components/dma-ip/rapids/dv/tbclasses/` - **Test Imports:** Updated to import from project area

Complete Documentation: See `/PRD.md` Section 2.3 for repository-wide organizational standards.

29.5 3. Architecture Overview

29.5.1 3.1 Top-Level Block Diagram

```
RAPIDS Beats (Rapid AXI Programmable In-band Descriptor System)
├── Scheduler Group (scheduler_group_beats /
scheduler_group_array_beats)
│   ├── Scheduler          (Per-channel FSM, scheduler_beats.sv)
│   ├── Descriptor Engine  (Descriptor fetch/parsing,
descriptor_engine_beats.sv)
│   └── Control Engines    (ctrlrd_engine.sv / ctrlwr_engine.sv)
│
├── Sink Data Path (AXIS Network → SRAM → System Memory)
│   ├── AXIS Slave ingress   (snk_data_path_axis_beats.sv)
│   ├── Sink SRAM Controller (snk_sram_controller_beats.sv +
alloc/drain ctrl)
│   └── AXI Write Engine     (axi_write_engine_beats.sv)
│
├── Source Data Path (System Memory → SRAM → AXIS Network)
│   ├── AXI Read Engine        (axi_read_engine_beats.sv)
│   ├── Source SRAM Controller (src_sram_controller_beats.sv +
alloc/drain ctrl)
│   └── AXIS Master egress     (src_data_path_axis_beats.sv)
│
└── Control and Monitoring
    ├── APB CSRs           (rapids_config_block.sv, PeakRDL
rapids_regs.rdl)
    └── MonBus Group       (macro/monbus_axil_group_2in.sv)
```

See: `docs/rapids_beats_mas/ch01_overview/01_architecture.md` for detailed architecture

29.5.2 3.2 Data Flow

Sink Path (Receive): 1. Network packets arrive via Network Slave 2. Buffered in Sink SRAM 3. DMA'd to system memory via AXI4 Write Engine 4. Completion reported via MonBus

Source Path (Transmit): 1. Descriptor specifies data location in system memory 2. Source AXI Reader fetches data to Source SRAM 3. Network Master transmits to network 4. Completion reported via MonBus

Scheduler Coordination: - Parses descriptors from Descriptor Engine - Manages credit-based flow control - Sequences program engine operations - Coordinates sink/source data paths

---

29.6 4. Key Features

29.6.1 4.1 Descriptor-Based Operation

Feature	Status	Description
Descriptor FIFO	yes	Queued descriptor processing
Multi-field parsing	yes	Address, length, control fields
Chained descriptors	⏳	Future enhancement
Completion reporting	yes	Via MonBus packets

29.6.2 4.2 Data Path Features

Feature	Status	Description
SRAM buffering	yes	Decouple network from memory
AXI4 burst support	yes	Efficient memory transfers
Backpressure handling	yes	Flow control on all interfaces
Data alignment	yes	Handle unaligned transfers

29.6.3 4.3 Scheduler Features

Feature	Status	Description
Task FSM	yes	Multi-state coordination
Credit management	⏳	Pre-beats scheduler had exponential encoding (0→1, 1→2, ..., 15→∞); scheduler_beats.sv has no credit management yet (planned later phase)
Program sequencing	yes	Coordinated operations
Error detection	yes	Timeout, overflow detection

Credit Management Details (pre-beats, historical): - Uses **exponential credit encoding** for compact configuration - 4-bit `cfg_initial_credit` decodes to actual credit counts: - `0` → 1 credit

(2^0), 4 → 16 credits (2^4), 8 → 256 credits (2^8) - 15 → ∞ (unlimited credits, 0xFFFFFFFF) - Encoding applied at initialization; runtime operations are linear (increment/decrement by 1) - Provides wide range (1 to 16384) with minimal configuration overhead

See: `docs/rapids_beats_mas/ch02_fub_blocks/01_scheduler.md` for the current scheduler specification

29.6.4 4.4 Monitoring Integration

Feature	Status	Description
MonBus packets	yes	StandardAMBA 64-bit format
Descriptor events	yes	Start/complete reporting
Error events	yes	Timeout, overflow, underflow
Performance metrics	⏳	Future enhancement

29.7 5. Interfaces

29.7.1 5.1 External Interfaces

Interface	Type	Width	Purpose
APB	Slave	32-bit	Control/status registers
AXI4 (Sink)	Master	Configurable	Write to system memory
AXI4 (Source)	Master	Configurable	Read from system memory
AXIS (Sink)	Slave	Configurable	Network ingress (tid = channel)
AXIS (Source)	Master	Configurable	Network egress (tid = channel)
MonBus	Master	64-bit	Monitor packet output

See: `docs/rapids_beats_mas/ch04_interfaces/` and `docs/rapids_beats_has/ch03_interfaces/` for complete interface specs

29.7.25.2 Configuration Parameters

```
// Example RAPIDS instantiation sketch (see
rtl/top_beats/rapids_beats_top.sv for the
// full parameter/port list)
rapids_beats_top #(
    .ADDR_WIDTH(64),
    .DATA_WIDTH(512),
    .NUM_CHANNELS(8)
) u_rapids (
    .clk                (clk),
    .rst_n              (rst_n),
    // APB control interface
    .s_apb_*            (...),
    // AXI4 memory interfaces (read + write masters)
    .m_axi_*            (...),
    // AXIS network interfaces (tid = channel)
    .s_axis_*           (...),
    .m_axis_*           (...),
    // MonBus / monitor group egress
    .mon_*              (...)
);
```

29.8 6. Use Cases

29.8.16.1 DMA-Style Transfers

Scenario: Move data from network to system memory

Flow: 1. Software writes descriptor to Descriptor Engine 2. Scheduler parses descriptor, activates Sink path 3. Network packets arrive via Network Slave 4. Data buffered in Sink SRAM 5. AXI4 Write Engine DMAs to system memory 6. Completion packet on MonBus

29.8.26.2 Network Packet Processing

Scenario: Read data from memory, transmit to network

Flow: 1. Descriptor specifies source address, length 2. Source AXI Reader fetches data to SRAM 3. Network Master transmits to network 4. Completion/error reporting via MonBus

29.8.36.3 Custom Data Path Acceleration

Educational value: Shows how to build custom accelerators - Descriptor-based control - Multi-block FSM coordination - Buffering strategies - Error handling - Performance monitoring

---

29.9 7. Test Coverage

29.9.17.1 Current Status

> **Status (2026-07-22):** The numbers below are a pre-beats snapshot (they include the retired program engine). Current beats regression: fub 232+ and macro 182 tests passing; run the beats suites for live numbers.

Overall (pre-beats snapshot): ~85% functional coverage (basic scenarios validated, descriptor engine complete)

Component	Test Coverage	Status
Scheduler	~95%	Credit encoding fixed and verified (43/43 tests passing)
Descriptor Engine	100%	**All tests passing** (14/14 tests, 100% success rate)
Program Engine	~85%	Alignment tested
Sink Data Path	~75%	Basic flows working
Source Data Path	~70%	Basic flows working
SRAM Controllers	~80%	Buffer management tested
Integration	~60%	More stress testing needed

Test Location: `projects/components/dma-ip/rapids/dv/tests/fub_beats/` and `projects/components/dma-ip/rapids/dv/tests/macro_beats/` (plus `fub/`, `macro/`, `top_beats/`)

Recent Achievements: - **Descriptor Engine (2025-10-13):** Achieved 100% test pass rate using continuous background monitoring pattern - 14/14 tests passing across all test levels (basic, medium, full) - All test classes passing (APB_ONLY, MIXED) - All delay profiles passing (fast_producer, fast_consumer, fixed_delay, minimal_delay) - Applied continuous monitoring methodology for asynchronous output capture

See: `vault/Tasks/projects/components/dma-ip/rapids/RAPIDS_Validation_Status_Report.md` for detailed results

29.9.2 7.2 Test Strategy

FUB (Functional Unit Block) Tests: - Individual block testing - Located in `projects/components/dma-ip/rapids/dv/tests/fub_beats/` (+ `fub/` for control engines) - Focus on module-level functionality

Macro Tests: - Multi-block scenarios - Located in `projects/components/dma-ip/rapids/dv/tests/macro_beats/` (+ `macro/` for the monbus group) - End-to-end data flow validation

Top Tests: - Full RAPIDS operation - Located in `projects/components/dma-ip/rapids/dv/tests/top_beats/` - Realistic traffic patterns

---

29.10 8. Known Issues Summary

29.10.1 8.1 Critical Issues

Scheduler Credit Counter Initialization - FIXED (2025-10-11, pre-beats) - **File:** the retired pre-beats scheduler.sv (replaced by `rtl/fub_beats/scheduler_beats.sv`, which currently has no credit management - planned for a later phase) - **Issue:** Credit counter was initializing to 0 instead of using exponential encoding - **Fix Applied:** Implemented exponential credit encoding - **Status:** Fixed and verified pre-beats; the known_issues/scheduler.md write-up was retired with that RTL (see `known_issues/README.md`)

Fix Details:

```
// Previous (wrong):
r_descriptor_credit_counter <= 32'h0;

// Fixed - Exponential encoding:
// 0→1, 1→2, 2→4, 3→8, ..., 14→16384, 15→∞
r_descriptor_credit_counter <= (cfg_initial_credit == 4'hF) ?
32'hFFFFFFFF :
                               (cfg_initial_credit == 4'h0) ?
32'h00000001 :
                               (32'h1 << cfg_initial_credit);
```

Encoding Rationale: - Compact 4-bit configuration covers 1 to 16384 credits - Fine-grained control for low traffic (1, 2, 4, 8) - High-throughput support (256, 1024, 16384) - Special unlimited mode (15 → ∞) - Exponential encoding applied at initialization only; runtime operations are linear

29.10.2 8.2 Medium Priority Issues

Descriptor Engine Edge Cases: - Some stress test failures under high load - Edge case handling needs improvement - **Priority:** P2

SRAM Control Timing: - Rare timing issues in back-to-back operations - **Priority:** P2

See: `known_issues/` directory for complete issue tracking

29.11 9. Integration Guidelines

29.11.1 9.1 Quick Start

```
rapids_beats_top #(
    .ADDR_WIDTH(64),
    .DATA_WIDTH(512),
    .NUM_CHANNELS(8)
) u_rapids (
    // Clocking & Reset
    .clk                (system_clk),
    .rst_n              (system_rst_n),

    // APB Control (connect to control fabric)
    .s_apb_paddr        (ctrl_paddr),
    .s_apb_psel         (ctrl_psel),
    .s_apb_penable      (ctrl_penable),
    // ... other APB signals

    // AXI4 Memory (connect to memory controller)
    // ... AXI write channels (sink), AXI read channels (source)

    // AXIS Network (connect to network fabric; tid = channel)
    .s_axis_tdata       (net_rx_data),
    .s_axis_tvalid      (net_rx_valid),
    // ... AXIS slave (receive), AXIS master (transmit)

    // Monitor group egress (AXI-Lite drain / IRQ)
    // ... mon_* signals
);
```

See `rtl/top_beats/rapids_beats_top.sv` for the complete, authoritative port list.

29.11.2 9.2 Configuration Steps

1. **Initialize via APB registers** (use the generated regmap, access registers by name):

- Configure channel enables and thresholds
- Set timeout thresholds

2. **Load Descriptors:**
   - Write descriptors to Descriptor Engine FIFO
   - Each descriptor specifies: address, length, control bits
3. **Enable Operation:**
   - Set enable bits via APB registers
   - Monitor MonBus for completion/error packets

See: `docs/rapids_beats_has/ch05_programming/` for register and programming details

29.12 10. Development Status

29.12.1 10.1 Current Phase

Phase: Validation and Bug Fixing (In Progress)

- Core architecture implemented
- Basic functionality verified
- Scheduler credit counter bug fixed (exponential encoding implemented)
- ⏳ Credit management tests need verification (remove workarounds)
- ⏳ Stress testing ongoing
- ⏳ Edge case refinement

See: `TASKS.md` for detailed work items

29.12.2 10.2 Roadmap

Near-Term (Q4 2025): - Fix scheduler credit counter bug (completed 2025-10-11) - ⏳ Verify credit management tests (remove workarounds) - ⏳ Complete descriptor engine stress testing - ⏳ Integration test suite expansion - ⏳ Performance benchmarking

Long-Term (2026+): - Chained descriptor support - Advanced error recovery - Performance optimizations - Multi-channel support

---

29.13 11. Performance Characteristics

29.13.1 11.1 Throughput

Target: Match network/memory interface bandwidth

Bottlenecks: - SRAM buffer size - AXI4 burst efficiency - Scheduler overhead

Optimization: - Increase SRAM depth for larger packets - Tune AXI4 burst parameters - Pipeline scheduler operations

29.13.2 11.2 Latency

Components: - Descriptor parsing: ~10 cycles - SRAM buffering: Configurable depth - AXI4 memory access: System dependent - End-to-end: Typically <100 cycles for small packets

29.13.3 11.3 Resource Utilization

Area: - Scheduler: ~2K LUTs - Each data path: ~3K LUTs - SRAM buffers: Configurable (dominant area) - Total: ~10K LUTs + SRAM

Power: - Clock gating opportunities in idle blocks - SRAM power depends on depth/width

---

29.14 12. Verification Infrastructure

29.14.1 12.1 Test Organization

Location: `projects/components/dma-ip/rapids/dv/tests/`

Structure:

```
projects/components/dma-ip/rapids/dv/tests/
├── fub/                    # Control engine tests (ctrlrd/ctrlwr)
├── fub_beats/              # Beats FUB tests (scheduler, descriptor
engine,
│                           #   alloc/drain ctrl, latency bridge)
├── macro/                  # MonBus group test
├── macro_beats/            # Multi-block scenarios (scheduler group,
data paths,
│                           #   SRAM controllers)
└── top_beats/              # Full RAPIDS operation
```

29.14.2 12.2 CocoTB Framework

Location: `projects/components/dma-ip/rapids/dv/tbclasses/` (RAPIDS-specific) plus shared BFMs in `bin/TBClasses/`

Components: - RAPIDS-specific drivers - Descriptor generators - Traffic patterns - Monitor checkers

See: `../RTLDesignSherpa-DV/docs/components/<family>/` for shared framework docs

29.14.3 12.2.1 MANDATORY: BFM Usage for FUB Tests

CRITICAL DESIGN REQUIREMENT

All RAPIDS FUB (Functional Unit Block) level tests MUST use CocoTB Framework BFMs. Manual handshake driving is NOT allowed.

Required BFM Components:

Interface Type	Framework Location	BFM Component
Custom valid/ready	`bin/TBClasses/gaxi/`	GAXI Master/Slave
AXI4	`bin/TBClasses/axi4/`	AXI4 Master/Slave
AXI4-Lite (AXIL)	`bin/TBClasses/axil4/`	AXIL Master/Slave
APB	`bin/TBClasses/apb/`	APB Master/Slave
AXI-Stream (AXIS)	`bin/TBClasses/axis4/`	AXIS Master/Slave (RAPIDS network interfaces are AXIS)
MonBus	`bin/TBClasses/monbus/`	MonBus drivers

Rationale: 1. **Consistency**: All tests use standardized handshake protocols 2. **Correctness**: BFMs handle complex timing scenarios (backpressure, randomization) 3. **Reusability**: Same BFM across all RAPIDS tests 4. **Maintainability**: Fix once in BFM, all tests benefit 5. **Coverage**: BFMs include comprehensive timing profiles

Example - Program Engine:

```
# WRONG: Manual handshake (violates design requirement)
async def send_request(self, addr, data):
    self.dut.program_valid.value = 1
    self.dut.program_pkt_addr.value = addr
    # ... manual handshaking logic ...

# CORRECT: Use GAXI Master BFM
from CocoTBFramework.components.gaxi.gaxi_master import GAXIMaster

class ProgramEngineTB(TBBase):
    def __init__(self, dut):
        super().__init__(dut)
        self.program_master = GAXIMaster(
            dut=dut,
```

```
            clock=dut.clk,
            valid_signal='program_valid',
            ready_signal='program_ready',
            data_signals=['program_pkt_addr', 'program_pkt_data'],
            data_widths=[64, 32]
        )

    async def send_request(self, addr, data):
        await self.program_master.write({'program_pkt_addr': addr,
'program_pkt_data': data})
```

See: - `projects/components/dma-ip/rapids/CLAUDE.md` - Rule #1 for complete BFM usage guidelines - `../RTLDesignSherpa-DV/docs/components/gaxi/` - GAXI BFM documentation - `bin/TBClasses/axi4/` - AXI4 BFM sources (full framework docs in the RTLDesignSherpa-DV repo)

29.14.4 12.3 Test File Structure (Standard Pattern)

MANDATORY: All RAPIDS tests must follow this structure

RAPIDS tests follow the same pattern as AMBA tests for consistency across the repository:

```
# Example:
projects/components/dma-ip/rapids/dv/tests/fub_beats/test_scheduler_be
ats.py

import os
import pytest
import cocotb
from cocotb_test.simulator import run

# Import REUSABLE testbench class (NOT defined in this file!)
from projects.components.dma_ip.rapids.dv.tbclasses.scheduler_tb
import SchedulerTB
from TBClasses.shared.utilities import get_paths, create_view_cmd
from TBClasses.shared.tbbase import TBBase

#
==========================================================================
=====
# COCOTB TEST FUNCTIONS - prefix with "cocotb_" to prevent pytest
collection
#
==========================================================================
=====

@cocotb.test(timeout_time=100, timeout_unit="ms")
async def cocotb_test_basic_flow(dut):
```

```
    """Test basic descriptor flow."""
    tb = SchedulerTB(dut)
    await tb.setup_clocks_and_reset()  # Mandatory init method
    await tb.initialize_test()
    result = await tb.test_basic_descriptor_flow()
    assert result, "Basic descriptor flow test failed"

@cocotb.test(timeout_time=100, timeout_unit="ms")
async def cocotb_test_credit_encoding(dut):
    """Test exponential credit encoding."""
    tb = SchedulerTB(dut)
    await tb.setup_clocks_and_reset()  # Mandatory init method
    await tb.initialize_test()
    result = await tb.test_exponential_encoding_all_values()
    assert result, "Credit encoding test failed"

#
================================================================================
=====
# PARAMETER GENERATION - at bottom of file
#
================================================================================
=====

def generate_scheduler_test_params():
    """Generate test parameters for scheduler tests."""
    return [
        # (channel_id, num_channels, data_width, credit_width)
        (0, 8, 512, 8),  # Standard configuration
        # Add more parameter sets as needed
    ]

scheduler_params = generate_scheduler_test_params()

#
================================================================================
=====
# PYTEST WRAPPER FUNCTIONS - at bottom of file
#
================================================================================
=====

@pytest.mark.parametrize("channel_id, num_channels, data_width,
credit_width", scheduler_params)
def test_basic_flow(request, channel_id, num_channels, data_width,
credit_width):
    """
```

```
    Scheduler basic flow test.

    Run with: pytest
projects/components/dma-ip/rapids/dv/tests/fub_beats/test_scheduler_be
ats.py::test_basic_flow -v
    """
    module, repo_root, tests_dir, log_dir, rtl_dict = get_paths({
        'rtl_fub_beats': '../../rtl/fub_beats'
    })

    dut_name = "scheduler_beats"
    toplevel = dut_name

    verilog_sources = [
        os.path.join(repo_root, 'rtl', 'amba', 'includes',
'monitor_pkg.sv'),
        # RAPIDS package(s) from the project includes area
        # (prefer get_sources_from_filelist() with rtl/filelists/ in
real tests)
        os.path.join(rtl_dict['rtl_fub_beats'], '..', 'includes',
'rapids_pkg.sv'),
        os.path.join(rtl_dict['rtl_fub_beats'], f'{dut_name}.sv'),
    ]

    # Format parameters for unique test name
    cid_str = TBBase.format_dec(channel_id, 2)
    nc_str = TBBase.format_dec(num_channels, 2)
    dw_str = TBBase.format_dec(data_width, 4)
    cw_str = TBBase.format_dec(credit_width, 2)
    test_name_plus_params =
f"test_{dut_name}_cid{cid_str}_nc{nc_str}_dw{dw_str}_cw{cw_str}"

    # Add worker ID for pytest-xdist parallel execution
    worker_id = os.environ.get('PYTEST_XDIST_WORKER', '')
    if worker_id:
        test_name_plus_params = f"{test_name_plus_params}_{worker_id}"

    log_path = os.path.join(log_dir, f'{test_name_plus_params}.log')
    sim_build = os.path.join(tests_dir, 'local_sim_build',
test_name_plus_params)
    os.makedirs(sim_build, exist_ok=True)
    os.makedirs(log_dir, exist_ok=True)

    rtl_parameters = {
        'CHANNEL_ID': channel_id,
        'NUM_CHANNELS': num_channels,
```

```
        'DATA_WIDTH': data_width,
        'CREDIT_WIDTH': credit_width,
        # Add other RTL parameters as needed
    }

    extra_env = {
        'LOG_PATH': log_path,
        'TEST_CHANNEL_ID': str(channel_id),
        'TEST_NUM_CHANNELS': str(num_channels),
        'TEST_DATA_WIDTH': str(data_width),
    }

    compile_args = ["-Wno-TIMESCALEMOD"]
    sim_args = []
    plusargs = []

    cmd_filename = create_view_cmd(log_dir, log_path, sim_build,
module, test_name_plus_params)

    try:
        run(
            python_search=[tests_dir],
            verilog_sources=verilog_sources,
            includes=[
                os.path.join(repo_root, 'projects', 'components',
'dma-ip', 'rapids', 'rtl', 'includes'),
                os.path.join(repo_root, 'rtl', 'amba', 'includes'),
            ],
            toplevel=toplevel,
            module=module,
            testcase="cocotb_test_basic_flow",  # ← cocotb test
function name
            parameters=rtl_parameters,
            sim_build=sim_build,
            extra_env=extra_env,
            waves=False,
            keep_files=True,
            compile_args=compile_args,
            sim_args=sim_args,
            plusargs=plusargs,
        )

        print(f"Scheduler basic flow test completed!")
        print(f"Logs: {log_path}")

    except Exception as e:
        print(f"Scheduler basic flow test failed: {str(e)}")
```

```
        print(f"Logs preserved at: {log_path}")
        raise
```

Key Structure Requirements:

1. **Testbench Class Location:**
   - ALWAYS in `projects/components/dma-ip/rapids/dv/tbclasses/`
   - NEVER inline in test file
   - Reusable across multiple test files
2. **CocoTB Test Functions:**
   - Prefix with `cocotb_` to prevent pytest collection
   - Located at top of test file
   - Use `@cocotb.test()` decorator
   - Call testbench methods
3. **Parameter Generation:**
   - Function returns list of parameter tuples
   - Located near bottom of file (before pytest wrappers)
   - Stored in variable (e.g., `scheduler_params`)
4. **Pytest Wrapper Functions:**
   - Located at bottom of file
   - Use `@pytest.mark.parametrize()` with parameter variable
   - Build unique test names with `TBBase.format_dec()`
   - Call `run()` with `testcase="cocotb_test_function_name"`
   - Handle parallel execution (`PYTEST_XDIST_WORKER`)
5. **Mandatory TB Methods:**
   - `async def setup_clocks_and_reset(self)` - Complete initialization
   - `async def assert_reset(self)` - Assert reset signal(s)
   - `async def deassert_reset(self)` - Deassert reset signal(s)

See: - `val/amba/test_apb4_slave.py` - Reference example - `projects/components/dma-ip/rapids/CLAUDE.md` - Detailed TB requirements

---

29.15 13. Quick Reference

29.15.1 13.1 Key Files

File	Purpose
projects/components/dma-ip/rapids/PRD.md	This document (overview)
projects/components/dma-ip/rapids/CLAUDE.md	AI assistance guide
projects/components/dma-ip/rapids/TASKS.md	Work items (largely pre-beats history)
projects/components/dma-ip/rapids/docs/rapids_beats_has/	**Architecture specification**
projects/components/dma-ip/rapids/docs/rapids_beats_mas/	**Micro-architecture specification**
projects/components/dma-ip/rapids/known_issues/	Bug tracking
vault/Tasks/projects/components/dma-ip/rapids/RAPIDS_Validation_Status_Report.md	Test results (pre-beats snapshot)

29.15.2 13.2 Commands

```
# Run RAPIDS tests
pytest projects/components/dma-ip/rapids/dv/tests/fub_beats/ -v     #
Individual blocks
pytest projects/components/dma-ip/rapids/dv/tests/macro_beats/ -v   #
Multi-block
pytest projects/components/dma-ip/rapids/dv/tests/top_beats/ -v     #
Full system

# Run specific FUB test
pytest
projects/components/dma-ip/rapids/dv/tests/fub_beats/test_scheduler_be
ats.py -v

# Lint RAPIDS RTL
verilator --lint-only
projects/components/dma-ip/rapids/rtl/fub_beats/scheduler_beats.sv

# View specifications
cat
projects/components/dma-ip/rapids/docs/rapids_beats_mas/rapids_beats_m
```

```
as_index.md
cat
projects/components/dma-ip/rapids/docs/rapids_beats_mas/ch02_fub_block
s/01_scheduler.md
```

29.16 14. Success Criteria

29.16.1 14.1 Functional

- All major blocks implemented
- Basic data flows working
- Scheduler credit bug fixed (exponential encoding implemented)
- ⏳ Credit management tests verified (remove workarounds, run full suite)
- ⏳ 100% descriptor test pass rate (currently ~80%)
- ⏳ Stress tests passing

29.16.2 14.2 Quality

- ⏳ >90% functional coverage (currently ~80%)
- ⏳ >85% code coverage
- All FSMs documented
- ⏳ Integration guide complete

29.16.3 14.3 Documentation

- Complete specification in docs/rapids_beats_has/ + docs/rapids_beats_mas/
- Known issues documented
- ⏳ Integration examples
- ⏳ Performance characterization

29.17 15. Educational Value

RAPIDS demonstrates: - Complex FSM coordination (scheduler ↔️ data paths) - Descriptor-based DMA design patterns - Buffer management strategies - Credit-based flow control with exponential encoding - Multi-interface integration - Comprehensive monitoring - Error detection and reporting - Compact configuration encoding strategies

Target Audience: - Advanced RTL designers - Accelerator architects - DMA engine developers - System integration engineers

29.18 16. Documentation Generation

29.18.1 16.1 Generating PDF/DOCX from Specification

Tool: `bin/md_to_docx.py` (driven by the wrapper scripts in `docs/` - preferred)

Use the wrapper scripts to convert the linked HAS/MAS spec indexes into single all-inclusive PDF/DOCX files:

Basic Usage:

```
# Preferred: wrapper scripts (they call md_to_docx.py with the house
style)
cd projects/components/dma-ip/rapids/docs
./generate_has_pdf.sh --rev 0.8     # builds
RAPIDS_Beats_HAS_v0.8.docx/.pdf
./generate_mas_pdf.sh --rev 0.7     # builds
RAPIDS_Beats_MAS_v0.7.docx/.pdf

# Direct tool invocation (from repo root), if you need custom options
python bin/md_to_docx.py \

projects/components/dma-ip/rapids/docs/rapids_beats_mas/rapids_beats_m
as_index.md \
    -o
projects/components/dma-ip/rapids/docs/RAPIDS_Beats_MAS_draft.docx \
    --toc \
    --title-page \
    --pdf
```

Key Features: - **Recursive Collection:** Follows all markdown links in the index file - **Heading Demotion:** Automatically adjusts heading levels for included files - **Table of Contents:** `--toc` flag generates automatic ToC - **Title Page:** `--title-page` flag creates title page from first heading - **PDF Export:** `--pdf` flag generates both DOCX and PDF - **Image Support:** Resolves images relative to source directory - **Template Support:** Optional custom DOCX/DOTX template via `-t` flag

Common Workflow:

```
# 1. Update spec content under rapids_beats_has/ or rapids_beats_mas/
# 2. Generate documentation with a bumped revision
cd projects/components/dma-ip/rapids/docs
./generate_mas_pdf.sh --rev 0.8

# 3. Output files created in docs/:
#    - RAPIDS_Beats_MAS_v0.8.docx
#    - RAPIDS_Beats_MAS_v0.8.pdf
```

Debug Mode:

```
# Generate debug markdown to see combined output
python bin/md_to_docx.py \

projects/components/dma-ip/rapids/docs/rapids_beats_mas/rapids_beats_m
as_index.md \
    -o output.docx \
    --debug-md

# This creates debug.md showing the complete merged content
```

Tool Requirements: - Python 3.6+ - Pandoc installed and in PATH - For PDF generation: LaTeX (e.g., texlive) or use Pandoc's built-in PDF writer

See: `bin/md_to_docx.py` for complete implementation details

29.19 16.2 PDF Generation Location

IMPORTANT: PDF files should be generated in the docs directory:

`projects/components/dma-ip/rapids/docs/`

Quick Command: Use the provided shell scripts:

```
cd projects/components/dma-ip/rapids/docs
./generate_has_pdf.sh    # Architecture spec (HAS)
./generate_mas_pdf.sh    # Micro-architecture spec (MAS)
```

The shell scripts will automatically: 1. Use the md_to_docx.py tool from bin/ 2. Process the rapids_beats_has / rapids_beats_mas index files 3. Generate both DOCX and PDF files in the docs/ directory 4. Create table of contents and title page

See: `bin/md_to_docx.py` for complete implementation details

29.20 17. References

29.20.1 16.1 Internal Documentation

- **Complete Spec:** `docs/rapids_beats_has/` + `docs/rapids_beats_mas/` ← **Primary technical reference**
- **Validation:** `vault/Tasks/projects/components/dma-ip/rapids/RAPIDS_Validation_Status_Report.md` (pre-beats snapshot)

- **Master PRD:** `/PRD.md`
- **Repository Guide:** `/CLAUDE.md`

29.20.2 16.2 Related Subsystems

- **AMBA:** `rtl/amba/` - Monitor infrastructure used in RAPIDS (monitors in `rtl/amba/ocelus/`)
- **Common:** `rtl/common/` - Building blocks (counters, FIFOs, etc.)
- **CocoTB Framework:** `bin/TBClasses/` (shared) + `projects/components/dma-ip/rapids/dv/tbclasses/` (RAPIDS)

29.20.3 16.3 External References

- AXI4 Specification: ARM IHI0022E
- AXIL4 Specification: ARM IHI0022E (subset)
- Network interface specs (custom Network protocol)

Document Version: 1.0 **Last Updated:** 2025-09-30 **Review Cycle:** Monthly during active development **Next Review:** 2025-10-30 **Owner:** RTL Design Sherpa Project

---

29.21 Navigation

- **← Back to Root:** `/PRD.md`
- **Complete Specification:** `docs/rapids_beats_has/rapids_beats_has_index.md` + `docs/rapids_beats_mas/rapids_beats_mas_index.md`
- **AI Guidance:** `CLAUDE.md`
- **Tasks:** `TASKS.md`
- **Issues:** `known_issues/`

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

30 Address Increment Patterns for DMA and Descriptor-Based Data Movement

Document Version: 1.0 **Last Updated:** 2026-01-13 **Scope:** Comprehensive analysis of address increment patterns for descriptor-based systems **Reference Implementation:** RAPIDS (Rapid AXI Programmable In-band Descriptor System)

30.1 Table of Contents

1. Executive Summary
2. RAPIDS Current Implementation
3. AXI Protocol Burst Modes
4. Linear Address Increment Patterns
5. 2D Transfer Patterns (Stride-Based)
6. 3D Transfer Patterns
7. Scatter-Gather Patterns
8. Circular and Ring Buffer Patterns
9. Tensor Memory Access Patterns
10. Implementation Comparison Matrix
11. RAPIDS Extension Recommendations
12. References

30.2 1. Executive Summary

Address increment patterns determine how DMA engines and descriptor-based data movement systems calculate memory addresses during transfers. The choice of pattern significantly impacts:

- **Bandwidth efficiency**: Alignment and burst optimization
- **Application fit**: Different workloads require different patterns
- **Hardware complexity**: More patterns = more control logic
- **Software flexibility**: Richer patterns = simpler programming model

30.2.1 Pattern Categories

Category	Description	Use Cases
Linear/Contiguous	Sequential address	Block copy, streaming

Category	Description	Use Cases
	increment	data
2D Stride	Row-major with inter-row gaps	Image processing, video frames
3D Volume	Multi-plane with inter-plane gaps	3D imaging, tensor operations
Scatter-Gather	Non-contiguous fragment list	Protocol buffers, fragmented memory
Circular	Wrap-around at boundary	Audio streaming, network buffers
Tensor	Multi-dimensional with strides	Neural network accelerators

30.3 2. RAPIDS Current Implementation

30.3.1 2.1 Descriptor Format (256-bit)

RAPIDS uses a fixed 256-bit descriptor structure defined in `rapids_pkg.sv`:

```
Bit Field          Width   Description
──────────────────────────────────────────────────────────
[63:0]             64-bit  src_addr - Source address (byte-aligned)
[127:64]           64-bit  dst_addr - Destination address (byte-
aligned)
[159:128]          32-bit  length - Transfer length in BEATS (not
bytes)
[191:160]          32-bit  next_descriptor_ptr - Next descriptor
address
[207:200]           8-bit  desc_priority - Transfer priority
[199:196]           4-bit  channel_id - Channel ID
[195]               1-bit  error - Error flag
[194]               1-bit  last - Last descriptor in chain
[193]               1-bit  gen_irq - Generate interrupt
[192]               1-bit  valid - Valid descriptor
[255:208]          48-bit  reserved - Future use
```

30.3.2 2.2 Current Address Increment Modes

Mode 1: Beat-Based Linear (Default)

```
Address Calculation:
  next_addr = current_addr + (beat_count × 64 bytes)
```

```
  Where: RAPIDS_BYTES_PER_BEAT = 64 bytes (512-bit data width)
```

- All transfers aligned to 64-byte boundaries for optimal throughput
- Descriptor length field specifies count in beats
- Software converts byte count to beats: `beats = (bytes + 63) >> 6`

Mode 2: Chunk-Based Alignment (Sub-Beat Transfers)

For alignment/final phases handling partial beats:

```
Address Calculation for Alignment Phase:
  offset = address[5:0]                    // 6-bit offset (0-63)
  bytes_to_boundary = (offset == 0) ? 0 : 64 - offset

  Transfer Phases:
  Phase 1 (Alignment): Transfer bytes_to_boundary to reach 64B
boundary
  Phase 2 (Streaming): Transfer full 64B beats
  Phase 3 (Final):     Transfer remaining < 64 bytes
```

30.3.3 2.3 Alignment Information Structure

```
typedef struct packed {
    logic [15:0] first_chunk_enables;    // Phase 1 chunk mask
    logic [15:0] streaming_chunk_enables; // Phase 2 (always 0xFFFF)
    logic [15:0] final_chunk_enables;     // Phase 3 chunk mask
    logic [31:0] first_transfer_bytes;    // Phase 1 size
    logic [31:0] streaming_bytes;         // Phase 2 size
    logic [31:0] final_transfer_bytes;    // Phase 3 size
    logic [3:0]  first_start_chunk;       // Starting chunk (0-15)
    logic [3:0]  first_valid_chunks;      // Chunks in phase 1
    logic [3:0]  final_valid_chunks;      // Chunks in phase 3
} alignment_info_t;
```

30.3.4 2.4 AXI Translation

Transfer Phase	AxSIZE	AxBURST	Address Increment
Alignment	3'b010	INCR	4 bytes × (AxLEN+1)
Streaming	3'b110	INCR	64 bytes × (AxLEN+1)
Final	3'b010	INCR	4 bytes × (AxLEN+1)

30.4 3. AXI Protocol Burst Modes

The AXI protocol defines three burst types that constrain address generation:

30.4.1 3.1 FIXED Burst (AxBURST = 2'b00)

```
Address Formula:
  beat[n].addr = AxADDR   (constant for all beats)
```

Use Cases: - FIFO access (single address register) - Memory-mapped I/O ports - Audio codec sample registers

Constraints: - All beats transfer to same address - Cannot cross 4KB boundary (trivially satisfied)

30.4.2 3.2 INCREMENT Burst (AxBURST = 2'b01)

```
Address Formula:
  beat[n].addr = AxADDR + n × (1 << AxSIZE)

  Example (AxSIZE=6, 64-byte):
    beat[0] = 0x1000
    beat[1] = 0x1040
    beat[2] = 0x1080
    ...
```

Use Cases: - Linear memory access - Block copy operations - Stream data to/from memory

Constraints: - Cannot cross 4KB boundary - Maximum burst length varies by protocol version

30.4.3 3.3 WRAP Burst (AxBURST = 2'b10)

```
Address Formula:
  wrap_boundary = AxADDR & ~((AxLEN+1) × (1<<AxSIZE) - 1)
  wrap_size = (AxLEN+1) × (1 << AxSIZE)

  beat[n].addr = wrap_boundary + ((AxADDR + n×(1<<AxSIZE)) mod
wrap_size)

  Example (AxADDR=0x1020, AxLEN=3, AxSIZE=4):
    Wrap boundary: 0x1000
    Wrap size: 64 bytes (4 × 16)
    beat[0] = 0x1020
    beat[1] = 0x1030
    beat[2] = 0x1000  (wrapped)
    beat[3] = 0x1010
```

Use Cases: - Cache line fills (critical-word-first) - Circular buffer access

Constraints: - AxLEN must be 1, 3, 7, or 15 - Starting address must be aligned to transfer size

---

30.5 4. Linear Address Increment Patterns

30.5.1 4.1 Simple Contiguous

The most basic pattern - sequential addresses:

```
Descriptor Fields:
  src_addr:  Starting source address
  dst_addr:  Starting destination address
  length:    Total transfer size

Address Calculation:
  for each beat b in [0, length/beat_size):
    src = src_addr + b × beat_size
    dst = dst_addr + b × beat_size
```

RAPIDS Implementation: Native support via beat-based transfers.

30.5.2 4.2 Unaligned Start/End

Handling transfers that don't start or end on natural boundaries:

```
Example: Transfer 200 bytes from 0x1040
  Phase 1 (Align):  Transfer 64 bytes (0x1040-0x107F) to reach 0x1080
  Phase 2 (Stream): Transfer 128 bytes (0x1080-0x10FF) in 2 full beats
  Phase 3 (Final):  Transfer 8 bytes (0x1100-0x1107)
```

RAPIDS Implementation: Native support via alignment_info_t structure.

30.5.3 4.3 Size-Constrained Bursts

Breaking large transfers into AXI-compliant bursts:

```
Constraints:
  - AXI4: Max 256 beats per burst
  - No 4KB boundary crossing

Address Calculation:
  remaining = total_length
  current_addr = start_addr

  while (remaining > 0):
    burst_len = min(remaining, 256×beat_size)
    burst_len = min(burst_len, next_4kb_boundary - current_addr)
    issue_burst(current_addr, burst_len)
    current_addr += burst_len
    remaining -= burst_len
```

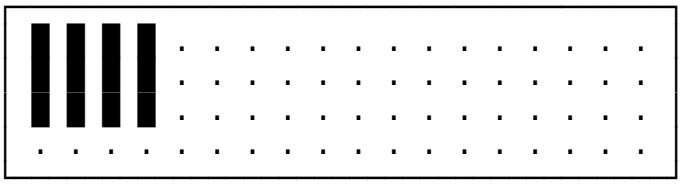
RAPIDS Implementation: Handled by AXI engine burst splitting.

30.6 5. 2D Transfer Patterns (Stride-Based)

30.6.1 5.1 Concept

2D transfers move rectangular regions where source and destination may have different memory layouts (pitches/strides).

Memory Layout:



30.6.2 5.2 Descriptor Fields (Extended)

```
2D Descriptor Format:
  src_addr:    Starting source address (top-left corner)
  dst_addr:    Starting destination address
  x_length:    Bytes per row (width of transfer region)
  y_length:    Number of rows
  src_stride:  Source bytes between row starts
  dst_stride:  Destination bytes between row starts
```

30.6.3 5.3 Address Calculation

```
for row in [0, y_length):
  row_src = src_addr + row × src_stride
  row_dst = dst_addr + row × dst_stride
  transfer(row_src, row_dst, x_length)
```

30.6.4 5.4 Use Cases

Image Processing:

```
Example: Extract 640×480 ROI from 1920×1080 frame (32-bit pixels)
  src_addr:   frame_base + (y_offset × 1920 + x_offset) × 4
  dst_addr:   roi_buffer
  x_length:   640 × 4 = 2560 bytes
  y_length:   480
  src_stride: 1920 × 4 = 7680 bytes
  dst_stride: 640 × 4 = 2560 bytes (packed destination)
```

Video Frame Tiling:

```
Example: Copy 16×16 macroblock
  x_length:   16 × bytes_per_pixel
  y_length:   16
  src_stride: frame_width × bytes_per_pixel
  dst_stride: 16 × bytes_per_pixel
```

30.6.5 5.5 Analog Devices AXI DMAC Implementation

Reference implementation from ADI's high-speed DMA controller:

```
Registers:
  X_LENGTH  (0x418): Row width - 1
  Y_LENGTH  (0x41C): Row count - 1
  SRC_STRIDE (0x424): Source row spacing
  DEST_STRIDE (0x420): Destination row spacing

Address Formulas:
  ROW_SRC_ADDRESS = SRC_ADDRESS + SRC_STRIDE × N
  ROW_DEST_ADDRESS = DEST_ADDRESS + DEST_STRIDE × N
```

30.7 6. 3D Transfer Patterns

30.7.1 6.1 Concept

3D transfers extend 2D with an additional depth/plane dimension:

```
Memory Layout (Z planes stacked):
  Plane 0:    Plane 1:    Plane 2:
```



30.7.2 6.2 Descriptor Fields (Extended)

```
3D Descriptor Format:
  src_addr:     Starting address (origin corner)
  dst_addr:     Destination address
  x_length:     Width in bytes
  y_length:     Height in rows
  z_length:     Depth in planes
  src_stride_x: Source row stride (unused, implicit x_length)
  src_stride_y: Source row-to-row stride
  src_stride_z: Source plane-to-plane stride
  dst_stride_y: Destination row stride
  dst_stride_z: Destination plane stride
```

30.7.3 6.3 Address Calculation

```
for plane in [0, z_length):
  for row in [0, y_length):
    src = src_addr + plane×src_stride_z + row×src_stride_y
    dst = dst_addr + plane×dst_stride_z + row×dst_stride_y
    transfer(src, dst, x_length)
```

30.7.4 6.4 Use Cases

3D Medical Imaging:

```
Example: Extract 64×64×64 VOI from 512×512×256 CT volume
  src_addr:     volume_base + z_off×512×512 + y_off×512 + x_off
  x_length:     64 bytes
  y_length:     64
  z_length:     64
  src_stride_y: 512
  src_stride_z: 512 × 512
```

Neural Network Tensors:

```
Example: Move 4D tensor slice [batch, channel, H, W]
  Requires nested 3D operations or generalized tensor addressing
```

30.8 7. Scatter-Gather Patterns

30.8.1 7.1 Concept

Scatter-Gather (SG) handles non-contiguous memory regions using linked lists of descriptors:

```
Descriptor Chain:

┌──────────┐     ┌──────────┐     ┌──────────┐
│ Desc 0   │────→│ Desc 1   │────→│ Desc 2   │────→ NULL
│ addr=A0  │     │ addr=A1  │     │ addr=A2  │
│ len=L0   │     │ len=L1   │     │ len=L2   │
└──────────┘     └──────────┘     └──────────┘
```

30.8.2 7.2 SG Descriptor Fields

```
Scatter-Gather Descriptor:
  buffer_addr:      Address of this fragment
  buffer_length:    Size of this fragment
  next_desc_ptr:    Address of next descriptor (0 = last)
  control_flags:    Completion interrupt, error handling, etc.
  status:           Completion status (written by DMA)
```

30.8.37.3 Operation Modes

Gather (Multiple Source -> Single Destination):

```
Source fragments: [A0:L0], [A1:L1], [A2:L2]
Destination: Contiguous buffer B

Result: B = concat(mem[A0:A0+L0], mem[A1:A1+L1], mem[A2:A2+L2])
```

Scatter (Single Source -> Multiple Destinations):

```
Source: Contiguous buffer B of length L0+L1+L2
Destination fragments: [A0:L0], [A1:L1], [A2:L2]

Result: mem[A0:A0+L0] = B[0:L0]
        mem[A1:A1+L1] = B[L0:L0+L1]
        mem[A2:A2+L2] = B[L0+L1:end]
```

30.8.47.4 AMD/Xilinx AXI DMA SG Descriptor Format

```
Standard Descriptor (32-bit addressing):
  Offset  Field            Description
  ──────────────────────────────────────────────
  0x00    NXTDESC          Next descriptor pointer
  0x04    Reserved
  0x08    BUFFER_ADDRESS   Data buffer pointer
  0x0C    Reserved
  0x10    Reserved
  0x14    Reserved
  0x18    CONTROL          Transfer length, SOF, EOF
  0x1C    STATUS           Completion status
```

30.8.57.5 Use Cases

Network Packet Assembly:

```
Example: Assemble TCP packet from scattered headers and payload
  Desc 0: Ethernet header (14 bytes at addr_eth)
  Desc 1: IP header (20 bytes at addr_ip)
  Desc 2: TCP header (20 bytes at addr_tcp)
  Desc 3: Payload (1460 bytes at addr_payload)
```

RAPIDS Implementation: Native support via next_descriptor_ptr chaining.

30.9 8. Circular and Ring Buffer Patterns

30.9.1 8.1 Concept

Circular buffers wrap addresses at a boundary, enabling continuous streaming without descriptor reload:

```
Memory Layout:

  ┌─────────────────────────────────────────┐
  │ 7 | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | (wraps)
  └─────────────────────────────────────────┘

    ↑               ↑
  write           read
```

30.9.2 8.2 Descriptor Fields (Extended)

```
Circular Buffer Descriptor:
  base_addr:    Buffer base address
  buffer_size:  Total buffer size (must be power of 2 for efficient
wrap)
  current_ptr:  Current read/write position
  wrap_enable:  Enable address wrapping
```

30.9.3 8.3 Address Calculation

```
Efficient wrap (power-of-2 size):
  next_addr = base_addr + ((current_ptr + increment) & (buffer_size -
1))

General wrap:
  offset = (current_ptr + increment) % buffer_size
  next_addr = base_addr + offset
```

30.9.4 8.4 Ping-Pong Double Buffering

Common implementation using two descriptors pointing to each other:

```
Descriptor A:                Descriptor B:
  buffer_addr = BUF_A          buffer_addr = BUF_B
  next_desc = &Desc_B          next_desc = &Desc_A

Operation:
  While DMA processes BUF_A, CPU fills BUF_B
  On completion, DMA auto-loads Desc_B, starts BUF_B
  CPU processes BUF_A while DMA runs BUF_B
  (repeat)
```

30.9.5 8.5 Use Cases

Audio Streaming:

```
Example: 48kHz stereo audio, 20ms buffers
  buffer_size: 48000 × 2 channels × 2 bytes × 0.02s = 3840 bytes
  Ring with 4 buffers: 15360 bytes total
  Wrap at: base + 15360
```

Network RX Ring:

```
Example: Ethernet receive ring
  descriptor_count: 256 (power of 2)
  Each descriptor points to packet buffer
  Last descriptor.next points to first descriptor
  Hardware auto-advances through ring
```

30.10 9. Tensor Accelerator Memory Access Patterns (PRIORITY 0)

Tensor accelerators for neural network inference represent the most demanding and sophisticated address generation requirements. This section provides comprehensive coverage of industry-standard approaches.

30.10.1 9.1 Systolic Array Dataflow Fundamentals

Systolic arrays (used in Google TPU, NVIDIA Tensor Cores, etc.) require coordinated data movement across three matrices: inputs (activations), weights, and outputs (partial sums).

Three Primary Dataflows:

Dataflow	Stationary Data	Moving Data	Best For
Weight Stationary (WS)	Weights preloaded in PEs	Inputs + partial sums flow	Weight reuse (many inputs)
Input Stationary (IS)	Inputs held in PEs	Weights + partial sums flow	Input reuse (many kernels)
Output Stationary (OS)	Partial sums accumulate in PEs	Inputs + weights flow	Output accumulation

Address Pattern Implications:

```
Weight Stationary:
  - Weights: Load once per layer, stream through array
  - Inputs: Stream row-by-row, reuse across weight tiles
```

```
  - Address pattern: 2D tiled with weight-tile-aligned boundaries

Output Stationary:
  - Outputs: Accumulate in place
  - Inputs/Weights: Double-buffered streaming
  - Address pattern: Output-tile-centric addressing
```

30.10.2 9.2 Data Cube Model (NVDLA Reference)

NVIDIA's Deep Learning Accelerator (NVDLA) defines tensor addressing using a "data cube" model with three key stride parameters:

Data Cube Structure:

```
Dimensions: Width (W) × Height (H) × Channels (C)

Memory Layout:
  - Data divided into 1×1×32byte "atom cubes"
  - Scanning order: C'(32B) → W → H → C (surfaces)
  - C' changes fastest (innermost loop)

Address Calculation:
  element_addr = base_addr
                 + surface_index × surface_stride
                 + line_index × line_stride
                 + element_offset_within_line
```

Stride Parameters:

Parameter	Description	Alignment
`line_stride`	Bytes from line N to line N+1	32 bytes
`surface_stride`	Bytes from surface N to surface N+1	32 bytes
`base_addr`	Starting address	32 bytes

NVDLA Register Configuration:

```
D_DATAIN_FORMAT     : Data format selection
D_DATAIN_SIZE_0     : Width × Height
D_DATAIN_SIZE_1     : Channels
D_LINE_STRIDE       : Line-to-line spacing (0x5040)
D_SURF_STRIDE       : Surface-to-surface spacing

Example:
  Width=26, Height=32, Channels=3
```

```
  line_stride = 0x1A0 (aligned width)
  surface_stride = 0x1520 (line_stride × height)
```

30.10.3 9.3 Tensor Memory Accelerator (TMA) - NVIDIA Hopper

NVIDIA's Hopper architecture introduces hardware TMA for efficient multi-dimensional tensor transfers:

TMA Descriptor Format:

```
CUtensorMap structure (64-bit packed descriptor):
  - Global memory base address
  - Tensor rank (1D to 5D supported)
  - Dimension sizes and strides
  - Swizzle pattern for bank conflict avoidance
  - Box dimensions (tile size for transfer)
```

Key Constraints:

```
Alignment Requirements:
  - Contiguous dimension must have stride=1
  - All other strides must be multiples of 16 bytes
  - For float tensors [M,N] with stride [N,1]: N % 4 == 0

Tile Quantization:
  - TPU/GPU systolic arrays process 128×128 or 128×8 tiles
  - Tensors must align to tile boundaries to avoid padding waste
  - Performance penalty for partial tiles (idle MACs)
```

Address Generation (Hardware):

```
TMA eliminates per-thread address calculation:
  Traditional: Each thread computes addr = base + thread_id × stride
  TMA: Single descriptor, hardware generates all addresses

Coordinate-based access:
  ArithTuple(row, col) → Hardware translates to physical address
  Automatic out-of-bounds predication (no manual boundary checks)
```

30.10.4 9.4 Convolution Address Patterns

30.10.4.1 9.4.1 Direct Convolution

```
Input Feature Map:  [N, C_in, H_in, W_in]
Weight Kernel:      [C_out, C_in, K_h, K_w]
Output Feature Map: [N, C_out, H_out, W_out]

Output position (n, c_out, h_out, w_out):
  For each (c_in, k_h, k_w):
    input_addr = base_in + n×(C_in×H_in×W_in)
```

```
                          + c_in×(H_in×W_in)
                          + (h_out×stride_h + k_h)×W_in
                          + (w_out×stride_w + k_w)

    weight_addr = base_w + c_out×(C_in×K_h×K_w)
                          + c_in×(K_h×K_w)
                          + k_h×K_w
                          + k_w
```

30.10.4.2 9.4.2 Im2Col Transformation

Converts convolution to GEMM for systolic array efficiency:

```
Original Convolution:
  Output[h,w] = sum(Input[h+kh, w+kw] × Weight[kh, kw])

Im2Col Transformation:
  - Unroll each receptive field into a column
  - Weight kernels become rows
  - Convolution becomes matrix multiplication

Address Pattern for Im2Col:
  For output position (h_out, w_out):
    col_index = h_out × W_out + w_out
    For kernel position (c_in, k_h, k_w):
      row_index = c_in × K_h × K_w + k_h × K_w + k_w
      src_addr = base + c_in×(H×W) + (h_out×s_h+k_h)×W +
(w_out×s_w+k_w)
      dst_addr = im2col_base + row_index × num_output_positions +
col_index

Data Expansion Factor:
  Naive: K_h × K_w × data replication
  Hardware Im2Col: Generate addresses on-the-fly, no physical
expansion
```

30.10.4.3 9.4.3 Strided and Dilated Convolution

```
Strided Convolution (stride > 1):
  input_h = output_h × stride + kernel_h
  input_w = output_w × stride + kernel_w
  Reduces output spatial dimensions

Dilated/Atrous Convolution:
  input_h = output_h + kernel_h × dilation
  input_w = output_w + kernel_w × dilation
  Expands receptive field without increasing parameters
```

```
Address Modification:
  base_addr + (h × stride + kh × dilation) × line_stride
            + (w × stride + kw × dilation) × element_size
```

30.10.4.4 9.4.4 Depthwise Separable Convolution

```
Standard Convolution: C_in × C_out × K × K multiplications
Depthwise Separable: C_in × K × K + C_in × C_out multiplications

Phase 1 - Depthwise (per-channel):
  For each channel c:
    output[c, h, w] = conv2d(input[c], kernel[c])

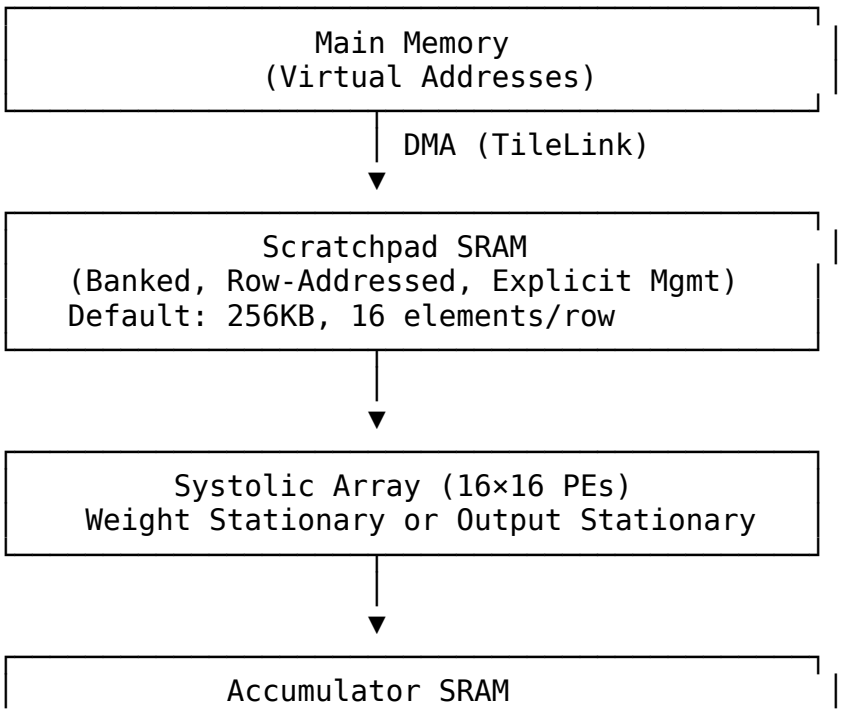
  Address Pattern:
    Each channel processes independently
    No cross-channel data movement
    input_addr = base + c × H × W + h × W + w

Phase 2 - Pointwise (1×1 convolution):
  Standard GEMM across channels
  input_addr = base + c × (H × W) + position
```

30.10.5 9.5 Gemmini Accelerator Architecture (UC Berkeley Reference)

Open-source systolic array generator with well-documented DMA and addressing:

Memory Hierarchy:



```
   (32-bit accumulators, in-place addition)
```

Scratchpad Address Format (32-bit):

```
Bit 31:    0=Scratchpad, 1=Accumulator
Bit 30:    Accumulator mode (0=overwrite, 1=accumulate)
Bit 29:    Read format (0=scaled inputType, 1=raw accType)
Bits 28:0: Row address within selected memory

Row Width:
  Scratchpad: DIM × inputType bits (default: 16 × 8 = 128 bits)
  Accumulator: DIM × accType bits (default: 16 × 32 = 512 bits)
```

DMA Operations:

```
// mvin: Move data from DRAM to scratchpad
// Configuration registers set stride
mvin(
    dram_addr,          // Virtual address (64-bit)
    sp_addr,            // Scratchpad row address
    rows,               // Number of rows to transfer
    cols                // Elements per row (≤ DIM)
);

// mvout: Move data from accumulator to DRAM
// Supports integrated max-pooling
mvout(
    dram_addr,          // Destination address
    acc_addr,           // Accumulator row address
    rows, cols,
    pool_size,          // Optional pooling window
    pool_stride
);
```

Stride Configuration:

```
config_mvin:
  stride[0]: Source memory stride for mvin_A
  stride[1]: Source memory stride for mvin_B
  stride[2]: Source memory stride for mvin_D

config_mvout:
  stride: Destination memory stride
  pool_params: Pooling window and stride
```

30.10.6 9.6 Tiling for Large Matrices

When matrices exceed on-chip memory, tiling is essential:

Hierarchical Tiling (CUTLASS Model):

```
Level 1 - Thread Block Tile:
  Load tile from global memory to shared memory
  Tile size: M_tile × K_tile (A), K_tile × N_tile (B)

Level 2 - Warp Tile:
  Load from shared memory to registers
  Tile size: Warp_M × Warp_K (A), Warp_K × Warp_N (B)

Level 3 - Thread Tile:
  Individual thread's computation
  Register-to-register operations

Address Generation per Level:
  Global: base + block_id × block_tile_size
  Shared: smem_base + warp_id × warp_tile_size
  Register: Direct indexing within tile
```

Double-Buffering for Latency Hiding:

```
Buffer Structure:
  tile_buffer[2][TILE_ROWS][TILE_COLS]

Pipeline:
  Cycle N:   Compute on buffer[0], Load to buffer[1]
  Cycle N+1: Compute on buffer[1], Load to buffer[0]

Address Pattern:
  compute_addr = base + (cycle % 2) × tile_size + element_offset
  load_addr    = base + ((cycle + 1) % 2) × tile_size + element_offset
```

30.10.7 9.7 Sparse Tensor Patterns

Sparse accelerators require index-based indirect addressing:

Compressed Sparse Row (CSR) Format:

```
Dense: [a 0 b]    CSR: values = [a, b, c, d]
       [0 c 0]         col_idx = [0, 2, 1, 0]
       [d 0 0]         row_ptr = [0, 2, 3, 4]

Address Calculation:
  For row r, iterate col_idx[row_ptr[r]] to col_idx[row_ptr[r+1]-1]
  value_addr = values_base + idx
  col_addr = col_idx_base + idx

Indirect Access:
```

```
  actual_col = memory[col_addr]
  dense_addr = base + r × stride + actual_col × element_size
```

Hardware Implications:

```
Dense Accelerator (TPU): Initiation interval = 1 (fully pipelined)
Sparse Accelerator (EIE): Initiation interval = 17 (metadata overhead)

Trade-off:
  - Sparse saves memory bandwidth (skip zeros)
  - But adds index indirection latency
  - Effective for >90% sparsity
```

30.10.8 9.8 Transformer/Attention Patterns

Modern LLM accelerators require specialized patterns:

Q, K, V Matrix Generation:

```
Input: X [seq_len, d_model]
Weights: W_Q, W_K, W_V [d_model, d_k]

Q = X × W_Q  →  [seq_len, d_k]
K = X × W_K  →  [seq_len, d_k]
V = X × W_V  →  [seq_len, d_k]

Address Pattern: Standard GEMM with different weight matrices
  q_addr = q_base + seq_idx × d_k + head_idx
  k_addr = k_base + seq_idx × d_k + head_idx
  v_addr = v_base + seq_idx × d_k + head_idx
```

Attention Score Computation:

```
Attention = softmax(Q × K^T / sqrt(d_k)) × V

Memory Challenge:
  Q × K^T produces [seq_len × seq_len] matrix
  O(n^2) memory for sequence length n

FlashAttention Tiling:
  Tile Q, K, V into blocks that fit in SRAM
  Compute attention incrementally without materializing full matrix

  Tile Address Pattern:
    q_tile_addr = q_base + tile_row × tile_size × d_k
    k_tile_addr = k_base + tile_col × tile_size × d_k

  Online Softmax:
```

```
    Track running max and sum across tiles
    No need to store intermediate attention matrix
```

30.10.9 9.9 Programmable Address Generation Unit (PAGU)

Academic reference for flexible tensor addressing:

PAGU Instruction Set:

```
Operations Supported:
  - Standard Convolution
  - Padded Convolution
  - Strided Convolution
  - Dilated (Atrous) Convolution
  - Pooling (Max, Average)
  - Upsampled Convolution
  - Transposed Convolution

Performance: 1.7 cycles per address for convolution
Area Overhead: ~4.6× vs fixed datapath (justified by flexibility)
```

Configuration Registers:

```
tensor_config_t:
  base_addr[63:0]           // Tensor base address
  dim_sizes[4][31:0]        // Size of each dimension
  dim_strides[4][31:0]      // Stride of each dimension
  kernel_size[15:0]         // Convolution kernel size
  conv_stride[7:0]          // Convolution stride
  dilation[7:0]             // Dilation factor
  padding[7:0]              // Padding amount
  operation_mode[3:0]       // Select operation type
```

Address Generation FSM:

```
always_ff @(posedge clk) begin
    case (state)
        IDLE: begin
            if (start) begin
                dim_counters <= '0;
                state <= GENERATE;
            end
        end

        GENERATE: begin
            // Multi-dimensional address calculation
            addr <= base_addr;
            for (int d = 0; d < num_dims; d++) begin
                case (operation_mode)
```

```
                    CONV_STANDARD:
                        addr <= addr + dim_counters[d] *
dim_strides[d];
                    CONV_STRIDED:
                        addr <= addr + (dim_counters[d] * conv_stride)
* dim_strides[d];
                    CONV_DILATED:
                        addr <= addr + (dim_counters[d] * dilation) *
dim_strides[d];
                endcase
            end

            // Increment counters (innermost first)
            increment_counters();

            if (done) state <= IDLE;
        end
    endcase
end
```

30.11 10. Implementation Comparison Matrix

30.11.1 10.1 Pattern Complexity vs. Capability

Pattern	Descriptor Fields	Address Logic	Hardware Cost	SW Flexibility
Linear	3 (src, dst, len)	Simple add	Low	Limited
2D Stride	6 (+x_len, y_len, strides)	Nested loop	Medium	Good
3D Volume	9 (+z_len, z_strides)	Triple nested	Medium-High	Very Good
Scatter-Gather	4 per fragment	List traversal	Medium	Excellent
Circular	4 (+size, wrap)	Modulo/mask	Low-Medium	Good
Tensor	N (variable stride)	Sum-of-products	High	Excellent

30.11.2 10.2 Use Case Mapping

Application Domain	Primary Pattern	Secondary Pattern
Block memory copy	Linear	-
Network packets	Scatter-Gather	Linear
Audio streaming	Circular	Linear
Image processing	2D Stride	Linear
Video encode/decode	2D Stride	Scatter-Gather
3D graphics	3D Volume	2D Stride
Neural networks	Tensor	2D Stride
Database operations	Scatter-Gather	Linear

30.11.3 10.3 RAPIDS Current vs. Extended Capability

Capability	Current RAPIDS	Industry Standard
Linear contiguous	YES	YES
Unaligned transfers	YES (3-phase)	Varies
Descriptor chaining	YES	YES
2D stride	NO	Common
3D volume	NO	Rare
Scatter-Gather	Partial (chain)	Full SG
Circular wrap	NO	Common
Tensor addressing	NO	Emerging

30.12 11. RAPIDS Extension Recommendations

30.12.1 11.0 PRIORITY 0: Tensor Accelerator Addressing (CRITICAL)

Rationale: Essential for AI/ML workloads. Tensor accelerators represent the highest-growth market for DMA engines. Without tensor addressing, RAPIDS cannot serve neural network inference applications.

Implementation Approach: Data Cube Model (NVDLA-style)

This approach provides the best balance of flexibility and hardware complexity, supporting convolution, pooling, and GEMM operations.

Extended Descriptor Format:

```
typedef struct packed {
    // === Base Fields (existing, 256 bits) ===
    logic [63:0]  src_addr;           // Base source address
    logic [63:0]  dst_addr;           // Base destination address
    logic [31:0]  length;             // For linear mode: beats
    logic [31:0]  next_descriptor_ptr;
    logic [7:0]   desc_priority;
    logic [3:0]   channel_id;
    logic         error;
    logic         last;
    logic         gen_irq;
    logic         valid;

    // === Tensor Extension (repurpose reserved bits + add 2nd
descriptor word) ===
    // Mode selection
    logic [3:0]   transfer_mode;      // 0=linear, 1=2D, 2=3D/tensor,
3=conv, 4=pool

    // Data cube dimensions
    logic [15:0]  width;              // W dimension (elements)
    logic [15:0]  height;             // H dimension (lines)
    logic [15:0]  channels;           // C dimension (surfaces)

    // Stride parameters (32-byte aligned)
    logic [31:0]  line_stride;        // Bytes between lines
    logic [31:0]  surface_stride;     // Bytes between surfaces

    // Convolution parameters (when transfer_mode == CONV)
    logic [7:0]   kernel_width;       // Kw
    logic [7:0]   kernel_height;      // Kh
    logic [7:0]   conv_stride_x;      // Horizontal stride
    logic [7:0]   conv_stride_y;      // Vertical stride
    logic [7:0]   dilation_x;         // Horizontal dilation
    logic [7:0]   dilation_y;         // Vertical dilation
    logic [7:0]   pad_left;           // Left padding
    logic [7:0]   pad_right;          // Right padding
    logic [7:0]   pad_top;            // Top padding
    logic [7:0]   pad_bottom;         // Bottom padding

    // Pooling parameters (when transfer_mode == POOL)
    logic [7:0]   pool_width;
    logic [7:0]   pool_height;
    logic [7:0]   pool_stride_x;
    logic [7:0]   pool_stride_y;
    logic [1:0]   pool_type;          // 0=max, 1=avg, 2=min
```

```
    // Data type and precision
    logic [3:0]   data_type;          // 0=int8, 1=int16, 2=fp16,
3=bf16, 4=fp32
    logic [3:0]   element_size;       // Bytes per element (1, 2, 4)
} descriptor_tensor_t;
```

Address Generation Unit (AGU) Architecture:

```
module tensor_address_generator #(
    parameter int PIPE_STAGES = 3     // Pipeline for multiply-
accumulate
) (
    input  logic        clk,
    input  logic        rst_n,

    // Configuration from descriptor
    input  descriptor_tensor_t desc,
    input  logic        start,

    // Address output stream
    output logic [63:0] addr,
    output logic        addr_valid,
    input  logic        addr_ready,

    // Status
    output logic        done
);

    // Dimension counters
    logic [15:0] w_cnt, h_cnt, c_cnt;
    logic [7:0]  kw_cnt, kh_cnt;       // Kernel position (for conv)

    // Pipeline registers for address calculation
    logic [63:0] addr_stage[PIPE_STAGES];

    // FSM states
    typedef enum logic [2:0] {
        IDLE,
        CALC_BASE,
        CALC_LINE,
        CALC_SURFACE,
        OUTPUT
    } state_t;
    state_t state;

    // Address calculation based on mode
    always_ff @(posedge clk or negedge rst_n) begin
```

```
        if (!rst_n) begin
            state <= IDLE;
            w_cnt <= '0;
            h_cnt <= '0;
            c_cnt <= '0;
        end else begin
            case (state)
                IDLE: begin
                    if (start) begin
                        w_cnt <= '0;
                        h_cnt <= '0;
                        c_cnt <= '0;
                        state <= CALC_BASE;
                    end
                end

                CALC_BASE: begin
                    // Stage 1: Base address
                    addr_stage[0] <= desc.src_addr;
                    state <= CALC_LINE;
                end

                CALC_LINE: begin
                    // Stage 2: Add line offset
                    case (desc.transfer_mode)
                        MODE_LINEAR:
                            addr_stage[1] <= addr_stage[0] + (w_cnt *
desc.element_size);

                        MODE_2D, MODE_3D:
                            addr_stage[1] <= addr_stage[0]
                                           + (h_cnt *
desc.line_stride)
                                           + (w_cnt *
desc.element_size);

                        MODE_CONV:
                            // Convolution: account for stride and
kernel position
                            addr_stage[1] <= addr_stage[0]
                                           + ((h_cnt *
desc.conv_stride_y + kh_cnt * desc.dilation_y) * desc.line_stride)
                                           + ((w_cnt *
desc.conv_stride_x + kw_cnt * desc.dilation_x) * desc.element_size);

                        MODE_POOL:
                            addr_stage[1] <= addr_stage[0]
```

```
                                            + ((h_cnt *
desc.pool_stride_y) * desc.line_stride)
                                            + ((w_cnt *
desc.pool_stride_x) * desc.element_size);
                    endcase
                    state <= CALC_SURFACE;
                end

                CALC_SURFACE: begin
                    // Stage 3: Add surface offset (for 3D/tensor)
                    if (desc.transfer_mode inside {MODE_3D,
MODE_CONV}) begin
                        addr_stage[2] <= addr_stage[1] + (c_cnt *
desc.surface_stride);
                    end else begin
                        addr_stage[2] <= addr_stage[1];
                    end
                    state <= OUTPUT;
                end

                OUTPUT: begin
                    if (addr_ready) begin
                        // Increment counters (innermost first)
                        if (desc.transfer_mode == MODE_CONV) begin
                            // Convolution: iterate kernel, then
spatial, then channels
                            if (kw_cnt < desc.kernel_width - 1) begin
                                kw_cnt <= kw_cnt + 1;
                            end else begin
                                kw_cnt <= '0;
                                if (kh_cnt < desc.kernel_height - 1)
begin
                                    kh_cnt <= kh_cnt + 1;
                                end else begin
                                    kh_cnt <= '0;
                                    // Continue to spatial/channel
iteration
                                    increment_spatial_counters();
                                end
                            end
                        end else begin
                            increment_spatial_counters();
                        end

                        if (transfer_complete())
                            state <= IDLE;
                        else
                            state <= CALC_BASE;
```

```
                    end
                end
            endcase
        end
    end

    assign addr = addr_stage[PIPE_STAGES-1];
    assign addr_valid = (state == OUTPUT);
    assign done = (state == IDLE) && !start;

endmodule
```

Scratchpad Integration (Gemmini-style):

```
// Memory address format for tensor scratchpad
typedef struct packed {
    logic        mem_select;       // 0=scratchpad, 1=accumulator
    logic        acc_mode;         // 0=overwrite, 1=accumulate
    logic        read_format;      // 0=scaled, 1=raw
    logic [28:0] row_addr;         // Row address within memory
} scratchpad_addr_t;

// DMA configuration for tensor loads
typedef struct packed {
    logic [63:0] dram_addr;        // Virtual address in main memory
    scratchpad_addr_t sp_addr;     // Scratchpad destination
    logic [15:0] rows;             // Number of rows to transfer
    logic [15:0] cols;             // Elements per row
    logic [31:0] dram_stride;      // Source memory stride
    logic [15:0] sp_stride;        // Scratchpad row stride (usually
1)
} tensor_dma_config_t;
```

Alignment Requirements:

```
All tensor addresses must satisfy:
  - base_addr aligned to 32 bytes (NVDLA requirement)
  - line_stride aligned to 32 bytes
  - surface_stride aligned to 32 bytes
  - For optimal performance: align to 64 bytes (RAPIDS beat size)

Verification:
  assert((desc.src_addr & 32'h1F) == 0);
  assert((desc.line_stride & 32'h1F) == 0);
  assert((desc.surface_stride & 32'h1F) == 0);
```

Performance Considerations:

```
Address Generation Rate:
  - Target: 1 address per cycle (pipelined)
  - Convolution: 1.7 cycles/address typical (due to kernel iteration)

Memory Bandwidth:
  - TPU-class: 600+ GB/s (HBM)
  - FPGA-class: 20-80 GB/s (DDR4/5)
  - RAPIDS target: Match AXI interface bandwidth

Tiling Support:
  - Hardware computes tile boundaries
  - Double-buffering via ping-pong descriptors
  - Software provides tile dimensions, hardware handles addressing
```

30.12.2 11.1 Priority 1: 2D Stride Support

Rationale: High value for image/video processing with moderate complexity. Also serves as foundation for tensor addressing.

Descriptor Extension:

```
typedef struct packed {
    // Existing fields (256 bits)...

    // 2D extension (96 bits in reserved space)
    logic [31:0] x_length;        // Bytes per row
    logic [15:0] y_length;        // Number of rows
    logic [31:0] src_stride;      // Source row stride
    logic [31:0] dst_stride;      // Destination row stride
    logic        mode_2d;         // Enable 2D mode
} descriptor_2d_t;
```

Address Generation:

```
always_comb begin
    if (mode_2d) begin
        row_src_addr = src_addr + current_row * src_stride;
        row_dst_addr = dst_addr + current_row * dst_stride;
        row_length   = x_length;
    end else begin
        row_src_addr = src_addr;
        row_dst_addr = dst_addr;
        row_length   = length * BYTES_PER_BEAT;
    end
end
```

30.12.3 11.2 Priority 2: Circular Buffer Mode

Rationale: Essential for streaming applications, low complexity addition.

Descriptor Extension:

```
typedef struct packed {
    // Existing fields...

    // Circular buffer extension
    logic [31:0] buffer_size;     // Total buffer size
    logic [31:0] buffer_mask;     // Size-1 for power-of-2
    logic        circular_mode;   // Enable wrap-around
    logic        src_circular;    // Apply to source
    logic        dst_circular;    // Apply to destination
} descriptor_circular_t;
```

Address Generation:

```
function automatic logic [63:0] wrap_address(
    logic [63:0] base,
    logic [63:0] offset,
    logic [31:0] mask,
    logic        enable
);
    if (enable)
        return base + (offset & {32'b0, mask});
    else
        return base + offset;
endfunction
```

30.12.4 11.3 Priority 3: Enhanced Scatter-Gather

Rationale: Full SG capability for network and fragmented memory use cases.

Current Limitation: Chain-only SG requires one descriptor per fragment.

Enhancement: Add fragment list support within single descriptor:

```
typedef struct packed {
    logic [63:0] fragment_list_ptr;  // Pointer to fragment array
    logic [15:0] fragment_count;     // Number of fragments
    logic        sg_mode;            // Enable SG mode

    // Fragment entry format (in memory):
    // [63:0]  address
    // [31:0]  length
    // [31:0]  flags
} descriptor_sg_t;
```

30.13 12. References

30.13.1 Industry Documentation

1. AMD AXI DMA Product Guide (PG021) - Scatter-Gather descriptor format
2. Analog Devices AXI DMAC - 2D transfer implementation
3. Understanding AXI Addressing (ZipCPU) - Burst mode analysis
4. WRAP Address Calculation (Verification Guide) - Wrap burst formulas

30.13.2 GPU and Accelerator Patterns

5. NVIDIA CUDA Memory Coalescing - Coalesced access patterns
6. CUDA Pitch Linear Memory - 2D array allocation
7. NVIDIA TMA Tutorial (CUTLASS) - Tensor Memory Accelerator
8. Efficient GEMM in CUDA (CUTLASS) - Hierarchical tiling

30.13.3 DMA Architecture

10. DMA330 Microcode - Linked-list implementation

30.13.4 Neural Network Accelerators

11. NVDLA Hardware Architecture - Data cube model, CDMA
12. NVDLA In-Memory Data Formats - Line stride, surface stride
13. NVDLA Unit Description - Convolution DMA details
14. Gemmini Accelerator (UC Berkeley) - Open-source systolic array
15. Gemmini Documentation (Chipyard) - DMA and scratchpad
16. Google TPU Architecture - Systolic array overview
17. TPU System Architecture - Memory hierarchy
18. Programmable AGU for DNNs - Flexible address generation

30.13.5 Convolution and Tensor Operations

19. Im2Col Characterization - Convolution memory patterns
20. ST Neural-ART Programming Model - NPU descriptor concepts
21. Systolic Array Dataflows Survey - Weight/input/output stationary
22. FlashAttention Analysis - Attention tiling patterns
23. Depthwise Separable Convolutions - Channel-wise addressing

30.13.6 Sparse Tensor Formats

24. FLAASH Sparse Tensor Accelerator - CSR/CSF formats

25. SparTen CNN Accelerator - Sparse tensor addressing
26. Indirection Stream Architecture - Hardware indirect addressing

---

30.14 Document History

Version	Date	Author	Changes
1.0	2026-01-13	RTL Design Sherpa	Initial comprehensive analysis

This document is part of the RAPIDS (Rapid AXI Programmable In-band Descriptor System) design documentation.